

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности

Работа допущена к защите
руководитель ОП
_____ А.В. Щукин
«_____» _____ 202_ г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
РАБОТА БАКАЛАВРА
РАЗРАБОТКА ПРОГРАММНОГО РЕШЕНИЯ ДЛЯ ПЕРЕВОДА
ЕСТЕСТВЕННОГО ЯЗЫКА В SQL-КОД.

по направлению подготовки 09.03.03 Прикладная информатика

Направленность (профиль) 09.03.03_03 Интеллектуальные инфокоммуникационные технологии

Выполнил
студент гр. 5130903/10302

М.А. Зайцева

Руководитель
старший преподаватель.

Е.Е. Андрианова

Консультант
по нормоконтролю

Е.Е. Андрианова

Санкт-Петербург
2025

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности

УТВЕРЖДАЮ

руководитель ОП

_____ А.В. Щукин

« _____ » _____ 202_ г.

ЗАДАНИЕ
на выполнение выпускной квалификационной работы

студенту Зайцевой Марии Александровне, гр. 5130903/10302

1. Тема работы: Разработка программного решения для перевода естественного языка в SQL-код.
2. Срок сдачи студентом законченной работы: 17.05.2025.
3. Исходные данные по работе: Исходные данные по работе: репозитории с сайта [3.3], набор данных с Hugging Face [3.2], статья [3.1].
 - 3.1. *Arun Solanki A. K. A system to transform natural language queries into SQL queries // International Journal of Information Technology. — 2022. — № 2. — С. 437—446.*
 - 3.2. Hugging Face. Large-scale complex and cross-domain semantic parsing and text-to-SQL dataset. — URL: <https://huggingface.co/datasets/xlangai/spider> (дата обращения: 06.02.2025).
 - 3.3. Сайт UCI Machine Learning Repository. — URL: <http://archive.ics.uci.edu/ml> (дата обращения: 12.01.2025).
4. Содержание работы (перечень подлежащих разработке вопросов):
 - 4.1. Актуальность разработки системы для перевода запросов на естественном языке в SQL-код, цели и задачи работы, ожидаемые результаты.
 - 4.2. Обзор существующих решений, которые предлагают преобразование запросов на естественном языке в SQL-код, выявление сильных и слабых сторон программных решений.

- 4.3. Изучение основ работы с естественным языком и принципы его преобразования в SQL-запросы, изучение методов обработки естественного языка (NLP), а также алгоритмов и моделей, применяемых для таких преобразований.
- 4.4. Практическая реализация и описание процесса разработки программного решения для преобразования запросов на естественном языке в SQL.
- 4.5. Тестирование и апробация разработанного инструмента на реальных примерах. Будут сделаны выводы о точности работы системы и возможных путях для ее улучшения.
5. Перечень графического материала (с указанием обязательных чертежей):
 - 5.1. Структурная схема системы.
 - 5.2. Диаграмма последовательности.
 - 5.3. Диаграмма классов.
6. Перечень используемых информационных технологий, в том числе программное обеспечение, облачные сервисы, базы данных и прочие сквозные цифровые технологии:

Java и Spring Boot (включая Spring Data JPA и Spring Security) для серверной логики, PostgreSQL в качестве СУБД, модели на Python с использованием Keras и TensorFlow для генерации SQL, взаимодействие через REST API с JSON, контейнеризация с Docker и Docker Compose, а также DBeaver для работы с базой данных.
7. Консультанты по работе:
 - 7.1. Старший преподаватель, Е.Е. Андрианова (нормоконтроль).
8. Дата выдачи задания: 22.12.2024.

Руководитель ВКР _____ Е.Е. Андрианова

Задание принял к исполнению 22.12.2024

Студент _____ М.А. Зайцева

РЕФЕРАТ

На 102 с., 75 рис., 0 табл., 2 прил.

Ключевые слова: Перевод естественного языка, трансформер, машинное обучение, веб-приложение

Тема выпускной квалификационной работы: «Разработка программного решения для перевода естественного языка в SQL-код.»

Выпускная квалификационная работа посвящена разработке веб-приложения на базе Java Spring, предназначенного для преобразования текстовых запросов на естественном языке в SQL-запросы. Целью данной работы является создание инструмента, упрощающего взаимодействие с базами данных для пользователей. Объект исследования — информационные технологии и системы, обеспечивающие взаимодействие пользователя и баз данных через естественно-языковой интерфейс. Предмет исследования — архитектура и компоненты программного приложения для автоматизированного перевода запросов с естественного языка в SQL и взаимодействия с пользователем. В ходе работы была реализована монолитная многоуровневая архитектура: фронтенд взаимодействует с backend-приложением, написанным на Spring Boot, которое, в свою очередь, обращается к сервису перевода, использующему модель трансформер для генерации SQL-запросов. В результате была создана система, позволяющая пользователю авторизоваться, подключиться к своей базе данных, ввести запрос на естественном языке, получить соответствующий SQL-запрос, выполнить его и сохранить результат. Разработанное приложение может использоваться в образовательных, исследовательских и бизнес-средах для упрощения работы с базами данных. Выводы показали, что предложенный подход эффективен для начального уровня запросов и может быть расширен за счёт использования более мощных моделей и добавления поддержки сложных SQL-конструкций.

ABSTRACT

102 pages, 75 figures, 0 tables, 2 appendices

Keywords: Natural Language Translation, Transformer, Machine Learning, Web App

The subject of the graduate qualification work is «Title of the thesis».

The final qualifying work is devoted to the development of a web application based on Java Spring, designed to transform text queries in natural language into SQL queries. The goal of this work is to create a tool that simplifies interaction with databases for users. Research object — information technologies and systems, ensuring interaction between the user and databases through a natural language interface. Subject of research — architecture and components of a software application for automatic translation of queries from natural language to SQL and interaction with the user. During the work, a monolithic multi-level architecture was implemented: the frontend interacts with the backend application written in Spring Boot, which, in turn, accesses the Python service using the transformer model to generate SQL queries. As a result, a system was created that allows the user to log in, connect to their database, enter a query in natural language, get the corresponding SQL query, execute it and save the result. The developed application can be used in educational, research and business environments to simplify working with databases. The findings showed that the proposed approach is effective for the initial level of queries and can be expanded by using more powerful models and adding support for complex SQL constructs.

СОДЕРЖАНИЕ

Введение	8
1 Обзор существующих инструментов, решающих проблему перевода текста в SQL код	10
1.1 Dataherald	10
1.2 DB-GBT	12
1.3 SQLthroughAI	14
1.4 Выводы	14
2 Изучение методов, алгоритмов и моделей преобразования естественного языка в SQL	15
2.1 Основы реляционных баз данных и SQL	15
2.2 Обработка естественного языка (NLP)	17
2.3 Модели машинного обучения для NL2SQL	18
2.3.1 LSTM-модели	18
2.3.2 Transformer	21
2.4 Вывод	27
3 Проектирование инструмента для перевода естественного языка в SQL-код	28
3.1 Постановка задачи и формализация требований	28
3.2 Архитектура системы и выбор технологий	30
3.3 Проектирование модели перевода	34
3.3.1 LSTM модель	34
3.3.2 Transformer	36
3.4 Проектирование веб-приложения	37
3.4.1 Проектирование backend-части	37
3.4.2 Проектирование frontend-части	41
3.5 Вывод	43
4 Программирование инструмента для перевода естественного языка в SQL-код	44
4.1 Программирование модели перевода текста в SQL-код	44
4.1.1 LSTM-модель	44
4.1.2 Transformer	53
4.1.3 Сравнение результатов	57
4.2 Программирование приложения для взаимодействия пользователя и модели перевода	57
4.2.1 Модели	57
4.2.2 Контроллеры	59

4.2.3 Репозитории	60
4.2.4 Безопасность	61
4.2.5 Сервисы	63
4.2.6 Пользовательский интерфейс	64
4.3 Вывод	69
5 Тестирование и апробация инструмента для перевода естественного языка в SQL-код	69
5.1 Функциональное тестирование.....	69
5.2 Модульное тестирование	78
5.2.1 Тестирование модуля перевода.....	78
5.2.2 Тестирование репозитория.	81
5.2.3 Тестирование сервисов.....	83
5.2.4 Тестирование контроллеров.....	87
5.3 Интеграционное тестирование	90
5.3.1 Интеграция с базой данных пользователя.....	90
5.3.2 Интеграция с моделью перевода.....	92
5.4 Апробация инструмента для перевода естественного языка в SQL-код	93
5.4.1 Развертывание системы	93
5.4.2 Апробация в моделируемых условиях.....	94
5.5 Вывод	96
Заключение	99
Гипотезы для дальнейших исследований.	101
Список использованных источников.....	102
Приложение 1 Сравнительная таблица	103
Приложение 2 Код проекта	107

ВВЕДЕНИЕ

Программисты и обычные пользователи общаются с базами данных с помощью специальных языков. В наше время часто используются системы поиска информации, которые переводят элементы человеческого языка так, чтобы компьютер мог выполнять задачи. Естественный язык — это то, что люди используют в повседневной жизни. Для большинства пользователей он проще, чем команды для компьютеров. Поэтому сейчас активно ведутся исследования по обработке естественного языка. Но для машин понять естественный язык сложно. Чтобы перевести текст на этом языке в команды для компьютера, используют синтаксический анализ. Понять семантику таких текстов также непросто, потому что одно и то же значение можно выразить разными способами.

Актуальность: навык SQL является одним из самых востребованных среди программистов и специалистов в области IT. Например, на Glassdoor [5] он упоминается в 2291 объявлении о вакансиях, а на Hh.ru [2] в 16845 вакансиях опубликованных за месяц. Это подчеркивает высокий уровень запросов на специалистов с знанием SQL.

Технология перевода естественного языка в SQL набирает популярность. Данные играют ключевую роль в принятии решений, в компаниях огромные объемы данных, но доступ нетехнических специалистов к ним ограничен. Например менеджеры, аналитики и маркетологи могли бы принимать решения быстрее, если бы у них не было потребности обращаться к техническим специалистам для извлечения данных, они могли бы получать данные обращаясь к базам на естественном языке, и при этом получать ответ в одном приложении. Такая технология сэкономила бы время и снижала зависимость от программистов и специалистов по базам данных. Около 70% задач нетехнических специалистов можно решить с помощью простых SQL-запросов, и знание SQL становится важным для повышения их работоспособности.

На рынке потребность таких решений растет вместе с популярностью аналитики самообслуживания. Компаниям важно предоставить своим сотрудникам инструменты, которые упростили бы доступ к данным, особенно это важно для малого и среднего бизнеса, где нет возможности постоянно привлекать специалистов по данным.

Основными заинтересованными лицами в создании такого инструмента являются:

1. Малые и средние бизнес-компании для повышения эффективности работы сотрудников и ускорения процессов принятия решений.
2. Разработчики, которые могут делегировать рутинные задачи извлечения данных такому инструменту.
3. Исследовательские институты, так как такая технология поможет им в автоматизации исследовательских процессов.
4. Конечные пользователи, которые хотят получить доступ к данным без изучения дополнительных технологий.

Объект исследования — информационные технологии и системы, обеспечивающие взаимодействие пользователя и баз данных через естественно-языковой интерфейс.

Предмет исследования — архитектура и компоненты программного приложения для автоматизированного перевода запросов с естественного языка в SQL и взаимодействия с пользователем.

Основная цель данной работы — создать инструмент, который переводит запросы на русском языке в SQL без необходимости знания программирования. Инструмент должен генерировать корректные SQL-запросы и ускорять получение данных нетехническими специалистами.

Задачи работы:

1. Изучить существующие инструменты перевода естественного языка в SQL, выявить их ограничения и недостатки.
2. Сформулировать ключевые требования к инструменту для перевода естественного языка в SQL-код.
3. Спроектировать инструмент для перевода текста в SQL-код.
4. Реализовать инструмент для перевода текста в SQL-код.
5. Провести тестирование и апробацию данного инструмента.
6. Сделать выводы о проделанной работе.

Практическая ценность данной работы заключается в том, что разработанный инструмент облегчит работу с базами данными, при этом знание языка запросов к базам данных не обязателен. Данный продукт уменьшит временные затраты на составление SQL запросов и количество обращений к техническим специалистам. Решение так же подойдет для школ и вузов, чтобы обучать студентов SQL. Работа так же может стать базой для создания новых умных систем для взаимодействия пользователей с базами данных, результаты данной работы могут стать основой для исследований в области обработки естественного языка.

1 ОБЗОР СУЩЕСТВУЮЩИХ ИНСТРУМЕНТОВ, РЕШАЮЩИХ ПРОБЛЕМУ ПЕРЕВОДА ТЕКСТА В SQL КОД

Перед тем, как приступить к созданию своего инструмента по переводу естественного языка в SQL-код, необходимо проанализировать аналогичные решения, для того чтобы понять какие подходы к переводу естественного языка в SQL-код применяются в современных инструментах. Такой анализ покажет уровень развития технологий в данной области и распространенность различных подходов.

Особое внимание необходимо уделить архитектуре систем, способам представления данных, интерфейсу. Я рассмотрю как коммерческие продукты так и открытые, чтобы понять, какие функции являются стандартом и какие решения стали популярны в бизнесе и образовании.

1.1 Dataherald

Dataherald — это движок для преобразования естественного языка в SQL, созданный для ответов на вопросы на уровне предприятия по структурированным данным. Он позволяет пользователю настроить API из его базы данных, который может отвечать на вопросы на простом английском языке. [3].

С текущей версией Dataherald можно работать с PostgreSQL, Databricks, BigQuery и Snowflake и настроить API для взаимодействия с данными.

Dataherald предлагает RESTful API который можно использовать для:

- Подключения и управления подключениями к базам данных.
- Добавление контекста в движок посредством сканирования баз данных, добавления инструкций на уровне базы данных, добавления описаний в таблицы и столбцы.
- Задания вопросов на естественном языке по данным в реляционных базах данных.

Dataherald построен на модульной архитектуре и предоставляет стандартизированные, расширяемые интерфейсы для замены любого из этих модулей собственной реализацией.

1. Context Store.

В хранилище контекста сохраняется информация, важная для понимания запроса системой. Сохраняются описания таблиц, столбцов, связей, при-

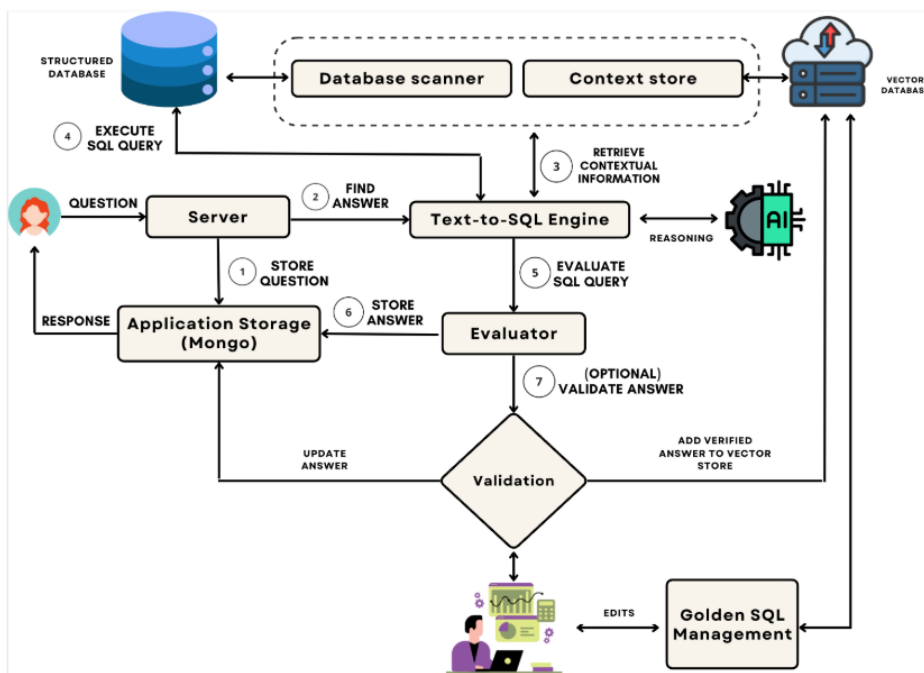


Рис.1.1. Архитектура Dataherald [3].

меры переводов. Благодаря этому модулю система обрабатывает запросы быстрее и точнее, так как опирается на уже существующий контекст [3].

2. Text-to-SQL Engine. Это основной модуль, который отвечает за перевод текста в SQL-код. В его основе лежит большая языковая модель, обученная на текстах и запросах, способная понимать смысл фраз и формировать корректные SQL-запросы. Она использует архитектуру трансформеров, которая хорошо справляется с обработкой последовательностей слов. [3].

3. API Server

Система предоставляет удобные REST API для взаимодействия с базами данных. Через API можно подключаться к базе данных, отправлять текстовые запросы и получать ответ в формате SQL-кода. REST API использует стандартные HTTP-методы, это упрощает интеграцию с внешними системами.

4. Evaluator.

После того как SQL-запрос сгенерирован, модуль оценивания проверяет его корректность, запросу присваивается оценка уверенности, это помогает определить насколько сгенерированный запрос соответствует требованию пользователя. Данный модуль основан на LLM [3].

5. Vector Store.

Это модуль в котором сохраняются векторные представления примеров из предыдущих пар текстов и запросов, которые были помечены как

верные. Разработчик может добавить поддержку собственного хранилища, реализовав специальный класс.

6. Database Integration. Этот компонент отвечает за установку и управление соединениями с различными СУБД. Поддерживаются популярные системы, такие как PostgreSQL, MySQL и другие, что делает систему универсальной.
7. Finetuning. Данный модуль позволяет дообучать модель на конкретных примерах для нужной базы данных. Это позволяет адаптировать систему под конкретные задачи и улучшить качество генерации [3] .

Система построена на основе этих модулей, и каждый из них может быть настроен и заменён на собственную реализацию через файл `.env`.

1.2 DB-GBT

DB-GPT — это фреймворк для разработки приложений с открытым исходным кодом для работы с данными на базе искусственного интеллекта. Она включает AWEL (Agentic Workflow Expression Language) и мультиагентные системы, чтобы упростить работу с большими языковыми моделями.

Агенты в контексте больших языковых моделей — это специальные программные сущности, которые используя мощности языковых моделей, могут принимать решения и выполнять задачи.

AWEL - это набор языков выражений рабочих процессов интеллектуальных агентов, специально разработанных для разработки приложений с большими моделями. Он обеспечивает большую функциональность и гибкость. Он позволяет не углубляться в детали , а сосредоточиться на бизнес-логике.

AWEL разделен на три уровня:

1. Слой операторов включает базовые операции в процессе разработки приложений с LLM. Например, извлечение, векторизация, взаимодействие с моделью, обработка подсказок. В последующей разработке фреймворк будет еще больше абстрагироваться и стандартизировать дизайн операторов [4].
2. Слой AgentFream далее инкапсулирует операторов и может выполнять цепные вычисления на основе операторов. Он поддерживает распределенные вычисления, поддерживая набор операций цепных вычислений, таких как фильтр, соединение, отображение, сокращение [4].

3. Слой DSL предоставляет структурированный язык, который управляет операциями на всех предыдущих уровнях. Это упрощает программирование, так как работа идет не на естественном языке, а на понятном платформе DSL [4].

Язык AWEL создан для разработки приложений на основе больших языковых моделей. В архитектуре DB-GPT агенты рассматриваются как объекты первого класса, и они зависят от таких ресурсов, как RAG и источники данных.

Архитектура DB-GPT включает несколько ключевых элементов, которые обеспечивают гибкость и масштабируемость.

1. Уровень данных нужен для обеспечения взаимодействия с источниками информации, например базы данных или файлы. DB-GPT может работать с PostgreSQL и MySQL, а также поддерживает Excel и CSV [4].
2. SMMF необходим для управления несколькими большими языковыми моделями через единый интерфейс. Система имеет поддержку моделей LLaMA и ChatGLM, это позволяет пользователю выбирать и комбинировать модели для лучшего результата.
3. RAG (Retrieval-Augmented Generation) — это фреймворк, который добавляет к запросам информацию из базы знаний. Это позволяет улучшать генерацию и отвечать даже на сложные запросы [4] .
4. AWEL (Agentic Workflow Expression Language) - это язык, который автоматизирует процессы и создает последовательности выполнения задач.
5. Система мультиагентов необходима для разделения задач между агентами, каждый из которых имеет свою зону ответственности. Агенты взаимодействуют друг с другом через стандартный протокол.
6. Модуль донастройки необходим для того чтобы языковые модели адаптировались под конкретный контекст, это позволяет эффективно обучать модели на пользовательских данных. [4] .
7. Модуль приватности необходим для обеспечения защиты, он позволяет разворачивать модели локально, без передачи данных в облако.

Взаимодействие компонентов:

- Пользователь вводит свой запрос на естественном языке.
- Система проводит анализ запроса и при необходимости обращается к базе для извлечения данных.

- С помощью Retrieval-Augmented Generation и векторного поиска система дополняет запрос релевантной информацией система добавляет к запросу контекст.
- Мультиагенты выполняют поставленные задачи, взаимодействуя с базами данных, моделями и другими источниками.
- Результат возвращается пользователю в удобном формате.

1.3 SQLthroughAI

SQLthroughAI - это платформа, которая позволяет пользователю создавать SQL-запросы, на основе их текстового описания. Платформа имеет пользовательский web-интерфейс и поддерживает разные диалекты SQL, включая MySQL и MongoDB. Главное преимущество данного инструмента по сравнению с предыдущими заключается в том, что пользователь взаимодействует с системой только через визуальный интерфейс, ему не нужно знать языки программирования для работы с ним [7].

SQLthroughAI предлагает добавление информации о базе данных через загрузку файлов или добавления столбцов вручную. У него простой для понимания пользователем интерфейс, но нет детальной настройки функций. Пользовательская подписка стоит около \$181, и есть бесплатная пробная версия [7].

SQLthroughAI основан на технологиях Open AI, но на сайте почти нет информации о том, как была улучшена модель.

1.4 Выводы

На основе исследования можно выделить несколько ключевых моментов. Перевод естественного языка в SQL — это важная область в технологиях обработки данных, которая делает работу с базами данных проще для пользователей. Уже есть много решений, которые показывают хороший прогресс в этой сфере.

Проанализировав несколько инструментов по переводу естественного языка была создана сравнительная таблица табл.П1.1.

Современные инструменты для такого перевода используют сложные технологии, как глубокое обучение и обработка естественного языка. Создание таких моделей требует много вычислительных ресурсов. Обычно обучение идет на мощ-

ных видеокартах или специализированных системах, что позволяет обрабатывать большие объемы данных быстро.

Качество и объем обучающего набора данных играют важную роль в создании таких систем. Чтобы достичь хорошей точности перевода, модели должны обучаться на разнообразных данных с разными формулировками на обычном языке и их SQL-эквивалентами. Подготовка таких данных непростая, и в ней участвуют эксперты, которые следят за точностью разметки.

Из этого следует, что создание инструмента для перевода естественного языка в SQL-код - сложная задача, которая требует значительных вложений в технологии, данные и инфраструктуру.

Несмотря на то, что инструментов для преобразования языка достаточно, большинство из них предназначено для специалистов с навыками программирования. Кроме этого большинство решений создано для английского языка, языковые модели, используемые в решениях обучены на иностранных языках, это создает трудности для русскоязычных пользователей. В итоге многим сотрудникам приходится учить SQL или обращаться к техническим специалистам, что замедляет рабочие процессы.

С учетом всех этих проблем, есть потребность в инструменте, который позволит писать SQL-запросы на обычном языке, не требуя глубоких технических знаний. Такой инструмент должен быть простым в использовании, поддерживать русский язык и правильно генерировать SQL-запросы. Это поможет ускорить работу специалистов и снизить зависимость бизнеса от IT-команд при выполнении стандартных рутинных операций.

2 ИЗУЧЕНИЕ МЕТОДОВ, АЛГОРИТМОВ И МОДЕЛЕЙ ПРЕОБРАЗОВАНИЯ ЕСТЕСТВЕННОГО ЯЗЫКА В SQL

2.1 Основы реляционных баз данных и SQL

Реляционные базы данных представляют собой системы управления данными, которые хранят информацию в виде таблиц. Каждая таблица состоит из строк и столбцов, где строки представляют собой записи (кортежи), а столбцы — атрибуты (поля) этих записей. Основные характеристики реляционных баз данных включают:

- Структурированность. Данные организованы в таблицы с четко определенными схемами.
- Отношения. Таблицы могут быть связаны друг с другом через ключи. Первичный ключ уникально идентифицирует каждую запись в таблице, а внешний ключ используется для создания связи между таблицами.
- SQL. Стандартный язык запросов для работы с реляционными базами данных — SQL (Structured Query Language). С его помощью можно выполнять операции выборки, вставки, обновления и удаления данных [6].

Примеры типичных пользовательских запросов Пользователи часто формулируют запросы на естественном языке, которые затем необходимо преобразовать в SQL-запросы.

1. "Покажи мне всех клиентов из Москвы."

SQL-эквивалент: `SELECT * FROM clients WHERE city = 'Москва';`

2. "Сколько заказов было сделано в прошлом месяце?"[3].

SQL-эквивалент: `SELECT COUNT(*) FROM orders WHERE order_date BETWEEN '2025-03-01' AND '2025-03-31';`

3. "Выведи имена сотрудников и их зарплаты."

SQL-эквивалент: `SELECT name, salary FROM employees;`

Сопоставление запросов на естественном языке и его SQL эквивалентом это сложная задача потому что:

1. Естественный язык может быть неоднозначным. Одно и тоже предложение можно интерпретировать по-разному.
2. Запросы могут зависеть от контекста и предыдущих взаимодействий с системой.
3. Пользователь может не знать структуру данных, точные названия таблиц, столбцов, это затрудняет формулировку запроса.
4. Запросы могут содержать сложные конструкции, которые требуют более тщательной обработки.

Для решения данных проблем ведутся исследования и разработки в области обработки естественного языка, которые помогают компьютерным системам интерпретировать естественный язык человека.

2.2 Обработка естественного языка (NLP)

Обработка естественного языка (Natural Language Processing, NLP) состоит из множества этапов, каждый из которых имеет свою роль в задаче анализа текстовой информации. Рассмотрим подробнее основные этапы обработки текстов связанные с задачей перевода текста в SQL-код.

Токенизация

Токенизация – это процесс разбиения фразы, предложения, абзаца или всего текстового документа на более мелкие единицы, например, отдельные слова или термины. Каждое из этих меньших подразделений называется токенами. Например, предложение "Я люблю программирование." будет разбито на токены: ["Я" "люблю" "программирование ".""] [14].

Токенизация может быть выполнена различными способами:

- По пробелам. Это самый простой метод, который разделяет текст на токены по пробелам.
- С учетом пунктуации. Это более сложный метод, который учитывает знаки пунктуации.
- С использованием регулярных выражений. Данный тип позволяет более гибко определять правила токенизации.

Лемматизация

Обычно тексты содержат разные грамматические формы одного и того же слова, а также могут встречаться однокоренные слова. Лемматизация преследует цель привести все встречающиеся словоформы к одной, нормальной словарной форме

Лемматизация — это процесс приведения слов к их базовой форме или лемме. Все слова приводятся к начальной форме. Лемматизация используется для улучшения качества анализа, уменьшая количество уникальных слов.

Удаление стоп-слов

Стоп-слова – это слова, которые не несут значимой информации для анализа текста. Когда мы применяем машинное обучение к текстам, такие слова могут добавить много шума, поэтому необходимо избавляться от нерелевантных слов.

Стоп-слова это обычно понимают артикли, междометия, союзы и т.д., которые не несут смысловой нагрузки. При этом надо понимать, что не существует универсального списка стоп-слов, все зависит от конкретного случая.

Распознавание сущностей (NER)

Распознавание именованных сущностей (Named Entity Recognition) — это задача выделения и классификации ключевых элементов текста на категории, такие как имена людей, организации, даты и местоположения. Это важный шаг для извлечения структурированной информации из неструктурированных данных. Методы NER могут варьироваться от простых правил до сложных моделей машинного обучения. Современные подходы часто используют нейронные сети и трансформеры для повышения точности распознавания [12].

Синтаксический анализ

Синтаксический анализ включает в себя разбор структуры предложения для определения грамматических отношений между словами. Это может быть выполнено с помощью деревьев зависимостей или синтаксических деревьев. Синтаксический анализ помогает понять структуру предложения и выявить отношения между его компонентами [12].

Семантический анализ

Семантический анализ направлен на понимание смысла текста. Он включает в себя задачи такие как определение семантических ролей слов (кто делает что), разрешение амфиболий (например, когда одно слово может иметь несколько значений) и построение семантических сетей [12].

2.3 Модели машинного обучения для NL2SQL

2.3.1 LSTM-модели

Рекуррентные нейронные сети (Recurrent Neural Networks, RNN). Это сети, содержащие обратные связи и позволяющие сохранять информацию. Рекуррентная нейронная сеть (англ. Recurrent Neural Network) использует циклы обратной связи, что отличает эту модель от других архитектур нейронных сетей. Циклы помогают RNN обрабатывать последовательность данных и позволяют обмениваться данными с разными узлами, выполняя прогнозы в соответствии с собранной информацией [13].

Сети LSTM представляют собой особый вид RNN. Они были введены для того, чтобы избежать так называемой проблемы долговременной зависимости. В обычной RNN такого рода проблемы часто возникают при соединении ранее полученной информации с новой. Чтобы запомнить информацию в течение длительного времени в поведении LSTM по умолчанию, сети LSTM имеют структуру,

аналогичную RNN, но модуль памяти, или модуль повторений, имеет другую структуру [13].

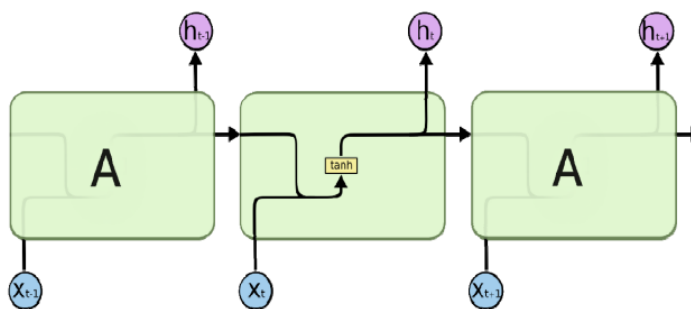


Рис.2.1. Модуль памяти в обычной RNN, содержащий один слой [10].

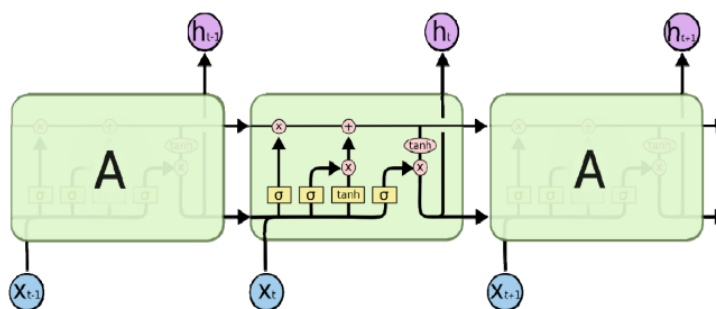


Рис.2.2. Модуль памяти в LSTM, содержащий четыре взаимосвязанных слоя [10].

На рисунке рис.2.2: приведена однонаправленная сеть LSTM, в которой сеть хранит только прямую информацию. Как следует из рисунков рис.2.1: и рис.2.2:, каждая линия переносит весь вектор от выхода узла до входа следующего узла, точечные операции – это математические операции над векторами, сливающаяся линия обеспечивает конкатенацию векторов, а расходящиеся линии посылают копии информации в различные узлы. Горизонтальная линия, проходящая через верхнюю часть модуля памяти, является конвейером данных; информация проходит через нижние части модуля. Благодаря такой структуре сети LSTM могут удалять или добавлять информацию. Некоторые параметры функции активации также присутствуют в LSTM.

x_t – входной вектор;

h_{t-1} – выход предыдущего модуля;

σ , $\tanh$ – сигмоидальная функция и гиперболический тангенс;

Первый слой блока рис.2.3, так называемый «слой забывания», решает, какую информацию из предыдущего блока нужно удалить. Пусть C_{t-1} — память, передающаяся из предыдущего блока. Эта память для каждого блока C_t называется состоянием. Слой вернет значения, находящиеся в диапазоне от 0 до 1, как показано в формуле (2.1).

$$f_t = \sigma(W_t \cdot [h_{t-1}, x_t] + b_f) \quad (2.1)$$

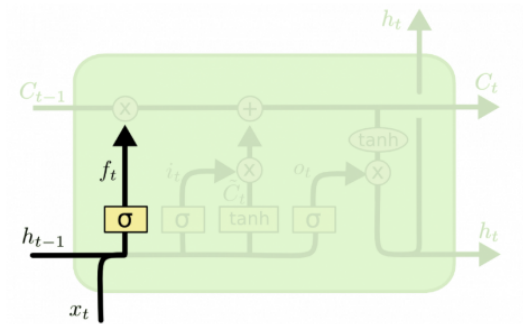


Рис.2.3. Вентиль забывания [10].

Следующий слой отвечает за то, какая память будет храниться в блоке. Он состоит из двух частей. Первая, называемая слоем входного фильтра, определяет, какие значения обновляются (2.2).

$$i_t = \sigma(W_t \cdot [h_{t-1}, x_t] + b_i) \quad (2.2)$$

Следующий слой на рис.2.4 строит вектор значений-кандидатов на добавление в состояние блока (2.3).

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (2.3)$$

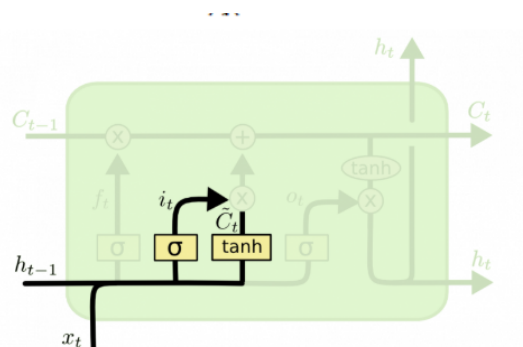


Рис.2.4. Вентиль входного состояния [10].

Тогда состояние блока C_t на рис.2.5 вычисляется как (2.4).

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \quad (2.4)$$

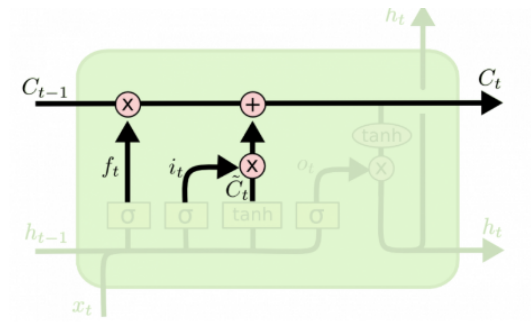


Рис.2.5. Обновление состояния блока [10].

Последний слой на рис.2.6 определяет, какая информация будет выходной – (2.5) и (2.6).

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (2.5)$$

$$h_t = o_t * \tanh(C_t) \quad (2.6)$$

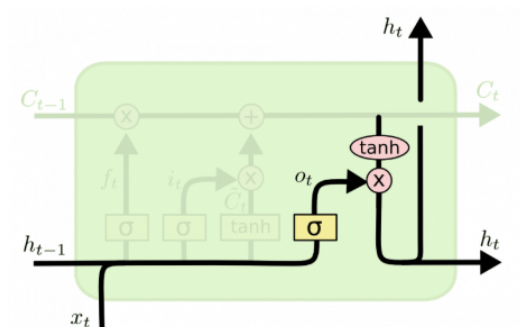


Рис.2.6. Вентиль выходного состояния [10].

Простейшая LSTM называется базовой или «ванильной» (англ. Vanilla LSTM). Эта сеть состоит также из выходного слоя для получения отклика модели и так называемого слоя дропаута. Слои дропаута используются для снижения переобучения. Коэффициент соответствует долей связей между слоями, которые вырождаются в ноль и не будут меняться в процессе обучения.

2.3.2 Transformer

Эта архитектура состоит главным образом из кодирующего и декодирующего модуля.

Кодирующий и декодирующий модули представляют из себя последовательности повторяющихся кодировщиков и декодировщиков, причем количество кодировщиков и декодировщиков всегда равно. В последствии выходы кодирую-

щего модуля передаются всем декодировщикам. На рис.2.7 изображена структура кодирующих и декодирующих модулей.

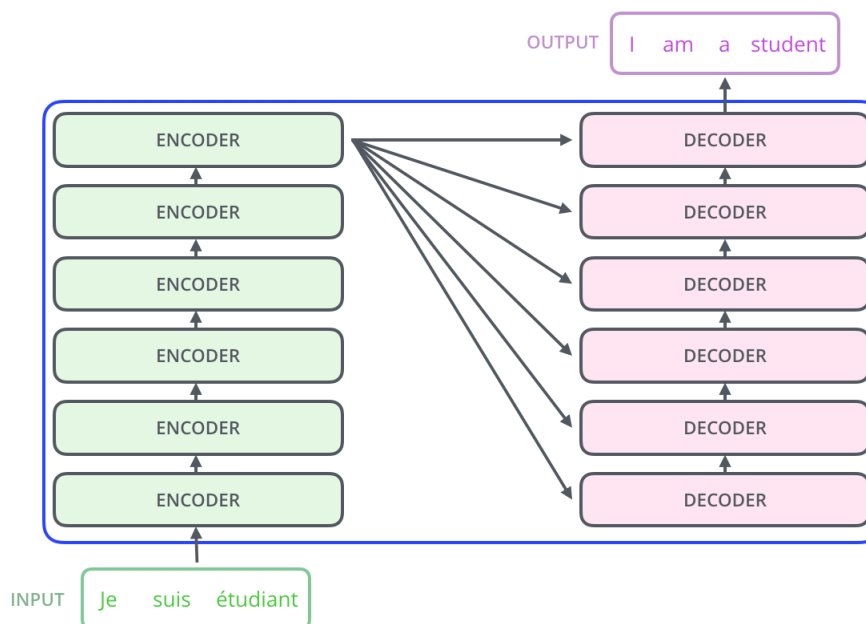


Рис.2.7. Общий вид кодирующего и декодирующего модуля [8].

Кодировщик состоит из двух частей: механизма самовнимания и линейного слоя. Декодировщик состоит из трех частей: механизма самовнимания, механизма внимания, который на вход дополнительно получает выходы кодирующего модуля и линейного слоя. Также входы и выходы каждого слоя складываются между собой и нормализуются. На рис.2.8 изображена структура кодировщика и декодировщика [1].

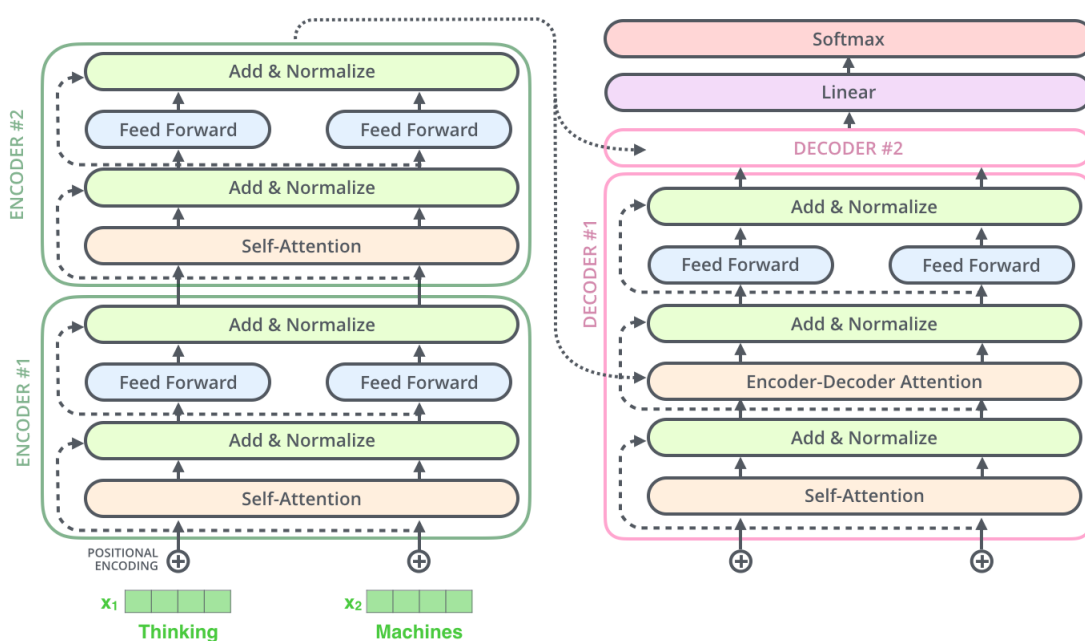


Рис.2.8. Внутренняя структура кодировщика и декодировщика [8].

Рассмотрим некоторые компоненты подробнее. Механизмы внимания

В первую очередь для каждого входного вектора создаются три вектора: вектор запроса, вектор ключа и вектор значения. Они вычисляются по формулам (2.7), (2.8) и (2.9) соответственно [1].

$$q_i = x_i \times W_q \quad (2.7)$$

$$k_i = x_i \times W_k \quad (2.8)$$

$$v_i = x_i \times W_v \quad (2.9)$$

W_q, W_k, W_v – матрицы весов;

q_i, k_i, v_i – векторы запроса, ключа и значения;

x_i – входной вектор;

Вторым шагом является подсчет внутренних коэффициентов внимания. Данные коэффициенты показывают степень связи входа i со входом j . Они вычисляются по формуле (2.10) [1].

$$ij = (q_i; k_j) \quad (2.10)$$

$(a; b)$ – скалярное произведение векторов a и b .

На выходе получается N векторов c_i размерности N , где N — это количество входных векторов, а c_i — это вектор коэффициентов для входа i . Далее необходимо нормализовать каждый вектор по формуле (2.11).

$$sc_i = \text{Softmax}\left(\frac{c_i}{\sqrt{d_k}}\right) \quad (2.11)$$

d_k – размерность вектора ключа; sc_i – нормализованный вектор коэффициентов;

Выходные векторы механизма внимания считаются по формуле (2.12).

$$z_i = \sum_{j=1}^N sc_{ij} \cdot v_j \quad (2.12)$$

Если представлять набор входных векторов как матрицу, то вычисления сводятся к нескольким простым формулам. Изначально вычисляются матрицы запроса, ключа и значения по формулам (2.13), (2.14) и (2.15) соответственно [1].

$$Q = X \times W_q \quad (2.13)$$

$$K = X \times W_k \quad (2.14)$$

$$V = X \times W_v \quad (2.15)$$

X – матрица входных векторов;

Матрица выходов, в свою очередь, считаются по формуле (2.16).

$$Z = Softmax\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) \times V \quad (2.16)$$

где Z – матрица выходных векторов;

Механизм множественного внимания

Множественное внимание представляет из себя несколько параллельных механизмов внимания, которые имеют свои собственные веса. Это позволяет определять больше различных связей между элементами входных данных, тем самым более точно решать поставленную задачу. В механизме множественного внимания каждый отдельный механизм внимания называют «головами». Принцип работы данного механизма заключается в том, что один и тот же вход поступает на каждую голову и выходы считаются параллельно. Далее выходы из каждой головы конкатенируются, а затем умножаются на выходную матрицу, полученную во время обучения [1]. Это позволяет объединить информацию, полученную из всех голов, и нормализовать размерность выхода. На рис.2.9 изображен весь процесс работы механизма множественного внимания. Именно множественное внимание чаще всего встречается в моделях, построенных на архитектуре Transformer.

Суммирование и нормализация

Данный слой складывает матрицы входов и выходов предыдущего слоя, а потом применяет к полученному вектору функцию LayerNorm. На рис.2.10 изображен данный слой подробнее. Рассмотрим подробнее функцию LayerNorm. Обозначим сумму матриц выходов и входов за Y . Данная функция выполняется последовательно для всех векторов Y_i в матрице Y . В первую очередь вычисляются средний коэффициент μ_i и дисперсия σ_i^2 по формулам (2.17) и (2.18) соответственно.

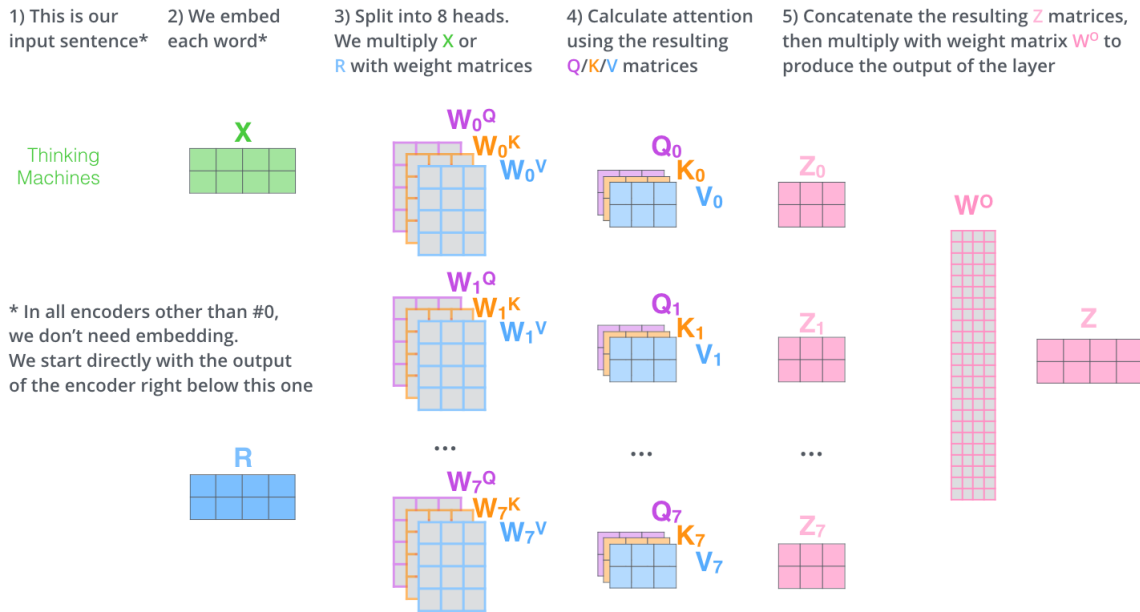


Рис.2.9. Процесс механизма множественного внимания [8].

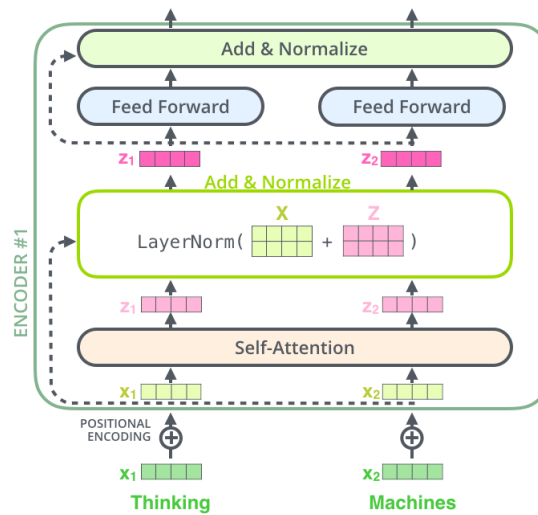


Рис.2.10. Процесс суммы и нормализации [8].

$$\mu_i = \frac{1}{K} \sum_{k=1}^K Y_{ik} \quad (2.17)$$

$$\sigma_i^2 = \frac{1}{K} \sum_{k=1}^K (Y_{ik} - \mu_i)^2 \quad (2.18)$$

где K – размерность вектора Y_i ;

Далее происходит сама нормализация вектора Y_i . Она вычисляется по формуле (2.19). ε необходимо, чтобы не произошло деление на ноль.

$$\hat{Y}_i = \frac{Y_i - \mu_i}{\sqrt{\sigma_i^2 + \varepsilon}} \quad (2.19)$$

где ε – достаточно малое ненулевое значение;

$\hat{Y}_i$ – нормализованный вектор Y_i ;

В конце нормализованный вектор масштабируется, сдвигается и передается в следующий слой. Векторы масштабирования и сдвига являются параметрами модели, которые были получены во время обучения. Выходной вектор вычисляется по формуле (2.20) [1].

$$N_i = \gamma \circ \hat{Y}_i + \beta \quad (2.20)$$

где γ – вектор масштабирования;

β – вектор сдвига;

$\circ$ – произведение Адамара (поэлементарное произведение);

Квантизация

Квантизация — это метод оптимизации модели нейронной сети, при котором параметры модели преобразуются из типа Float к типам Int8 или UInt8. Данный метод позволяет ускорить запуск модели и уменьшить размер модели при незначительной потере в точности. Ускорение достигается за счёт того, что операции сложения и умножения для целочисленных типов выполняются быстрее, чем для типов с плавающей запятой. Оптимизация размера модели достигается за счёт того, что типы Int8 и UInt8 занимают 1 байт памяти, в то время как Float занимает 4 байта памяти. Процесс квантизации числа $x \in [\alpha; \beta]$ в целое число $x_q \in [\alpha_q; \beta_q]$ описывается формулами (2.21) и (2.22).

$$x_q = clip(round(\frac{1}{8}x + z), \alpha_q, \beta_q) \quad (2.21)$$

$$clip(x, l, u) = \begin{cases} l, & x < l \\ x, & l \leq x \leq u \\ u, & x > u \end{cases} \quad (2.22)$$

Обратный процесс описывается формулой (2.23).

$$x = s(x_q - z) \quad (2.23)$$

Параметры s и z являются дополнительными параметрами квантизации и вычисляются отдельно для каждой матрицы. Они вычисляются по формулам (2.24) и (2.25) соответственно.

$$s = \frac{\beta - \alpha}{\beta_q - \alpha_q} \quad (2.24)$$

$$z = \text{round}\left(\frac{\beta\alpha_q - \alpha\beta_q}{\beta - \alpha}\right) \quad (2.25)$$

Стандартно параметры α и β определяют как максимальное и минимальное значения в квантизуемой матрице, а α_q и β_q – как максимальное и минимальное значения целочисленной переменной.

2.4 Вывод

Модели LSTM и трансформеры представляют собой два ключевых архитектурных подхода к решению задач обработки естественного языка, каждый из которых обладает своими особенностями, преимуществами и ограничениями. Архитектура LSTM, основанная на механизме рекуррентных связей и управляемой передачей информации через память, хорошо справляется с задачами, требующими учета последовательного контекста. Её основное преимущество заключается в способности сохранять информацию о предыдущих элементах последовательности на длительных интервалах, что критически важно при работе с языковыми конструкциями, где смысл текущего слова зависит от предшествующих. Однако LSTM-модели ограничены в масштабировании, плохо обрабатывают длинные входы и имеют проблемы с параллелизацией, что увеличивает время обучения. В отличие от них, трансформерные архитектуры, основанные на механизме самовнимания (self-attention), обеспечивают эффективную обработку всей последовательности целиком, выявляя связи между словами вне зависимости от их положения. Это делает трансформеры значительно более производительными при работе с длинными текстами и позволяет параллелизировать вычисления на всех

этапах, что критично при обучении на больших объемах данных. Такие модели, как BERT, GPT, T5 и другие, продемонстрировали превосходные результаты во множестве задач, включая машинный перевод, генерацию текста и извлечение информации. Однако они требуют существенно больше вычислительных ресурсов и объемов данных для обучения, чем LSTM. В задаче перевода естественного языка в SQL-запросы обе архитектуры могут быть успешно применены. LSTM-модели используются в классических sequence-to-sequence схемах, где один модуль кодирует входную фразу, а другой — поэтапно генерирует SQL-запрос. Этот подход относительно прост в реализации и позволяет достичь приемлемой точности при наличии хорошо размеченного обучающего корпуса. Трансформеры, в свою очередь, позволяют более гибко интерпретировать сложные и разнородные запросы, учитывая не только синтаксические, но и семантические связи между словами, что особенно важно в контексте многоуровневых SQL-запросов с вложенными условиями.

3 ПРОЕКТИРОВАНИЕ ИНСТРУМЕНТА ДЛЯ ПЕРЕВОДА ЕСТЕСТВЕННОГО ЯЗЫКА В SQL-КОД

3.1 Постановка задачи и формализация требований

Прежде чем приступить к реализации, необходимо чётко определить цели и требования к разрабатываемому инструменту. На данном этапе проводится анализ типов пользовательских запросов, определяются ограничения на язык запросов и формулируется основная задача: построение системы, способной автоматически переводить фразы на естественном языке в корректные SQL-запросы. Целью данного проекта является разработка интеллектуального программного инструмента, способного автоматически преобразовывать текстовые запросы на естественном языке в корректные SQL-запросы.

Требованию к продукту в формате User Story:

- Регистрация пользователя

Как незарегистрированный пользователь, я хочу иметь возможность зарегистрироваться, чтобы получить доступ к функциональности системы.

- Авторизация пользователя

Как зарегистрированный пользователь, я хочу иметь возможность войти в систему с помощью логина и пароля, чтобы использовать инструмент.

– Подключение к базе данных

Как авторизованный пользователь, я хочу иметь возможность ввести данные для подключения к моей локальной базе данных, чтобы использовать ее структуру для генерации и выполнения запросов.

– Ввод текстового запроса

Как авторизованный пользователь, я хочу иметь возможности ввести запрос на естественном языке, чтобы в дальнейшем получить его в формате SQL.

– Генерация SQL-запроса

Как авторизованный пользователь, я хочу получить сгенерированный SQL-запрос на основе введенного текста, чтобы в дальнейшем использовать его в своей базе данных.

– Редактирование SQL-запроса

Как авторизованный пользователь, я хочу иметь возможность вручную отредактировать полученный SQL-запрос, чтобы уточнить его перед выполнением.

– Выполнение запроса и просмотр результата

Как авторизованный пользователь, я хочу отправить SQL-запрос в свою базу данных и получить ответ в формате таблицы на экране, чтобы сразу увидеть результат.

– История запросов

Как авторизованный пользователь, я хочу иметь возможность просмотра истории моих запросов, чтобы при необходимости использовать их повторно.

Для удовлетворения данных требований необходимо решить следующие задачи:

1. Разработка системы регистрации и авторизации пользователей

- Реализовать формы регистрации и входа.
- Настроить безопасное хранение данных пользователей.
- Обеспечить разграничение доступа на уровне сессий.

2. Создание интерфейса для подключения к базе данных пользователя

- Реализовать форму ввода параметров подключения (тип БД, хост, порт, имя пользователя, пароль, имя БД).

- Проверять валидность подключения и выводить сообщения об ошибках.
- 3. Разработка модуля обработки естественного языка
 - Разработать или интегрировать модель LSTM и/или трансформер для перевода текста в SQL.
 - Обучить модель на соответствующем датасете.
 - Обеспечить взаимодействие модуля с backend частью приложения.
- 4. Реализация функции редактирования SQL-запроса
 - Предоставить пользователю возможность просматривать и редактировать сгенерированный SQL перед выполнением.
- 5. Выполнение SQL-запросов на подключённой базе данных
 - Обеспечить отправку отредактированного SQL-запроса в базу данных.
 - Отобразить полученные результаты в табличном виде.
- 6. Сохранение истории запросов
 - Реализовать структуру хранения запросов и ответов (естественный запрос, SQL, результат, дата).
 - Обеспечить доступ к истории через личный кабинет пользователя.
- 7. Интеграция всех компонентов в единое веб-приложение
 - Настроить взаимодействие backend и frontend .
 - Обеспечить взаимодействие backend и модели перевода.
 - Обеспечить пользовательский интерфейс.
- 8. Тестирование функционала
 - Провести модульное и интеграционное тестирование всех компонентов

3.2 Архитектура системы и выбор технологий

В рамках реализации программного инструмента для перевода запросов на естественном языке в SQL-формат было принято решение разработать веб-приложение. Такой выбор обусловлен рядом важных преимуществ, обеспечивающих удобство использования и расширяемость системы.

Во-первых, веб-приложение позволяет пользователю работать с системой через стандартный веб-браузер без необходимости установки дополнительного

программного обеспечения. Это делает продукт кроссплатформенным и доступным с любого устройства, подключённого к интернету.

Во-вторых, веб-архитектура способствует масштабируемости: к приложению могут подключаться сразу несколько пользователей, что делает систему пригодной как для индивидуального использования, так и для интеграции в корпоративные среды. Таким образом, выбор веб-приложения обусловлен необходимостью обеспечить широкую доступность, гибкость, удобство в эксплуатации и возможность дальнейшего развития проекта.

Для реализации данного инструмента необходимо организовать архитектуру приложения рис.3.1, включающую четыре основных компонента: Frontend, Backend, Translation Module и Database.

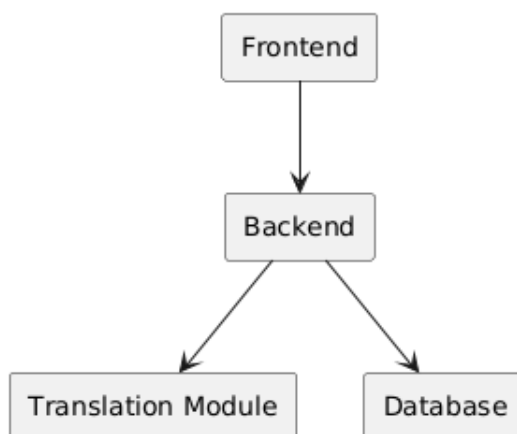


Рис.3.1. Архитектура приложения

Frontend представляет собой пользовательский интерфейс, через который осуществляется взаимодействие с системой. Пользователь вводит текстовый запрос на естественном языке, просматривает сгенерированный SQL-запрос, имеет возможность его отредактировать, отправить в базу данных и получить результат. Также на этом уровне реализованы функции регистрации, авторизации и просмотра истории запросов.

Backend выполняет роль промежуточного звена между пользовательским интерфейсом, модулем перевода и базой данных. Он обрабатывает пользовательские запросы, обеспечивает маршрутизацию данных, реализует бизнес-логику, а также отвечает за безопасность и хранение истории взаимодействий.

Translation Module включает реализацию модели, которая используется для преобразования текста на естественном языке в SQL-запрос. Backend взаимодействует с этим модулем, передавая текст запроса и получая SQL-код в ответ.

Такой подход позволяет изолировать интеллектуальную составляющую системы от остальной логики.

Database — это хранилище данных, с которым взаимодействует конечный пользователь. После генерации SQL-запроса и его возможной корректировки пользователь отправляет его на выполнение в свою базу данных, а результаты выводятся через интерфейс. Также отдельная база данных используется для хранения истории запросов и пользовательских данных (регистрация, авторизация).

Для реализации программного инструмента по переводу запросов на естественном языке в SQL-формат было принято решение разработать веб-приложение на языке программирования Java. Такой выбор обусловлен рядом весомых причин. Java является строго типизированным объектно-ориентированным языком с высокой стабильностью, надёжностью и производительностью. Эти качества делают его особенно подходящим для создания серверных приложений, обрабатывающих большое количество запросов и взаимодействующих с различными внешними модулями, в том числе с системами машинного обучения и базами данных. Благодаря виртуальной машине Java (JVM) обеспечивается кроссплатформенность, а богатая экосистема, в частности Spring Boot, предоставляет разработчику гибкие и мощные инструменты для построения REST API, настройки безопасности, работы с базой данных и внедрения зависимостей. Java широко применяется в корпоративной среде, имеет активное сообщество и большое количество квалифицированных специалистов.

В рамках fronted части было принято реализовать простые html экраны с использованием шаблонов, для этого будут использованы Thymeleaf и Bootstrap. Для взаимодействий с сервером будет использован JavaScript и Fetch API, который необходим для асинхронных запросов к контроллерам.

Backend будет реализован на Spring Boot, так как данный фреймворк предоставляет механизмы для упрощенного построения REST API. Он подходит для создания модульной архитектуры, которую будет легко масштабировать. Для взаимодействия с базой данных будет использован Spring Data JPA, так как он упрощает реализацию взаимодействия с репозиториями и сущностями. Контроль доступа и безопасность будет реализована с помощью Spring Security, это позволит реализовать регистрацию, авторизацию и защиту API.

Translation Module, отвечающий за преобразование текста на естественном языке в SQL-запрос, реализуется отдельно на Python, так как именно в этой среде представлены наиболее современные и эффективные инструменты для обработки

естественного языка и обучения нейронных сетей. В частности, используются библиотеки transformers, для работы с моделями типа T5, BERT, GPT, и TensorFlow для запуска и обучения моделей. Такой выбор позволяет использовать готовые предобученные модели, а также адаптировать их под конкретную предметную область. Модуль изолируется от остальной системы и взаимодействует с backend через HTTP-запросы, что упрощает масштабирование и независимую разработку интеллектуального ядра системы. Важно, что в этот модуль передаётся не только текстовый запрос, но и извлечённая схема базы данных пользователя — это позволяет модели корректно учитывать структуру таблиц и генерировать синтаксически и логически верные SQL-запросы.

В качестве Database используется PostgreSQL, так как это мощная и надёжная реляционная база данных с открытым исходным кодом, поддерживающая сложные SQL-запросы, транзакции, расширения, полнотекстовый поиск и хранение JSON-структур. В ней хранятся не только пользовательские данные и история запросов, но также осуществляется выполнение сгенерированных SQL-запросов к пользовательским таблицам.

Архитектура разработанного приложения представляет собой монолитную многоуровневую систему, построенную по классической модели с разделением на frontend, backend и модуль машинного обучения. Несмотря на наличие нескольких логически обособленных компонентов, приложение реализовано как единое целое, где все слои тесно связаны.

Многоуровневая структура рис.3.2 предполагает явное деление на презентационный слой (интерфейс), логический слой (обработка бизнес-логики) и слой работы с данными.

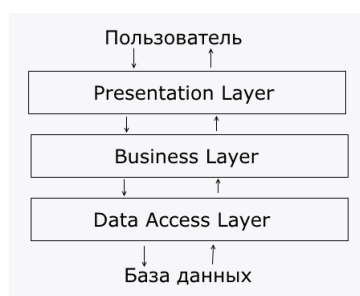


Рис.3.2. Схема многоуровневой архитектуры [9].

На верхнем уровне будет располагаться пользовательский интерфейс. Этот слой отвечает за взаимодействие с пользователем: ввод текстовых запросов, подключение к базе данных, отображение переведённых SQL-запросов и результатов выполнения, а также просмотр истории.

Второй уровень – это логика сервера. Здесь будут обрабатываться все основные процессы: регистрация пользователей, приём входных данных, взаимодействие с моделью машинного обучения, выполнение SQL-запросов и хранение истории действий. Этот слой служит связующим звеном между интерфейсом, модулем машинного обучения и базой данных.

Модуль машинного обучения также включён в состав монолита и будет функционировать как отдельный компонент внутри серверной части. Он отвечает за преобразование естественного языка в SQL-запросы.

Таким образом, вся система спроектирована как монолитное приложение, где каждый уровень представляет собой слой, выполняющий строго определённую роль. Такой подход обеспечивает простоту поддержки, хорошую согласованность компонентов и быструю проверку гипотез в рамках проекта.

3.3 Проектирование модели перевода

Основная задача всего проекта - создать и обучить модель, которая будет принимать на вход схему базы данных и текстовый запрос на русском языке, а возвращать SQL эквивалент запроса.

Для выполнения этой задачи будет реализовано две модели перевода. LSTM - типрекуррентной нейронной сети, которая создана для обработки последовательных данных, она может быть эффективна для простых запросов. И трансформер - данная модель сложнее LSTM, она использует механизм внимания и учитывает контекст на глубоком уровне. После реализации, будет проведено сравнение их эффективности и будет выбран наилучший подход.

3.3.1 LSTM модель

Архитектура состоит из:

- Encoder - LSTM, который обрабатывает вход (схема + вопрос)
- Decoder - LSTM, генерирующий SQL по выходу энкодера
- Attention - используется между энкодером и декодером для более точного контекста
- Tokenizer - BART-токенизатор, для преобразования текста в токены

Проектируя систему, способную переводить текстовые запросы пользователя на естественном языке в SQL-запросы, важно заранее продумать архитектуру и

взаимодействие всех компонентов. В основе приложения будет находиться модель seq2SQL, которая должна учитывать не только сам текст вопроса, но и структуру конкретной базы данных. Поэтому уже на этапе проектирования необходимо чётко выделить основные модули: обработку данных, представление схемы базы, саму модель генерации и компонент оценки качества.

В качестве обучающего датасета выбран датасет Spider [11]. Он представляет собой размеченный набор данных, предназначенный для обучения и тестирования моделей. Он включает в себя более 10 000 пар "вопрос — SQL-запрос" и более 200 различных схем баз данных.

На уровне подготовки данных необходимо предусмотреть отдельный класс, например, SpiderDataset, который будет отвечать за чтение, преобразование и токенизацию входных данных. Его задача — брать пары "естественный запрос — SQL-запрос" и связывать их с соответствующими схемами баз данных. Важно заранее определить, как именно будет кодироваться схема БД: предполагается, что таблицы и их поля будут превращаться в строковое представление, которое затем объединяется с пользовательским запросом.

В рамках архитектуры модели нужно предусмотреть класс LSTMModel — он станет контейнером для всех компонент генерации. Модель будет основана на принципе encoder-decoder, где входной запрос (вместе со схемой) сначала обрабатывается энкодером, а затем на основе скрытого состояния декодер будет по шагам генерировать SQL-запрос. Внутри модели важно предусмотреть компонент внимания — отдельный класс Attention, позволяющий на каждом шаге генерации фокусироваться на релевантных частях входа. При проектировании следует учитывать, что внимание должно работать не только с текстом запроса, но и с включённой в него схемой — это позволит модели выбирать правильные таблицы и поля при формировании SQL-запроса.

Также на уровне проектирования следует определить единый интерфейс оценки работы модели. Планируется создать класс BLEUEvaluator, который будет сравнивать предсказанные SQL-запросы с референсными и оценивать их с помощью метрик.

Архитектурно проект разбивается на независимые модули: модуль подготовки и представления данных, модуль генерации запроса и модуль оценки. На каждом уровне должны быть свои абстракции: классы, которые инкапсулируют свою зону ответственности и не зависят от конкретной реализации других частей.

Каждый класс должен выполнять одну задачу и быть независимым, чтобы в процессе реализации можно было проводить модульное тестирование, повторно использовать компоненты и обеспечивать стабильную интеграцию новых решений, будь то смена архитектуры модели или добавление новых языков запросов.

3.3.2 *Transformer*

Необходимо разработать нейронную сеть на основе трансформера для автоматического преобразования вопросов на естественном языке в SQL-запросы, с учётом структуры базы данных (схемы). Проект будет использовать предобученную модель семейства BART для выполнения задачи Sequence-to-Sequence (Seq2Seq) трансформации.

Многие принципы и классы я не буду менять, модернизирую прошлую модель, чтобы она использовала трансформер. BART состоит из двух основных компонентов: энкодера и декодера. Оба компонента являются трансформерами, но работают по-разному:

1. Энкодер - это модуль, который принимает на вход входной текст и преобразует его в скрытые представления. Он использует несколько слоёв внимания и полносвязных слоёв для извлечения информации из входной последовательности. В контексте задачи перевода текста в SQL энкодер будет работать с текстом схемы базы данных и вопросом на естественном языке. В результате работы энкодера получается набор скрытых состояний, которые содержат всю информацию, необходимую для дальнейшей генерации ответа.
2. Декодер это компонент, который генерирует выходной текст на основе скрытых состояний, полученных от энкодера. Декодер в BART работает автогереративно, то есть он генерирует один токен за раз, каждый новый токен зависит от ранее сгенерированных. На выходе декодер создаёт последовательность токенов, которая преобразуется в SQL-запрос.

Обучение модели происходит на основе обучающих данных, которые включают текстовые вопросы и соответствующие SQL-запросы. Система будет обучаться не только на вопросах, но и на схеме базы данных, что позволяет учитывать специфическую структуру данных при генерации SQL-запросов.

Входной текст состоит из двух частей — схемы базы данных и вопроса. Схема представляет собой строковое описание таблиц, столбцов и внешних ключей

базы данных, а вопрос — это запрос, сформулированный на естественном языке. Эти данные подаются на вход энкодера.

На основе этих данных модель должна обучиться генерации SQL-запроса.

Модель будет обучаться с использованием оптимизатора, он необходим для минимизации потерь и обновления параметров. Так же будет использован *learning rate scheduler*, который будет снижать скорость обучения, это поможет снизить риск переобучения и улучшит сходимость.

3.4 Проектирование веб-приложения

Для реализации веб-приложения, удовлетворяющего требованиям необходимо решить несколько задач.

Необходимо реализовать механизм регистрации и авторизации с сохранением данных пользователя в базе данных приложения, необходимо реализовать подключение к внешним базам данных пользователей, необходимо разработать процедуру извлечения схемы базы данных, требуется реализовать обращение к модели перевода, которая будет получать схему базы данных и текстовый запрос, а возвращать SQL-запрос, необходимо обеспечить редактирование и выполнение запроса пользователем, а так же организовать сохранение истории переводов пользователя.

3.4.1 Проектирование *backend*-части

Для решения всех задач я выделила основные классы и разделила их по зонам ответственности. Каждый модуль отвечает за строго определённую часть функционала приложения, чтобы обеспечить читаемость, расширяемость и удобство сопровождения проекта. Мной была придумана следующая модель приложения

На рис.3.3 представлена диаграмма классов, опишу каждый класс и его зону ответственности.

Класс *MyUser* представляет основную пользовательскую сущность в системе. Он предназначен для хранения информации о зарегистрированных пользователях, включая логин, пароль и роль. Этот класс будет реализовывать интерфейс *UserDetails* для интеграции со *Spring Security*, обеспечивая участие пользователей в механизме аутентификации и авторизации.

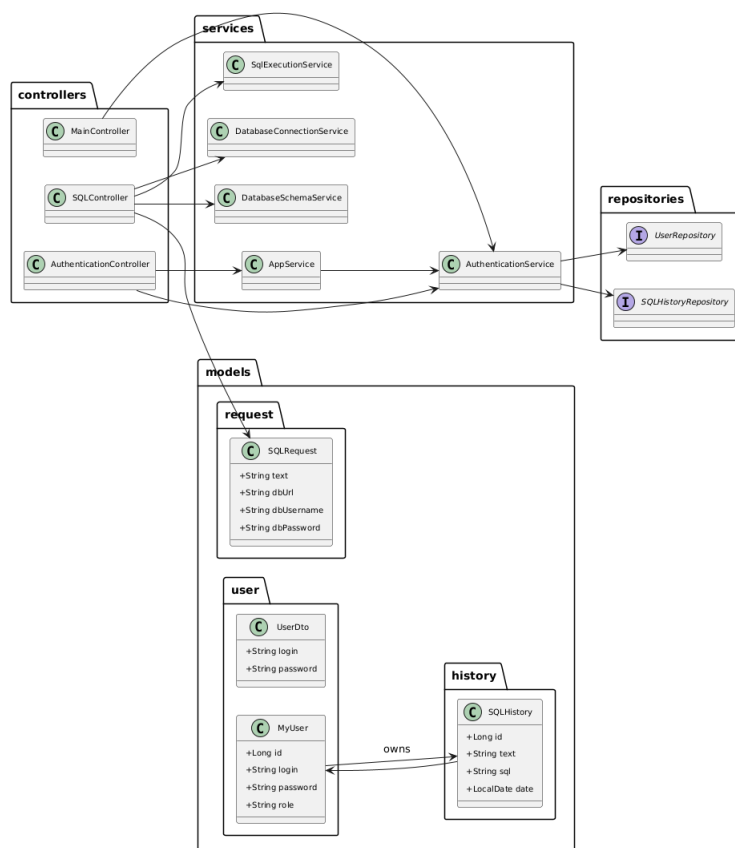


Рис.3.3. Диаграмма классов

Класс `UserDto` проектируется как простой объект передачи данных, необходимый для регистрации и входа в систему. Он инкапсулирует минимальный набор полей — логин и пароль — и используется как валидационная оболочка между интерфейсом и моделью пользователя.

Класс `SQLHistory` предназначен для хранения истории преобразованных SQL-запросов, выполненных пользователями. Каждый объект истории будет содержать исходный текстовый запрос, сгенерированный SQL-запрос, дату выполнения и ссылку на пользователя, к которому он относится. Это позволит реализовать возможность просмотра истории для каждого пользователя.

Класс `SQLRequest` играет роль временного транспортного объекта и используется для передачи текстового запроса и параметров подключения к внешней базе данных от клиента на сервер. Он служит интерфейсом между пользовательским вводом и внутренними сервисами обработки.

Интерфейсы `UserRepository` и `SQLHistoryRepository` проектируются как абстракции для доступа к соответствующим сущностям в базе данных приложения. Они позволяют использовать Spring Data JPA для декларативного описания операций над объектами `MyUser` и `SQLHistory`.

AuthenticationController отвечает за маршрутизацию запросов, связанных с регистрацией и аутентификацией. Он обрабатывает действия пользователя на страницах логина и регистрации, взаимодействуя с соответствующим сервисом для проверки данных и создания учетной записи.

SQLController является центральным звеном обработки запросов. Он принимает текстовые описания запросов, извлекает структуру базы данных, взаимодействует с моделью машинного обучения, выполняет SQL-запрос в базе данных пользователя, и сохраняет информацию о выполненных действиях.

MainController обслуживает отображение статичных и динамических HTML-страниц, таких как главная страница, форма генерации запроса и история. Он связывает представление с внутренней логикой приложения.

Сервис AuthenticationService инкапсулирует бизнес-логику, связанную с управлением пользователями: регистрацией, хешированием паролей, поиском пользователей и обработкой токенов.

AppService выступает как интеграционный сервис, координирующий работу между отдельными функциональными блоками: генерацией SQL, выполнением запроса, и сохранением истории. Он инкапсулирует основную бизнес-логику обработки одного запроса пользователя.

DatabaseConnectionService реализует функциональность по установлению соединения с базой данных пользователя на основе предоставленных им параметров. Он абстрагирует низкоуровневую работу с JDBC и позволяет системе динамически подключаться к произвольным базам данных.

DatabaseSchemaService отвечает за извлечение структуры базы данных: таблиц, колонок и связей. Эти данные используются моделью трансформации для генерации SQL-запросов, максимально соответствующих контексту.

SqlExecutionService предназначен для безопасного выполнения сгенерированных SQL-запросов. Он использует подключения, предоставленные DatabaseConnectionService, и возвращает результат в удобной для клиента форме.

Далее чтобы понять как все эти модули будут взаимодействовать друг с другом была спроектирована диаграмма последовательности рис.3.4.

1. Авторизация пользователя

Пользователь инициирует процесс авторизации или регистрации, отправляя запрос в AuthenticationController.

Контроллер вызывает сервис аутентификации AuthenticationService, который обрабатывает логику аутентификации или регистрации.

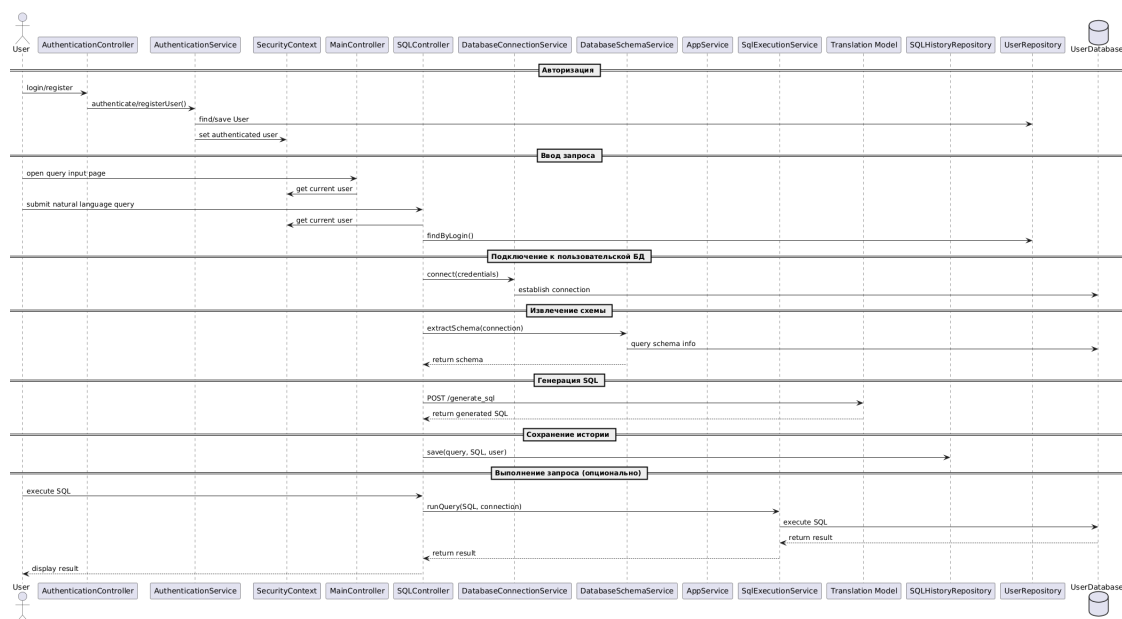


Рис.3.4. Диаграмма последовательности

Сервис аутентификации взаимодействует с репозиторием пользователей (UserRepository), чтобы найти или сохранить данные пользователя.

После успешной аутентификации или регистрации сервис устанавливает аутентифицированного пользователя в SecurityContext, который управляет текущим состоянием безопасности.

2. Ввод запроса на естественном языке.

Пользователь открывает страницу для ввода запроса.

MainController извлекает информацию о текущем пользователе из SecurityContext, чтобы узнать, кто вошёл в систему.

3. Отправка запроса на естественном языке в SQL.

Пользователь отправляет запрос на естественном языке через интерфейс пользователя в SQLController. Контроллер извлекает информацию о текущем пользователе.

4. Подключение к базе данных пользователя.

Контроллер обращается к DatabaseConnectionService и передает в него данные для подключения, сервис устанавливает соединение с базой данных.

5. Извлечение схемы базы данных пользователя.

Контроллер обращается к DatabaseSchemaService, сервис обращается к базе данных и извлекает таблицы, столбцы, поля, связи и возвращает информацию контроллеру.

6. Генерация SQL-запроса.

Контроллер отправляет запрос на естественном языке и схему базы данных в модель перевода (TranslationModel), которая преобразует запрос в SQL.

Модель перевода возвращает сгенерированный SQL-запрос в контроллер.

7. Сохранение истории запросов пользователя.

Контроллер сохраняет запрос на естественном языке, сгенерированный SQL-запрос и информацию о пользователе в репозитории истории (SQLHistoryRepository).

8. Выполнение SQL-запроса.

Если пользователь решает выполнить сгенерированный SQL, он отправляет его в SQLController.

Контроллер передает SQL-запрос в сервис исполнения SQL (SqlExecutionService).

Сервис исполнения SQL выполняет запрос в базе данных пользователя.

База данных возвращает результат выполнения SQL-запроса.

SQLController: Сервис исполнения SQL передает результат обратно в контроллер.

Контроллер выводит результат выполнения запроса пользователю на экран.

3.4.2 Проектирование frontend-части

Для удобства и логики работы приложения пользовательский интерфейс будет разбит на несколько основных экранов. Каждый экран отвечает за свою часть пользовательского пути — от авторизации до выполнения запроса и просмотра истории действий.

Основные экраны

1. Экран регистрации и авторизации рис.3.5.

Экран позволяет пользователю зарегистрироваться или войти в приложение, используя логин и пароль. После успешной авторизации пользователь будет перенаправлен к основному функционалу. На экране должны быть расположены поля ввода логина и пароля, кнопки для входа и регистрации, если пользователь еще не зарегистрирован. Если пользователь не зарегистрирован, ему будет предложено зарегистрироваться, для этого необходимо ввести свои личные данные.

Рис.3.5. Экран авторизации.

2. Экран работы с запросом рис.3.6.

Основной экран, где происходит работа с базой данных. Пользователь вводит данные подключения к БД, текстовый запрос, получает перевод на SQL и видит результат выполнения запроса. На экране должна быть форма для ввода данных для подключения к личной базе данных пользователя, так же поле для ввода запроса на естественном языке и кнопка "Перевести". Рядом будет находиться поле с ответом в виде SQL запроса, пользователь сможет внести изменения в этот запрос и отправить его в свою личную базу данных. Ниже пользователь получит ответ в виде таблиц.

Рис.3.6. Экран работы с запросом.

3. Экран истории переводов

Экран отображает список всех ранее переведённых запросов. Пользователь может просмотреть детали каждого запроса и результатов его выполнения.

Понимание пользовательского потока важно для проектирования последовательного и удобного опыта использования приложения. Ниже описан основной сценарий использования приложения обычным пользователем.

Поток работы пользователя

1. Пользователь открывает приложение, попадает на экран регистрации и авторизации, логинится или регистрируется.
2. После входа попадает на основной экран.
3. Вводит параметры подключения.
4. Пишет естественный текст запроса (например: "Показать всех сотрудников из отдела продаж").
5. Нажимает "Перевести запрос".
6. Получает SQL-код (например: `SELECT * FROM employees WHERE department = 'Sales'`).
7. Может отредактировать SQL вручную.
8. Нажимает "Выполнить запрос".
9. Видит результаты в виде таблицы.
10. Переходит на экран с историей, чтобы увидеть предыдущие запросы и результаты.

3.5 Вывод

В процессе проектирования приложения была создана его архитектура, которая включает в себя три ключевых компонента: модель машинного обучения, серверную часть приложения и пользовательский интерфейс.

В целях реализации модели перевода естественно-языковых запросов в SQL-запросы были созданы две специализированные модели: LSTM и transformer. Обе модели построены с учетом особенностей работы с текстовыми данными и допускают дообучение для повышения точности.

Серверная часть отвечает за взаимодействие пользователей с моделью перевода. Разработана она на базе Java Spring Boot. Для удобства взаимодействия с различными компонентами, были созданы отдельные сервисы: для работы с базой данных пользователя, с моделью перевода и с историей его запросов.

В процессе разработки пользовательского интерфейса были выявлены ключевые экраны приложения, каждый из которых содержит элементы управления и взаимодействия с его функциональностью.

В результате проектирование охватило все существенные элементы системы, создав фундамент для ее последующей эффективной реализации, тестирования и внедрения.

4 ПРОГРАММИРОВАНИЕ ИНСТРУМЕНТА ДЛЯ ПЕРЕВОДА ЕСТЕСТВЕННОГО ЯЗЫКА В SQL-КОД

4.1 Программирование модели перевода текста в SQL-код

4.1.1 LSTM-модель

Вначале работы необходимо импортировать все необходимые библиотеки рис.4.1. каждая из подключенных библиотек играет важную роль. `torch`, `torch.nn`, `torch.optim` и `torch.utils.data` предоставляют всё необходимое для построения, обучения и тестирования нейросети: от слоёв до оптимизаторов и загрузчиков данных. `pad_sequence` используется для выравнивания последовательностей до одинаковой длины в батче. `numpy` помогает с базовыми математическими операциями и преобразованием данных. `transformers`. `BartTokenizer` применяется для токенизации текстов на естественном языке, подготавливая входы для модели. `tqdm` обеспечивает визуальный прогресс-бар во время обучения. `os` нужен для работы с файлами и путями. `nltk.translate.bleu_score` и `nltk` используются для оценки качества перевода модели при помощи метрики BLEU. `json` — для загрузки и сохранения данных в формате JSON.

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, random_split
from torch.nn.utils.rnn import pad_sequence
import numpy as np
from transformers import BartTokenizer
from tqdm import tqdm
import os
from nltk.translate.bleu_score import sentence_bleu
import nltk
import random as rnd
import json
from transformers import BartTokenizer
from torch.utils.data import Dataset
```

Рис.4.1. Импорт библиотек.

Подготовка и обработка данных.

Класс SpiderDataset играет центральную роль в подготовке данных для модели. Его главная задача - преобразовать сырые данные из датасета Spider в формат, пригодный для обучения нейронной сети. При инициализации класс принимает пути к файлам с вопросами и схемами баз данных, а также токенизатор и максимальную длину последовательности, которая ограничивает размер входных данных для предотвращения проблем с памятью и ускорения обучения рис.4.2.

```
class SpiderDataset(Dataset):
    def __init__(self, data_path, schema_path, tokenizer, max_length=256):
        self.tokenizer = tokenizer
        self.max_length = max_length
        self.schema_dict = self.load_schema(schema_path)
        self.data = self.load_data(data_path)
```

Рис.4.2. Инициализация класса.

Особое внимание уделяется загрузке рис.4.3 и обработке схем баз данных рис.4.4. Каждая схема преобразуется в читаемую строку, содержащую список таблиц с их колонками и информацию о внешних ключах. Например, схема преобразуется в строку вида: "Table: Users (id, name, age); Table: Orders (id, user_id, product); Foreign Keys: Orders.user_id → Users.id". Такой формат позволяет модели легче понимать структуру базы данных при генерации SQL-запросов.

```
def load_schema(self, schema_path):
    with open(schema_path, 'r') as f:
        schema_data = json.load(f)

    schema_dict = {}
    for db in schema_data:
        db_id = db['db_id']
        schema_str = self._schema_to_string(db)
        schema_dict[db_id] = schema_str
    return schema_dict
```

Рис.4.3. Загрузка схем.

```
def _schema_to_string(self, db):
    schema_parts = []
    for table_idx, table in enumerate(db['table_names_original']):
        table_cols = []
        for col_idx, col_info in enumerate(db['column_names_original']):
            if col_info[0] == table_idx:
                table_cols.append(col_info[1])
        schema_parts.append(f"Table: {table} ({', '.join(table_cols)})")

    foreign_keys = []
    for fk in db['foreign_keys']:
        col1 = db['column_names_original'][fk[0]][1]
        col2 = db['column_names_original'][fk[1]][1]
        table1 = db['table_names_original'][db['column_names_original'][fk[0]][0]]
        table2 = db['table_names_original'][db['column_names_original'][fk[1]][0]]
        foreign_keys.append(f"{table1}.{col1} -> {table2}.{col2}")

    schema_str = "; ".join(schema_parts)
    if foreign_keys:
        schema_str += "; Foreign Keys: " + ", ".join(foreign_keys)
    return schema_str
```

Рис.4.4. Обработка схем.

Метод `getitem` класса рис.4.5 отвечает за подготовку отдельных примеров для обучения. Он комбинирует схему базы данных и вопрос в единую входную строку, которая затем токенизируется. SQL-запрос токенизируется отдельно как целевая последовательность. Важно отметить, что применяется паддинг, добавление специальных токенов, для выравнивания длин последовательностей, что необходимо для эффективной пакетной обработки данных.

```
def _schema_to_string(self, db):
    schema_parts = []
    for table_idx, table in enumerate(db['table_names_original']):
        table_cols = []
        for col_idx, col_info in enumerate(db['column_names_original']):
            if col_info[0] == table_idx:
                table_cols.append(col_info[1])
        schema_parts.append(f"Table: {table} ({', '.join(table_cols)})")

    foreign_keys = []
    for fk in db['foreign_keys']:
        col1 = db['column_names_original'][fk[0]][1]
        col2 = db['column_names_original'][fk[1]][1]
        table1 = db['table_names_original'][db['column_names_original'][fk[0]][0]]
        table2 = db['table_names_original'][db['column_names_original'][fk[1]][0]]
        foreign_keys.append(f"{table1}.{col1} -> {table2}.{col2}")

    schema_str = "; ".join(schema_parts)
    if foreign_keys:
        schema_str += "; Foreign Keys: " + ", ".join(foreign_keys)
    return schema_str
```

Рис.4.5. Метод `getitem`.

Механизм внимания

Механизм внимания в данной реализации играет ключевую роль, позволяя модели эффективно устанавливать связи между элементами входной последовательности (схемой базы данных и вопросом) и генерируемым SQL-запросом. Давайте детально разберём его работу от фундаментальных принципов до конкретной реализации в коде.

В основе механизма лежит концепция масштабированного скалярного произведения, которая была предложена в революционной работе "Attention Is All You Need"[1]. В нашем случае этот механизм адаптирован для работы с LSTM-архитектурой, создавая гибридную модель, сочетающую достоинства рекуррентных сетей и внимания.

Работа начинается с подготовки трех ключевых компонентов - запросов (Q), ключей (K) и значений (V).

Запросы (Q) представляют текущее состояние декодера - то, на каком этапе генерации SQL мы находимся

Ключи (K) и значения (V) происходят из выходных состояний энкодера, который обработал входную последовательность (схему БД + вопрос)

```

class Attention(nn.Module):
    def __init__(self, d_model, d_k, d_v, dropout=0.3, n_head=1):
        super(Attention, self).__init__()
        self.n_head = n_head
        self.d_k = d_k
        self.d_v = d_v
        self.dropout = dropout
        self.W_k = nn.Linear(in_features=d_model, out_features=d_k * n_head, bias=False)
        self.W_q = nn.Linear(in_features=d_model, out_features=d_k * n_head, bias=False)
        self.W_v = nn.Linear(in_features=d_model, out_features=d_v * n_head, bias=False)

        self.dot_product_attention = ScaledDotProductAttention(
            temperature=np.sqrt(d_k), #  $\sqrt{d_k}$ 
            attn_dropout=self.dropout
        )

```

Рис.4.6. Инициализация механизма внимания.

Процесс вычисления внимания происходит в несколько четко определенных этапов рис.4.7. Сначала мы вычисляем матрицу сходства между запросами и ключами через скалярное произведение. Это дает нам "сырые" оценки того, насколько каждый элемент входной последовательности важен для текущего шага декодера. Затем эти оценки масштабируются делением на квадратный корень из размерности ключей ($\sqrt{d_k}$) - этот технический прием предотвращает проблемы с исчезающими градиентами при больших размерностях.

Особое значение имеет этап маскирования. Хотя в данной конкретной реализации маска явно не используется, механизм предусматривает возможность зануления определенных элементов перед применением функции softmax. Это особенно полезно при работе с последовательностями переменной длины, где нужно игнорировать padding-токены.

После softmax мы получаем нормализованные веса внимания - вероятностное распределение, показывающее, какие части входной последовательности наиболее релевантны текущему шагу генерации. Важно отметить, что перед softmax применяется dropout - это служит мощным инструментом регуляризации, случайным образом "выключая" некоторые связи внимания во время обучения и тем самым предотвращая переобучение.

На финальном этапе веса внимания умножаются на значения (V), что дает нам взвешенную сумму - контекстный вектор, содержащий наиболее важную информацию из входа для текущего шага декодера.

Реализация многоголового внимания рис.4.8 добавляет еще один уровень сложности и выразительности. Вместо одного механизма внимания мы имеем несколько параллельных "голов" в данном случае - 2, каждая со своими обучаемыми матрицами проекций для Q , K и V . Это позволяет модели одновременно обращать внимание на разные аспекты входных данных - например, одна голова может фокусироваться на структуре таблиц, вторая - на условиях фильтрации в вопросе.

```

class ScaledDotProductAttention(nn.Module):
    def __init__(self, temperature, attn_dropout=0.3):
        super().__init__()
        self.temperature = temperature
        self.dropout = nn.Dropout(attn_dropout)

    def forward(self, q, k, v, mask=None):
        attn = torch.matmul(q / self.temperature, k.transpose(2, 3))

        if mask is not None:
            attn = attn.masked_fill(mask == 0, -1e9)

        attn = self.dropout(torch.nn.functional.softmax(attn, dim=-1))
        output = torch.matmul(attn, v)

        return output, attn

```

Рис.4.7. Реализация механизма внимания.

```

def forward(self, q, k, v, mask=None, return_attention=False):
    d_k, d_v, n_head = self.d_k, self.d_v, self.n_head
    sz_b, len_q, len_k, len_v = q.size(0), q.size(1), k.size(1), v.size(1)

    q = self.W_q(q).view(sz_b, len_q, n_head, d_k)
    k = self.W_k(k).view(sz_b, len_k, n_head, d_k)
    v = self.W_v(v).view(sz_b, len_v, n_head, d_v)

    q, k, v = q.transpose(1, 2), k.transpose(1, 2), v.transpose(1, 2)

    if mask is not None:
        mask = mask.unsqueeze(1)

    q, attn = self.dot_product_attention(q, k, v, mask=mask)
    if return_attention:
        return q, attn
    return q

```

Рис.4.8. Реализация многоголового внимания.

Интеграция механизма внимания в LSTM-архитектуру выполнена через конкатенацию выходного состояния декодера и контекстного вектора, полученного из внимания. Это объединенное представление затем пропускается через финальный линейный слой для предсказания следующего токена SQL. Такая архитектура позволяет модели сочетать долгосрочную память LSTM с целенаправленной способностью внимания выделять релевантные части входа.

В процессе генерации SQL механизм внимания демонстрирует свою ценность особенно ярко. На каждом шаге декодирования он динамически "подсвечивает" те части схемы базы данных и вопроса, которые наиболее важны для генерации текущего элемента SQL-запроса. Например, при генерации условия WHERE механизм внимания может автоматически усиливать вес тех частей входной последовательности, где упоминаются соответствующие столбцы и значения для фильтрации.

Технические детали реализации, такие как использование отдельных линейных слоев для проекций Q , K , V без bias-термов, тщательно подобранный коэффициент масштабирования, и правильная обработка размерностей при многоголовом внимании - все это критически важно для работоспособности механизма. Особенно стоит отметить обработку размерностей в методе forward класса Attention,

где view и transpose операции обеспечивают правильную организацию данных для параллельной обработки несколькими головами внимания.

LSTM модель

Модель начинается с эмбединг-слоя, который преобразует входные токены в плотные векторные представления размерностью 256. Эти эмбединги поступают в двунаправленный LSTM-энкодер, состоящий из двух слоёв с dropout 0.3 между ними, что предотвращает переобучение. Энкодер обрабатывает комбинацию схемы базы данных и вопроса пользователя, создавая скрытые представления размером 512, которые сохраняют информацию обо всех элементах входной последовательности. Это представлено на рис.4.9.

```
class LSTMModel(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, num_layers, dropout=0.3, tf=1, n_head=2):
        super(LSTMModel, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        self.dropout = nn.Dropout(dropout)
        self.tf = tf
        self.encoder_lstm = nn.LSTM(embedding_dim, hidden_dim, num_layers, batch_first=True, dropout=dropout)
        self.decoder_lstm = nn.LSTM(embedding_dim, hidden_dim, num_layers, batch_first=True, dropout=dropout)
        self.attention = Attention(
            d_model=hidden_dim, # Размер выхода энкодера
            d_k=hidden_dim // n_head, # Размер key/query на голову
            d_v=hidden_dim // n_head, # Размер value на голову
            dropout=dropout,
            n_head=n_head
        )
        # *2 - потому что объединяем с выходом Attention
        self.fc = nn.Linear(hidden_dim*2, vocab_size)
```

Рис.4.9. Инициализация LSTM модели.

Например входные токены [SELECT, name, FROM, users] преобразуются в эмбединги размером 256, проходят через 2 LSTM-слоя с dropout между ними. На выходе получается encoder_outputs - все скрытые состояния (форма [batch_size, seq_len, 512]) и (hidden, cell) - финальное состояние, контекст для декодера.

Особенность архитектуры проявляется в декодере, где LSTM последовательно генерирует SQL-запрос, используя два принципиально разных источника информации - сама LSTM-структура декодера поддерживает состояние, которое хранит контекст текущей позиции в SQL-запросе, а так же механизм много-голового внимания, который позволяет декодеру "заглядывать" в нужные части закодированной входной последовательности.

Технически это реализовано через конкатенацию выхода LSTM декодера и результата механизма внимания, что даёт объединённый вектор размерностью 1024. Этот вектор затем проецируется на пространство словаря SQL-токенов через финальный полносвязный слой. Важная деталь - модель поддерживает два режима работы: с teacher forcing во время обучения рис.4.10, когда на вход декодера подаются корректные предыдущие токены из обучающей выборки, и режим генерации,

когда используются собственные предсказания модели рис.4.11. Переключение между этими режимами управляется параметром `tf`, что демонстрирует гибкость архитектуры. Вся эта сложная система обучается end-to-end, минимизируя кросс-энтропийную потерю между предсказанными и фактическими SQL-токенами, при этом игнорируя padding-токены, что повышает эффективность обучения. Результатом является модель, способная учитывать как синтаксическую структуру SQL-запросов через LSTM, так и семантические связи между вопросом и схемой базы данных через механизм внимания.

```

if self.tf == 1:
    decoder_input = self.embedding( target_ids[:, :-1] )
    batch_size, seq_len, _ = decoder_input.size()
    outputs = []

    for t in range(seq_len):
        emb = decoder_input[:, t, :].unsqueeze(1)

        # с вероятностью 0.5 заменяем token обучения на предсказанный
        if t > 0 and rnd.random() > 0.7:
            emb = self.embedding(pred).unsqueeze(1)

        output, (hidden, cell) = self.decoder_lstm(emb, (hidden, cell) )

        # Применяем внимание: q из декодера, k и v из энкодера
        attn_output = self.attention(
            q=output,                # [bsize, 1, dec_hidden_dim]
            k=encoder_outputs,       # [bsize, src_len, enc_hidden_dim]
            v=encoder_outputs,       # [bsize, src_len, enc_hidden_dim]
            mask=None                # Опционально: маска для padding
        ) # attn_output: [bsize, n_head*enc_hidden_dim]

        out_w_attn = torch.cat( (output.squeeze(1), attn_output.squeeze(2).flatten(1)), dim=-1 )

        outputs.append( self.fc(out_w_attn) )

        pred = torch.argmax(self.fc(out_w_attn), dim=1) # [bs x 1]

        # outputs.append(self.fc(output.squeeze(1)))
    return torch.stack(outputs, dim=1)

```

Рис.4.10. Модель с teacher forcing.

Пример для генерации WHERE с teacher forcing.: на вход декодера подаётся [`<s>`, SELECT, name, FROM, Libraries], при генерации 5-го токена LSTM получает "FROM" и текущее состояние, механизм внимания выделяет "city" в схеме и "London" в вопросе, модель предсказывает "WHERE" с высокой вероятностью.

```

else:
    output, (hidden, cell) = self.decoder_lstm( self.embedding(target_ids), (hidden, cell) )

    # Применяем внимание: q из декодера, k и v из энкодера
    attn_output = self.attention(
        q=output,                # [bsize, 1, dec_hidden_dim]
        k=encoder_outputs,       # [bsize, src_len, enc_hidden_dim]
        v=encoder_outputs,       # [bsize, src_len, enc_hidden_dim]
        mask=None                # Опционально: маска для padding
    ) # attn_output: [bsize, n_head, seq_len, enc_hidden_dim]

    out_w_attn = torch.cat( (output.squeeze(1), attn_output.squeeze(2).flatten(1)), dim=-1 )

    output = self.fc(out_w_attn)
    return output, (hidden, cell)

```

Рис.4.11. Модель без teacher forcing.

Пример генерации без teacher forcing: генерация начинается с входного `<s>`, далее модель предсказывает SELECT, далее на вход поступает SELECT, модель

предсказывает name, на вход поступает name, модель предсказывает FROM, и так далее пока не сгенерируется </s>.

Процесс обучения модели.

Обучение модели организовано стандартным для PyTorch образом, но с некоторыми важными особенностями. В качестве функции потерь используется кросс-энтропия с игнорированием padding токенов. Это важно, так как мы не хотим, чтобы модель училась предсказывать эти искусственно добавленные токены.

Оптимизатор Adam с learning rate 0.001 обеспечивает эффективное обновление весов модели. Во время обучения мы отслеживаем как ошибку на обучающей выборке, так и на валидационной, что позволяет вовремя обнаружить переобучение. Модель сохраняется на диск только при улучшении показателей на валидации, что гарантирует, что мы всегда имеем доступ к наилучшей версии модели.

Оценка качества.

Для оценки качества модели используется метрика BLEU (4.1), первоначально разработанная для машинного перевода рис.4.12. Она сравнивает сгенерированный SQL с эталонным, учитывая совпадения n-грамм, последовательностей из n токенов. В нашей реализации мы используем BLEU-4, который учитывает совпадения от 1 до 4 последовательных токенов, с равными весами, 0.25 для каждого уровня. Хотя BLEU не идеальна для оценки SQL-запросов, так как не учитывает семантическую эквивалентность разных формулировок одного запроса, она дает полезную оценку поверхностного сходства.

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \cdot \log p_n \right) \quad (4.1)$$

Где: BP — штраф за длину

$$\text{BP} = \begin{cases} 1 & \text{если } c > r \\ e^{1-\frac{r}{c}} & \text{если } c \leq r \end{cases} \quad (4.2)$$

p_n — точность n-грамм

$$p_n = \frac{\text{Количество совпадающих n-грамм}}{\text{Общее количество n-грамм в гипотезе}} \quad (4.3)$$

- w_n — веса для n-грамм, $\sum_{n=1}^N w_n = 1$. - N — максимальный порядок n-грамм (обычно 4).

```
def calculate_bleu(model, valid_loader, tokenizer, device, max_length=128):
    model.eval()
    bleu_scores = []
    with torch.no_grad():
        for batch in tqdm(valid_loader, desc="Calculating BLEU"):
            input_ids = batch['input_ids'].to(device)
            questions = batch['question']
            reference_sqls = batch['sql']

            for i, question in enumerate(questions):
                # Генерируем SQL для каждого вопроса
                generated_sql = generate_sql(model, tokenizer, question, max_length=max_length, device=device)

                # Токенизируем эталонный и сгенерированный SQL для BLEU
                reference = nltk.word_tokenize(reference_sqls[i].lower())
                hypothesis = nltk.word_tokenize(generated_sql.lower())

                # Вычисляем BLEU (используем веса для 1-грамм до 4-грамм)
                bleu = sentence_bleu([reference], hypothesis, weights=(0.25, 0.25, 0.25, 0.25))
                bleu_scores.append(bleu)

    avg_bleu = sum(bleu_scores) / len(bleu_scores) if bleu_scores else 0.0
    model.train()
    model.tf=1
    return avg_bleu
```

Рис.4.12. Вычисление оценки качества.

Главный цикл программы и настройки.

Основной цикл программы объединяет все компоненты в рабочую систему рис.4.13. Он начинается с инициализации вычислительного устройства. Затем загружается токенизатор BART, который уже предобучен на больших объемах текстовых данных и "знает" как разбивать текст на осмысленные токены.

Параметры модели (размерность эмбедингов, скрытого состояния, количество слоев) подобраны как компромисс между качеством и вычислительной сложностью. Например, скрытая размерность 512 позволяет модели улавливать достаточно сложные зависимости, не делая ее чрезмерно большой для тренировки.

```
def main():
    seed = 42
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)

    np.random.seed(seed)

    data_path = "./spider/train_spider.json"
    schema_path = "./spider/tables.json"

    vocab_size = tokenizer.vocab_size
    embedding_dim = 256
    hidden_dim = 512
    num_layers = 2
    num_epochs = 10
    batch_size = 8
    is_train = 1
    train_split = 0.8

    seed = 42
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)

    np.random.seed(seed)

    # Устройство
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    # Датасет и загрузчик
    dataset = SpiderDataset(data_path, schema_path, tokenizer)
```

Рис.4.13. Главный цикл программы.

Программа поддерживает два режима работы: обучение и генерация. В режиме обучения данные разбиваются на обучающую и валидационную выборки, создаются загрузчики данных для эффективной пакетной обработки, и запускается процесс обучения. В режиме генерации загружается предварительно обученная модель и применяется для перевода примеров вопросов в SQL-запросы.

Эта реализация демонстрирует классический подход к задаче NL2SQL с использованием современных техник глубокого обучения.

Пример работы модели.

```
Модель загружена успешно из sql_generator_model.pth.
Входной вопрос: Schema: Table: Libraries (library_id, library_name, city, opening_hours, other_library_details); | Question: Which libraries in London?
Сгенерированный SQL: SELECT count(*) FROM budget WHERE budgeted > 3000 AND YEAR <= 2001
```

Рис.4.14. Пример работы модели.

Схема: Table: Libraries (library_id, library_name, city, opening_hours, other_library_details); Вопрос: Which libraries in London?

Сгенерированный моделью SQL: SELECT count(*) FROM budget WHERE budgeted > 3000 AND YEAR <= 2001

В данном случае рис.4.14 модель сгенерировала полностью некорректный SQL-запрос, который не имеет никакого отношения к входному вопросу, ни по структуре запроса, ни по используемым таблицам и столбцам.

Это пример генерации, когда модель потеряла контекст схемы и вопроса, и вместо ответа на естественный запрос подставила неуместный шаблон, вероятно заученный на других примерах.

Главная проблема LSTM - граничные способности к долгосрочному запоминанию контекста

4.1.2 Transformer

На первом этапе подготовки окружения происходит загрузка и распаковка датасета Spider аналогично прошлой модели, который содержит пары из вопросов на естественном языке и соответствующих SQL-запросов, а также описания схем баз данных. Кроме того, происходит установка и обновление библиотеки transformers, предоставляющей доступ к предобученным моделям и токенизаторам, необходимым для последующего построения архитектуры.

Внутри класса SpiderDataset реализованы методы для чтения и фильтрации данных. Метод load_data загружает примеры, формируя входной текст из схемы таблиц и вопроса пользователя. Если после токенизации длина последовательно-

сти превышает заданный предел, пример исключается. Метод `load_schema` читает схему базы данных и формирует строковое представление схемы, включая названия таблиц, их поля и связи между таблицами в виде внешних ключей. Метод `__getitem__` возвращает токенизированные входные и выходные последовательности, пригодные для подачи в модель. Реализация данного класса описана ранее.

После определения датасета задаются основные гиперпараметры модели: размер батча, число эпох, скорость обучения, количество warm-up шагов, максимально допустимая длина токенизированной последовательности, а также выбирается вычислительное устройство.

MBartForConditionalGeneration.

Реализация трансформера в данном коде основана на использовании предобученной модели `MBartForConditionalGeneration` из библиотеки `transformers` от Hugging Face. Эта модель представляет собой разновидность архитектуры BART (Bidirectional and Auto-Regressive Transformers), предназначенной для задач типа «sequence-to-sequence» — преобразование входной последовательности в выходную, что идеально подходит для задачи преобразования естественного языка в SQL-запрос.

Модель `MBartForConditionalGeneration` состоит из двух частей: энкодера и декодера, реализованных по архитектуре трансформеров. Энкодер принимает токенизированный текст, состоящий из вопроса пользователя и описания схемы базы данных, и превращает его в скрытые представления — контекстные эмбединги. Декодер, в свою очередь, использует эти представления, чтобы поэтапно (авторегрессивно) генерировать SQL-запрос токен за токеном.

Модель `MBart (Multilingual BART)` — это модифицированная версия BART, обученная на множестве языков. В данном случае используется модель `sn4kebyt3/ru-bart-large`, то есть русскоязычная версия MBart, адаптированная к специфике русского языка. Это важно, поскольку входные вопросы в данном проекте заданы на русском языке.

Во время обучения входной текст (вопрос + схема) преобразуется в токены с помощью токенизатора, затем эти токены подаются в модель рис.4.15. `input_ids` и `attention_mask` поступают в энкодер. Аргумент `labels` используется в качестве ожидаемой выходной последовательности для декодера. Когда указывается `labels`, модель автоматически смещает их на один токен вправо и применяет `teacher forcing`

— способ обучения, при котором на каждом шаге декодер получает правильный предыдущий токен, а не свой же предыдущий прогноз.

```
# Цикл обучения
for epoch in range(EPOCHS):
    model.train()
    print(f"Epoch {epoch + 1}/{EPOCHS}")
    total_loss = 0

    progress_bar = tqdm.tqdm(train_dataloader)
    for batch in progress_bar:
        input_ids = batch['input_ids'].to(DEVICE)
        attention_mask = batch['attention_mask'].to(DEVICE)
        target_ids = batch['target_ids'].to(DEVICE)

        optimizer.zero_grad()

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=target_ids
        )

        loss = outputs.loss
        total_loss += loss.item()

        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        scheduler.step()

    progress_bar.set_description(f"Loss: {loss.item():.4f}")

train_loss = total_loss / len(train_dataloader)
valid_loss = validate(model=model, valid_dataloader=DEV_Dataloader, device=DEVICE)
```

Рис.4.15. Цикл обучения модели.

Результат `outputs` содержит атрибут `loss`, который представляет собой функцию потерь — обычно это кросс-энтропийная ошибка между сгенерированными и реальными SQL-токенами.

После расчета потерь вызывается обратное распространение ошибки и обновление весов.

Во время предсказания используется метод `model.generate`, который реализует авторегрессивную генерацию SQL-запроса рис.4.16

```
def evaluate_example(model, tokenizer, schema, question):
    model.eval()
    input_text = f"Schema: {schema} | Question: {question}"

    inputs = tokenizer(
        input_text,
        max_length=MAX_LENGTH,
        padding='max_length',
        truncation=True,
        return_tensors='pt'
    )

    input_ids = inputs['input_ids'].to(DEVICE)
    attention_mask = inputs['attention_mask'].to(DEVICE)

    with torch.no_grad():
        outputs = model.generate(
            input_ids=input_ids,
            attention_mask=attention_mask,
            max_length=MAX_LENGTH,
            num_beams=4,
            early_stopping=True
        )

    predicted_sql = tokenizer.decode(outputs[0], skip_special_tokens=True)
    return predicted_sql
```

Рис.4.16. Предсказание модели.

Здесь применяется beam search — эвристика, при которой модель одновременно рассматривает несколько, в данном случае 4, наиболее вероятных вариантов на каждом шаге генерации, что повышает шанс получить грамматически и логически корректный SQL.

Выход — это сгенерированные токены, которые затем декодируются в строку SQL.

Пример работы модели.

```
Question: Покажи пользователей в статусе активен?
Predicted SQL: SELECT T1.name FROM users AS T1 JOIN statuses AS T2 ON T1.id T2.id WHERE T2.state 'active'
-----
Question: Покажи всех пользователей
Predicted SQL: SELECT count(*) FROM users
-----
Question: выведи всех студентов имеющих оценку больше тройки по дисциплине математика
Predicted SQL: SELECT T1.name FROM grades AS T1 JOIN students AS T2 ON T1.studentsid T2.studentsid WHERE T2.value > 3
-----
```

Рис.4.17. Пример работы модели.

- "Покажи пользователей в статусе активен?"

Предсказание: `SELECT T1.name FROM users AS T1 JOIN statuses AS T2 ON T1.id T2.id WHERE T2.state 'active'`

Связывание в `ON T1.id T2.id` некорректно - должны связываться `T1.status_id = T2.id`. У поля `statuses` имя `name`, а не `state`. Модель галлюцинировала поле `state`, которого нет в схеме. Модель распознала намерение ("status active"), но не смогла правильно сопоставить поля схемы. Вместо `T2.name = 'active'` вставила несуществующее поле `T2.state`. Возможная причина — слабое внимание к точной структуре таблиц, несмотря на понимание общего смысла.

- "Покажи всех пользователей"

Предсказание: `SELECT count(*) FROM users`

Вместо того чтобы вывести все записи (например, `SELECT * FROM users`), модель возвращает только `count(*)`, то есть количество, а не сами пользователи. Модель интерпретировала «покажи» как «подсчитай», что возможно связано с частыми шаблонами вида "How many users..." в обучающих данных. Это распространенная ошибка генеративных моделей: путать `SELECT *` и `SELECT count(*)`, особенно без четкой указательной фразы типа "сколько".

- "выведи всех студентов имеющих оценку больше тройки по дисциплине математика"

Предсказание: `SELECT T1.name FROM grades AS T1 JOIN students AS T2 ON T1.studentsid = T2.studentsid WHERE T2.value > 3`

Модель связывает таблицы по `studentsid`, хотя должно быть `T1.students_id = T2.id`. Условие фильтрации `T2.value > 3` - оценка `value` находится в `grades`, а не в `students`. Также отсутствует условие по `discipline = 'математика'`. Модель частично поняла структуру и намерение, но неверно распределила поля между таблицами. Пропущен важный фильтр по предмету — "`discipline = 'математика'`". Возможная причина — ошибки копирования имён полей, отсутствие структурного внимания к источнику поля.

4.1.3 Сравнение результатов

Результаты модели-трансформера:

Final Train Loss: 0.0786 Final Valid Loss: 0.0791 Final BLEU Score: 0.3620

Результаты LSTM-модели:

Final Train Loss: 0.1582 Final Valid Loss: 0.1615 Final BLEU Score: 0.1810

Такие результаты ожидаемы, учитывая архитектурные различия моделей.

Train и Valid Loss выше у LSTM-модели чем у трансформера. Это означает, что LSTM-модель хуже моделирует сложные зависимости, так как данная модель хуже понимает контекст.

BLEU Score примерно в два раза ниже, чем в модели трансформера. Запросы, генерируемые LSTM-моделью отличаются от эталонных сильнее, чем у трансформера, так как внимание LSTM-модели ограничено.

На основе полученных данных, можно сделать вывод, что модель трансформера лучше справляется с задачей. Поэтому для работы приложения была выбрана модель трансформера, так как она обеспечивает более корректную и надежную работу.

4.2 Программирование приложения для взаимодействия пользователя и модели перевода

4.2.1 Модели

Были реализованы четыре модели, каждая из которых выполняет конкретную роль в архитектуре приложения. Они определяют структуру и формат данных участвующих в процессах работы системы.

MyUser.

Модель MyUser это сущность пользователя в системе, которая связывает пользователя с его запросами, данная модель реализует интерфейс UserDetails предоставляя данные пользователя, для реализации безопасности.

- id - уникальный идентификатор пользователя.
- login - логин пользователя, это обязательное и уникальное поле, которое используется для аутентификации в системе.
- password - пароль пользователя, который хранится в зашифрованном виде и используется для аутентификации.
- role - роль пользователя в системе, используется для авторизации и определения прав доступа пользователя.
- sqlHistories - связь с сущностью SQLHistory, которая отображает историю SQL-запросов этого пользователя.
- getAuthorities() - метод, который возвращает права доступа пользователя.
- getPassword() - метод, который необходим для аутентификации, он возвращает пароль пользователя.
- getUsername() - метод, который возвращает логин данного пользователя.
- isAccountNonExpired(), isAccountNonLocked(), isCredentialsNonExpired(), isEnabled() - методы, которые используются для проверки состояния аккаунта пользователя

UserDto.

Модель UserDto - это сущность необходимая для передачи данных о пользователе, необходима для изоляции внутреннего пользователя от внешнего API.

- login - логин пользователя.
- password - пароль пользователя.
- confirmPassword - поле для подтверждения пароля.

SQLHistory.

Модель SQLHistory - это сущность истории запросов пользователя. Каждая запись хранит данные о запросе, его дату выполнения, текст на естественном языке и SQL-запрос.

- id - уникальный идентификатор записи.
- text - строка, содержащая запрос на естественном языке.
- sql - сгенерированный SQL-запрос.
- date - дата выполнения запроса.

- user - это обязательное поле, связывающее историю запросов с конкретным пользователем.
- getFormattedDate() - метод, который возвращает дату в формате dd.MM.yyyy.

SQLRequest.

Модель SQLRequest - это сущность необходимая для формирования SQL-запроса. В ней хранятся данные для подключения к базе данных пользователя и запрос.

- text - описание запроса на естественном языке.
- dbHost - хост базы данных.
- dbPort - порт базы данных.
- dbType - тип базы данных.
- dbName - имя базы данных.
- dbUsername - имя пользователя для подключения к базе данных.
- dbPassword - пароль пользователя для подключения к базе данных.
- sqlQuery - SQL-запрос, который был сгенерирован на основе естественного языка.

4.2.2 Контроллеры

AuthenticationController.

Контроллер управляет процессами аутентификации и регистрации. Метод loginPage обрабатывает GET-запрос по адресу "/login направляя пользователя на страницу входа. Метод logout обрабатывает запрос по адресу "/logout направляя пользователя на страницу входа с параметром logout.

Методы registerPage и registerUser реализуют регистрацию нового пользователя. первый метод отображает форму регистрации при обращении к адресу "/register второй метод реализует валидацию введенных данных при регистрации, регистрация осуществляется через сервис authenticationService.

MainController.

Контроллер MainController отвечает за отображение основной страницы приложения и страницы истории запросов. В методе getHomePage обрабатывается запрос к главной странице, где отображается информация, зависящая от состояния приложения. В методе getHistoryPage осуществляется фильтрация и вывод истории SQL-запросов пользователя на основе различных параметров поиска,

таких как текст запроса и дата. История запросов извлекается из репозитория `SQLHistoryRepository`, и отображается в зависимости от введенных критериев.

SQLController.

Главная задача этого контроллера - взаимодействие с сервисами, обеспечивающими необходимый функционал для перевода текста в SQL-код. Контроллер состоит из трех эндпойнтов, каждый из которых решает свои задачи.

Эндпоинт `/connect` реализует подключение к базе данных пользователя. Он вызывает сервис `DatabaseConnectionService`, передавая ему параметры подключения, введенные пользователем. При успешном подключении возвращается статус 200, при возникновении ошибки, возвращается ее описание.

Следующий эндпоинт `/generate` реализует саму генерацию запроса. Сначала происходит обращение к сервису `DatabaseSchemaService` и запрашивается схема базы данных пользователя, затем схема базы данных и текст запроса отправляются на Python API, который отвечает за генерацию SQL. После получения ответа от модели перевода, контроллер сохраняет запрос в историю через `SQLHistoryRepository`.

Третий эндпоинт — `/execute` — служит для выполнения SQL-запросов, сгенерированных в предыдущем шаге или предоставленных вручную. Этот эндпоинт принимает запрос с текстом SQL-запроса и данными подключения к базе данных. С помощью сервиса `SqlExecutionService` выполняется сам SQL-запрос на базе данных, и результаты выполнения запроса возвращаются в виде списка, где каждый элемент представляет собой строку с результатами запроса в формате `Map<String, Object>`. В случае ошибок выполнения SQL-запроса также возвращается сообщение об ошибке.

Контроллер также активно использует механизмы безопасности `Spring Security` для аутентификации пользователей. После успешной генерации SQL-запроса данные о запросе сохраняются с привязкой к пользователю, который выполнил запрос, что обеспечивает персонализированную историю запросов. Этот механизм обеспечивается через аутентификацию `SecurityContextHolder`, которая позволяет извлечь текущего пользователя, идентифицировать его по имени (логину) и сохранить информацию в репозитории истории запросов.

4.2.3 Репозитории

Для обеспечения корректной работы приложения и сохранения информации о пользователях и их историях поиска было реализовано два репозитория. В

обоих репозиториях используется технология Spring Data JPA для автоматизации взаимодействия с базой данных.

SQLHistoryRepository.

SQLHistoryRepository является расширением интерфейса JpaRepository и предоставляет готовые методы для работы с данными. В SQLHistoryRepository реализованы методы, которые позволяют находить историю запросов по пользователю, или фильтровать историю по различным параметрам, таким как дата или текст.

Метод findByUser принимает на вход объект типа MyUser и возвращает список запросов под данного пользователя. Метод findByUserAndTextContaining принимает на вход объект типа MyUser и текст по которому необходимо отфильтровать запросы, в результате пользователь получит свою историю запросов, отфильтрованную по введенному тексту. Так же реализованы специальные методы, аннотированные @Query, например метод searchByTextAndDate позволяет пользователю отфильтровать историю запросов по дате и тексту. Эти методы обеспечивают удобство поиска и отображения истории запросов.

UserRepository.

Репозиторий UserRepository тоже является расширением JpaRepository, он предназначен для работы с сущностью пользователя. В нем реализованы методы необходимые для регистрации и авторизации. Метод findByLogin позволяет найти пользователя по его уникальному логину, а метод existsByLogin проверяет, есть ли в базе данных пользователь с таким логином.

4.2.4 Безопасность

Безопасность приложения обеспечивается с помощью технологии Spring Security, она предоставляет решения для безопасной авторизации и регистрации. В проекте используются несколько ключевых компонентов, включая CustomUserDetailsService для загрузки пользователя по логину, а также конфигурацию безопасности, настроенную в классе SecurityConfig.

CustomUserDetailsService.

Класс CustomUserDetailsService реализует интерфейс UserDetailsService из Spring Security. Этот сервис отвечает за извлечение данных о пользователе при аутентификации. В методе loadUserByUsername, который переопределен в классе, происходит попытка найти пользователя по логину в базе данных.

В случае, если пользователь с таким логином не существует, выбрасывается исключение `UsernameNotFoundException`. Логирование производится с помощью `Slf4j`, что позволяет отслеживать, был ли найден пользователь или нет. Если пользователь найден, он возвращается как объект `UserDetails`, который содержит всю необходимую информацию о пользователе.

SecurityConfig.

Класс `SecurityConfig` содержит конфигурацию безопасности приложения. Класс `securityFilterChain` настраивает фильтры безопасности для различных URL-адресов, и настраивает разрешение на доступ для различных маршрутов. Страницы входы и регистрации доступны без аутентификации, все остальные страницы требуют аутентификации. Это реализовано через метод `authorizeHttpRequests`, который позволяет задать разграничения доступа для различных адресов. Так же важный элемент безопасности - это механизм `rememberMe`. Он позволяет пользователю оставаться на странице без повторной аутентификации. В конфигурации задается уникальный ключ для cookie устанавливается максимальное время жизни cookie, а также гарантируется его использование только с защищенным соединением.

Для аутентификации через форму используется стандартный механизм Spring Security, настроенный в методе `formLogin`. В нем указаны URL-адреса для страницы входа и перенаправления после успешной аутентификации. Также добавляется конфигурация для страницы выхода: при выходе из системы происходит удаление cookies и завершение сессии, а пользователя перенаправляют на страницу входа с параметром `logout`.

Важной частью является настройка сессий. В методе `sessionManagement` указано, что сессии будут создаваться всегда, что означает, что сессия будет сохраняться, даже если не происходит активного взаимодействия с пользователем.

Кроме того, в классе настроен компонент для шифрования паролей с помощью `BCryptPasswordEncoder`. Этот компонент является стандартным механизмом для безопасного хранения паролей в базе данных. Пароли, перед сохранением, проходят через хеширование с использованием алгоритма `bcrypt`, что делает их нечитабельными и защищенными от кражи, даже если база данных будет скомпрометирована.

Наконец, класс `SecurityConfig` включает в себя настройку менеджера аутентификации через `AuthenticationManager`, который необходим для работы с механизмами аутентификации в Spring Security. Этот менеджер взаимодействует с системой

хранения данных о пользователях (например, базой данных) и использует сервис CustomUserDetailsService для загрузки данных о пользователях по логину.

Таким образом, безопасность проекта включает защиту данных пользователей, аутентификацию с использованием логина и пароля, а также защиту от CSRF-атак. Все эти меры обеспечивают безопасный доступ к приложению и защиту данных пользователей.

4.2.5 Сервисы

В проекте реализовано пять основных сервисов, которые выполняют различные функциональные задачи, связанные с пользователями и состоянием приложения.

AppService.

Сервис AppService является вспомогательным и выполняет задачи, связанные с управлением состоянием приложения и получением информации о текущем пользователе. Этот сервис инкапсулирует логику для отображения состояния приложения на основе данных, полученных от пользователя. Метод getModelAppState преобразует значение состояния приложения, представленное в виде перечисления AppState, в строковое представление. Для этого используется метод toString().toLowerCase(), который конвертирует состояние в строку в нижнем регистре, что удобно для передачи в модель. Метод setModelState добавляет атрибут в модель, которая используется для рендеринга представлений в веб-приложении. Этот атрибут называется appState - это строковое представление текущего состояния приложения. Метод getUser нужен для извлечения информации о текущем пользователе, в нем проводится проверка, что пользователь аутентифицирован.

AuthenticationService.

Этот сервис отвечает за регистрацию нового пользователя и проверку существования пользователя в системе. Метод UserExist проверяет, есть ли пользователь с данным логином в базе данных приложения. Метод возвращает true, если пользователь существует и false, если такого пользователя нет. Метод registerUser отвечает за регистрацию нового пользователя, он принимает сущность UserDto, которая содержит данные для регистрации. Пароль пользователя хешируется через PasswordEncoder, чтобы обеспечить безопасность хранения паролей. Пароль

сохраняется в объекте MyUser и сохраняется в базе данных через репозиторий пользователя.

DatabaseConnectionService.

Данный сервис отвечает за установление соединения с базой данных пользователя. Метод getConnection принимает параметры подключения (хост, порт, тип базы данных, имя базы данных, имя пользователя и пароль) и строит строку подключения на основе типа базы данных. Для различных типов баз данных используются различные форматы строки подключения. Метод возвращает объект Connection, который используется для выполнения SQL-запросов. Метод buildDatabaseUrl формирует строку подключения для разных типов баз данных. В зависимости от типа базы данных (например, postgresql, mysql, oracle), он строит строку подключения в соответствующем формате.

DatabaseSchemaService.

Данный сервис реализует извлечение схемы базы данных пользователя. Он использует соединение с базой данных, предоставленное сервисом DatabaseConnectionService. Метод getDatabaseSchema извлекает список таблиц, для каждой таблицы организует список атрибутов и связей, для этого выполняется несколько SQL-запросов. Сначала выполняется запрос для получения списка таблиц в схеме базы данных, затем для каждой таблицы выполняется запрос для получения ее столбцов, а также для получения внешних ключей между таблицами, включая их имена и соответствующие столбцы.

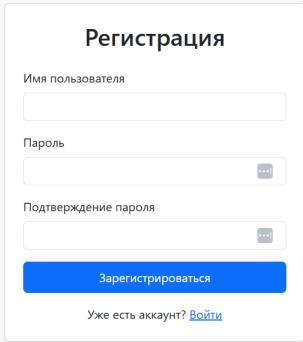
SqlExecutionService.

Данный сервис выполняет SQL-запрос к таблице пользователя и возвращает результат в формате таблицы. Метод executeQuery принимает на вход параметры подключения и SQL-запрос, возвращает результат в виде карт, каждая из которых представляет собой строку результатов, где ключом является название столбца, а значением соответствующие данные. Метод использует PreparedStatement для выполнения запроса и ResultSetMetaData для получения информации о столбцах результата. Этот метод позволяет гибко работать с результатами SQL-запросов, преобразуя их в список карт, что упрощает дальнейшую обработку данных в приложении.

4.2.6 Пользовательский интерфейс

Регистрация и авторизация.

Форма регистрации рис.4.18 включает в себя поля для ввода данных пользователя, это имя, email и пароль. Эти данные отправляются на сервер с помощью POST-запроса. В форме также есть валидация данных, чтобы убедиться, что email уникален, а пароль удовлетворяет минимальным требованиям безопасности (например, длина пароля не менее 6 символов). Когда пользователь нажимает кнопку "Зарегистрироваться" данные формы отправляются на сервер, где выполняется валидация. Если всё прошло успешно, создаётся новая запись в базе данных с данными пользователя, а также добавляется роль. После успешной регистрации пользователь перенаправляется на страницу входа. Если при регистрации возникли ошибки (например, email уже занят), пользователю показывается соответствующее сообщение об ошибке. Эти ошибки могут быть результатом проверок, выполненных на сервере.



The image shows a registration form titled "Регистрация". It contains three input fields: "Имя пользователя", "Пароль", and "Подтверждение пароля". The password fields have a small icon to toggle visibility. Below the fields is a blue button labeled "Зарегистрироваться" and a link "Уже есть аккаунт? Войти".

Рис.4.18. Форма регистрации.

Форма входа рис.4.19 включает два поля для ввода: email и пароль. Эти данные также отправляются на сервер с помощью POST-запроса. Валидация происходит на сервере, где проверяются введённые данные с теми, что хранятся в базе данных. Когда пользователь нажимает кнопку "Войти" введённые данные проверяются. Если пользователь существует и пароль верный, создаётся сессия пользователя с использованием Spring Security, и пользователь перенаправляется на главный экран. Если авторизация не удалась, показывается ошибка о неправильном email или пароле. Для защиты приложения от несанкционированного доступа используется Spring Security, который управляет процессом аутентификации и авторизации. Spring Security также обрабатывает редиректы в случае неудачной попытки входа.

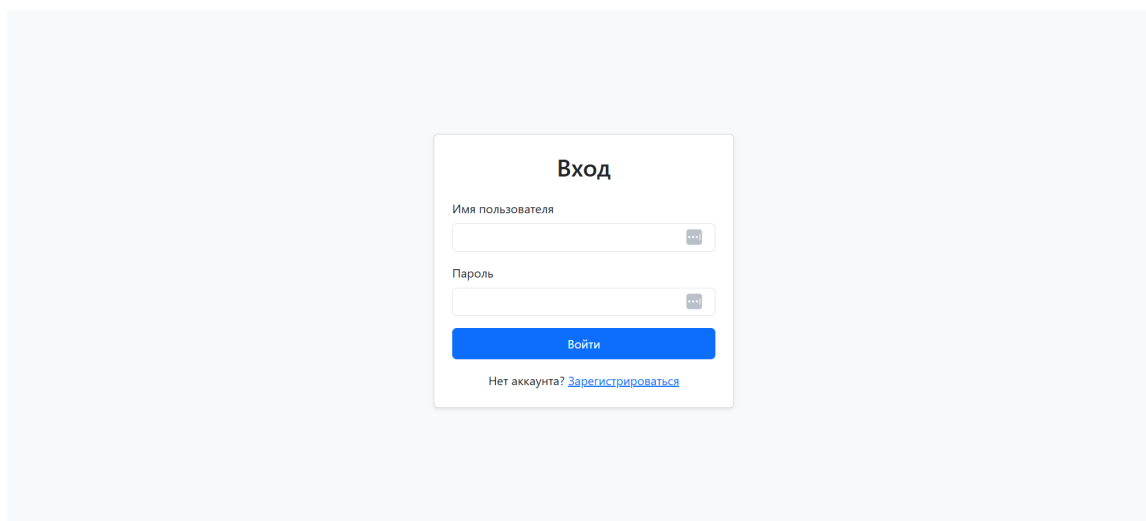


Рис.4.19. Форма авторизации.

Главный экран.

На главном экране реализованы следующие элементы:

1. Форма подключения к базе данных рис.4.20.

Форма подключения включает несколько полей для ввода информации: хост, порт, имя пользователя и пароль. Эти данные отправляются на сервер через POST-запрос для установления соединения с базой данных. Когда пользователь нажимает кнопку "Подключиться" данные из формы передаются в контроллер, который инициирует подключение к базе данных. Для этого используется JDBC или JPA для взаимодействия с базой данных. Если соединение удачное, пользователь получает сообщение о успешном подключении, а если произошла ошибка (например, неверные данные), отображается ошибка.

Рис.4.20. Форма подключения к базе данных пользователя.

2. Форма ввода запроса рис.4.21.

Поле ввода запроса на естественном языке - это текстовое поле, в котором пользователь может ввести запрос на естественном языке, например: "Выведи все записи из таблицы 'users'". Когда пользователь нажимает кнопку "Сгенерировать SQL" введенный текст отправляется в Python-сервис. Этот

сервис преобразует запрос в SQL, и сгенерированный SQL возвращается обратно на главный экран.

Генератор SQL

Опишите нужный запрос:

Сколько сотрудников работает в компании?

Сгенерировать SQL

Редактор SQL

SELECT count(*) FROM employees

Выполнить

Рис.4.21. Форма ввода запросы.

3. Выполнение запроса рис.4.22.

После того как запрос был сгенерирован, пользователь может нажать на кнопку "Выполнить SQL" чтобы выполнить запрос в базе данных. Запрос передаётся на сервер, который исполнил его и отобразил результат в виде таблицы. Когда запрос выполняется, результаты передаются обратно на экран и отображаются в таблице. Если запрос вызывает ошибку, например, из-за синтаксической ошибки, пользователю показывается сообщение с ошибкой выполнения. Результаты выполнения SQL-запроса отображаются в таблице на главном экране. Таблица динамически заполняется с помощью Thymeleaf, отображая полученные данные в виде строк и столбцов.

Редактор SQL

SELECT count(*) FROM employees

Выполнить

Результаты запроса

count
5

Найдено записей: 1

Рис.4.22. Результаты запроса.

Экран с историей запросов рис.4.23.

На экране с историей запросов пользователь может просматривать все ранее выполненные запросы.

Все запросы пользователя сохраняются в базе данных, и на экране отображается таблица, в которой каждый запрос представлен строкой с такими данными, как дата и время выполнения запроса, текст запроса, сгенерированный SQL и результат выполнения. Таблица заполняется данными из базы данных, с использованием Thymeleaf для рендеринга.

Когда пользователь заходит на экран с историей запросов, приложение выполняет запрос к базе данных и загружает все записи из таблицы истории. Эти данные передаются в модель и рендерятся на экране.

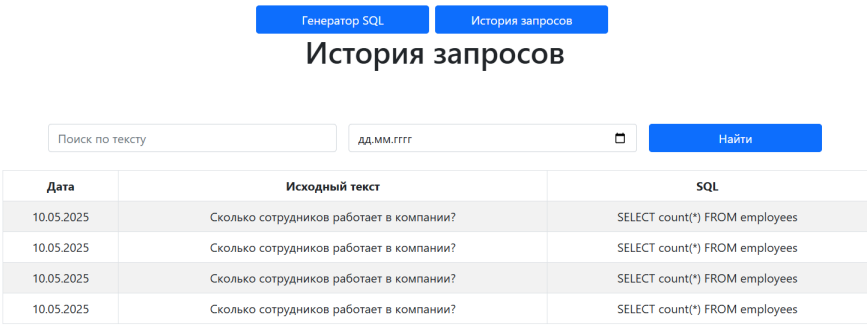


Рис.4.23. История запросова.

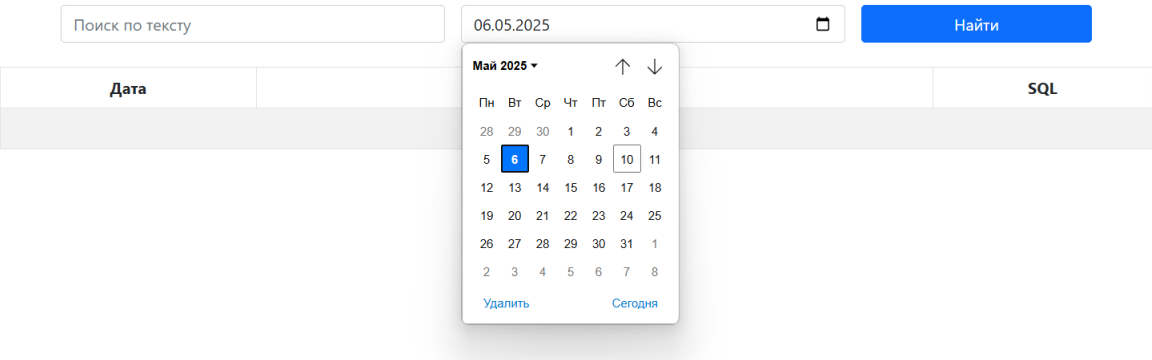


Рис.4.24. Выбор даты запроса.

4.3 Вывод

Основная сложность при повышении точности моделей LSTM и трансформеров для генерации SQL-запросов заключается в неоднозначности естественного языка и сложности правильного понимания структуры базы данных. Модели могут неправильно интерпретировать связь между таблицами, не учитывать правильные фильтры или не синтаксически корректно составить запрос. Также проблема возникает из-за того, что модели не имеют встроенной логики планирования SQL-запросов, они просто генерируют текст без учета правильной последовательности шагов (сначала таблицы, потом поля, затем условия). Ограниченность тренировочного набора данных и различия в языках могут дополнительно снижать точность.

В разработанном приложении после генерации запроса, пользователь имеет возможно скорректировать его, под свои требования и исправить возможные ошибки модели. В данном приложении модель служит помощником в написании запросов, предлагая шаблон, который может быть модифицирован.

В данной главе было реализовано полноценное приложение с веб-интерфейсом, которое предоставляет пользователю возможность генерировать запросы, выполнять их и отображать результат внутри приложения.

Важным элементом является интеграция с Spring Security, что гарантирует безопасность, разграничивая права доступа для авторизованных и не авторизованных пользователей.

5 ТЕСТИРОВАНИЕ И АПРОБАЦИЯ ИНСТРУМЕНТА ДЛЯ ПЕРЕВОДА ЕСТЕСТВЕННОГО ЯЗЫКА В SQL-КОД

5.1 Функциональное тестирование

В данной работе было проведено ручное функциональное тестирование с целью проверки работы ключевых функций системы. Тестировались различные сценарии, включая проверку регистрации пользователей, выполнения SQL-запросов и обработки ошибок. Особое внимание уделялось функциональности, требующей взаимодействия с интерфейсом, а также выявлению багов и несоответствий в работе системы при реальном использовании.

Для ручного функционального тестирования данного веб-приложения составлены следующие сценарии. Каждый сценарий описывает конкретные действия, ожидаемые результаты и возможные ошибки, которые следует учитывать при проведении тестирования.

Сценарий 1: Аутентификация зарегистрированного пользователя.

- Открыть страницу входа.
- Ввести зарегистрированный логин и пароль.
- Нажать кнопку «Войти».

Рис.5.1. Аутентификация зарегистрированного пользователя.

Появляется главный экран приложения с возможностью подключения к базе данных рис.5.1.

Пользователь успешно аутентифицирован и перенаправлен на главный экран.

Сценарий 2: Аутентификация незарегистрированного пользователя.

- Открыть страницу входа.
- Ввести незарегистрированный логин и пароль.
- Нажать кнопку «Войти».

Появляется сообщение о том, что пользователь не найден или введены неверные данные рис.5.2.

На экране остаются поля для ввода данных.

Сценарий 3: Ввод некорректного пароля.

- Открыть страницу входа.
- Ввести корректный логин, но некорректный пароль.
- Нажать кнопку «Войти».

Появляется сообщение об ошибке: «Неверный логин или пароль» рис.5.3.

Поли ввода остаются для повторной попытки.

Сценарий 4: Регистрация нового пользователя.

- Перейти на страницу регистрации.

The screenshot shows a login form with the title "Вход". At the top, a red error message box contains the text "Неверное имя пользователя или пароль". Below this, there are two input fields: "Имя пользователя" and "Пароль", each with a password toggle icon. A blue "Войти" button is positioned below the password field. At the bottom, there is a link that says "Нет аккаунта? [Зарегистрироваться](#)".

Рис.5.2. Аутентификация незарегистрированного пользователя.

This screenshot is identical to the one in Figure 5.2, showing the same login form with the error message "Неверное имя пользователя или пароль" and the "Войти" button.

Рис.5.3. Ввод некорректного пароля.

- Ввести корректные данные для нового пользователя (логин, пароль, подтверждение пароля).
 - Нажать кнопку «Зарегистрироваться».
- Пользователь успешно зарегистрирован.

Зарегистрироваться' (No account? [Register](#))." data-bbox="278 63 804 401"/>

Рис.5.4. Регистрация нового пользователя.

На экран выводится сообщение о успешной регистрации, и происходит редирект на страницу входа рис.5.4.

Сценарий 5: Ввод уже существующего логина при регистрации.

- Перейти на страницу регистрации.
- Ввести логин, который уже существует в базе данных, и попытаться зарегистрироваться.

Появляется сообщение об ошибке: «Пользователь с таким именем уже существует» рис.??.

Пользователь остается на странице регистрации с возможностью ввести новый логин.

Сценарий 6: Ввод некорректного пароля при регистрации.

- Перейти на страницу регистрации.
- Ввести логин и пароль, которые не соответствуют требованиям (например, слишком короткий пароль или несоответствие пароля и подтверждения).
- Нажать кнопку «Зарегистрироваться».

Появляется сообщение об ошибке с указанием, что введенные данные некорректны рис.5.6.

Поли ввода остаются активными, и пользователю предоставляется возможность исправить ошибки.

Регистрация

Имя пользователя

Мария

Пользователь с таким именем уже существует

Пароль

Пароль должен быть не менее 6 символов

Подтверждение пароля

Зарегистрироваться

Уже есть аккаунт? [Войти](#)

Рис.5.5. Ввод уже существующего логина при регистрации.

Сценарий 7: Главный экран — подключение к базе данных с некорректными данными.

- На главном экране ввести некорректные данные для подключения к базе данных (неправильный хост, порт, имя базы данных, логин или пароль).
- Нажать кнопку «Подключиться».

Появляется сообщение об ошибке: «Ошибка подключения» рис.5.7.

Поля ввода остаются активными, чтобы можно было ввести правильные данные.

Сценарий 8: Написание запроса на естественном языке без соединения с базой данных.

- На главном экране ввести запрос на естественном языке, например: «Вывести все записи из таблицы users».
- Нажать кнопку «Выполнить».

Появляется сообщение об ошибке: «Ошибка подключения» рис.5.8.

Поля ввода остаются активными, и пользователь может повторить попытку после установления соединения.

Сценарий 9: Ввод пустого текста в поле запроса.

- На главном экране оставить поле ввода запроса пустым.
- Нажать кнопку «Выполнить».

The image shows a registration form with the title "Регистрация". It contains three input fields: "Имя пользователя" (Username) with the value "kirill", "Пароль" (Password), and "Подтверждение пароля" (Confirm password). The password field has a red error message below it: "Пароль должен быть не менее 6 символов" (Password must be at least 6 characters). The confirm password field has a red error message below it: "Пароли не совпадают" (Passwords do not match). At the bottom, there is a blue button labeled "Зарегистрироваться" (Register) and a link "Уже есть аккаунт? Войти" (Already have an account? Log in).

Рис.5.6. Ввод некорректного пароля при регистрации.

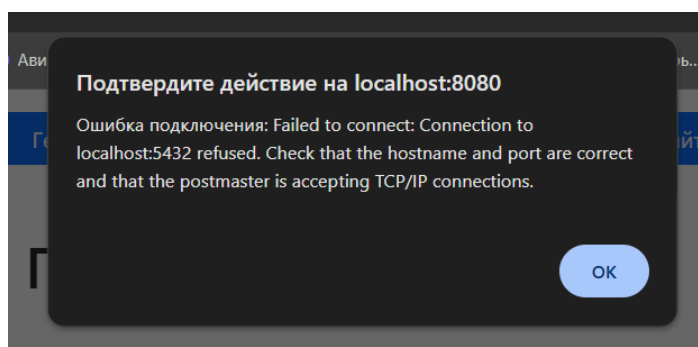


Рис.5.7. Главный экран — подключение к базе данных с некорректными данными.

Появляется сообщение: «Введите описание запроса» рис.5.9.

Поля ввода остаются активными для ввода корректного запроса.

Сценарий 10: Ввод потенциально опасных запросов.

- Ввести в поле запроса команду, например, DELETE FROM users или INSERT INTO users VALUES (1, 'admin').
- Нажать кнопку «Выполнить».

Появляется сообщение о запрете выполнения потенциально опасных запросов рис.5.10.

Запрос не выполняется, и пользователь уведомляется о небезопасности данной операции.

Сценарий 11: Выполнение некорректного SQL-запроса.

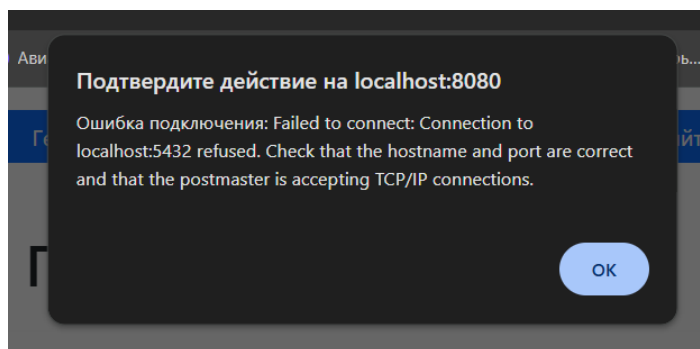


Рис.5.8. Написание запроса на естественном языке без соединения с базой данных.

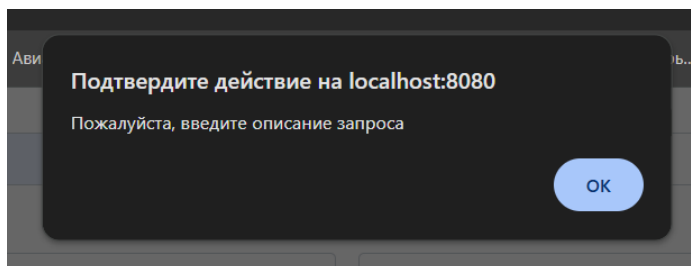


Рис.5.9. Ввод пустого текста в поле запроса.

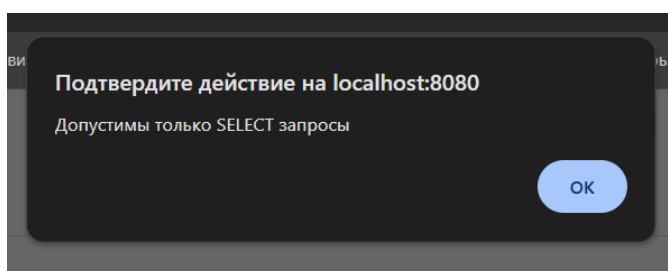


Рис.5.10. Ввод потенциально опасных запросов..

- Ввести некорректный SQL-запрос, например, синтаксическую ошибку или ошибку в структуре запроса.
- Нажать кнопку «Выполнить».

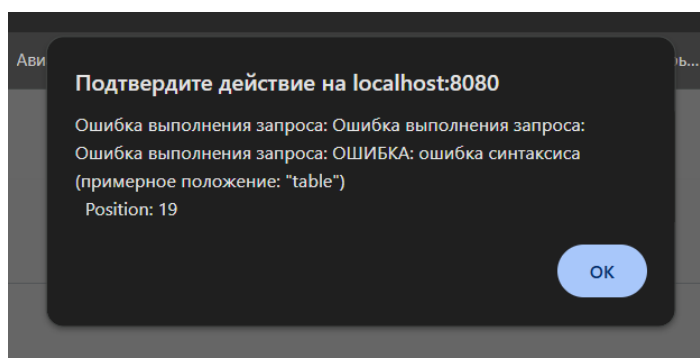


Рис.5.11. Выполнение некорректного SQL-запроса.

Появляется сообщение об ошибке: «Ошибка выполнения запроса» с указанием ошибки рис.5.11.

Запрос не выполняется, и пользователь может попробовать исправить ошибку

Сценарий 12: Отображение истории запросов.

- Перейти на вкладку истории запросов.
- Просмотреть список выполненных запросов.

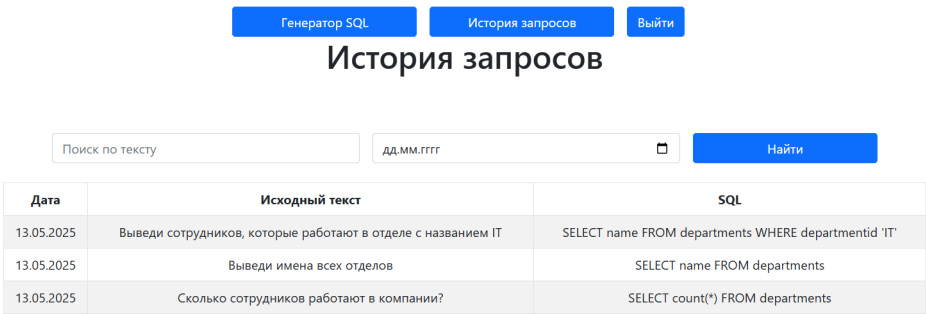


Рис.5.12. Отображение истории запросов.

История запросов отображается корректно с метками времени и результатами выполнения запросов рис.5.12.

Сценарий 13: Фильтрация истории запросов по дате.

- Перейти на вкладку истории запросов.
- Ввести фильтр по дате, указав дату в которую был выполнен запрос.
- Нажать кнопку фильтрации.

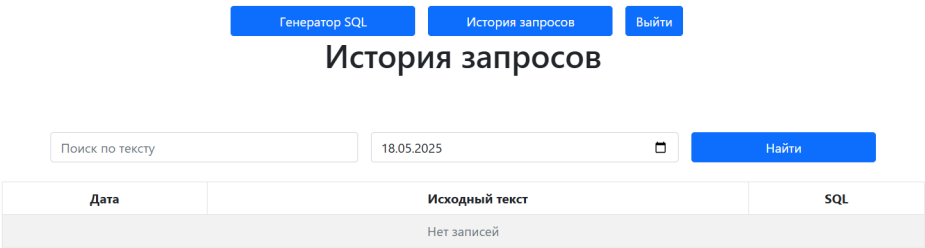


Рис.5.13. Фильтрация истории запросов по дате.

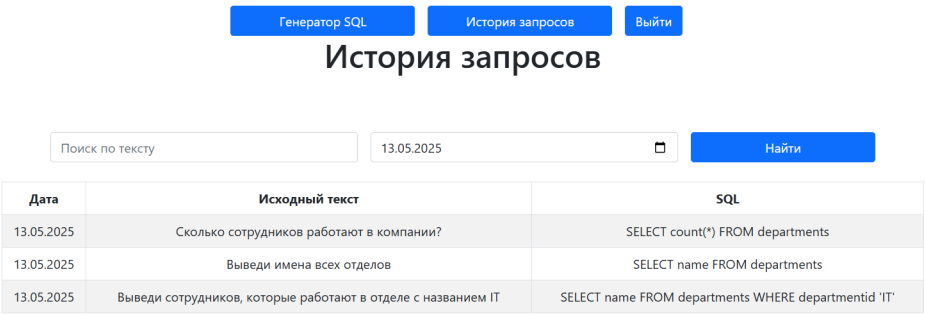


Рис.5.14. Фильтрация истории запросов по дате.

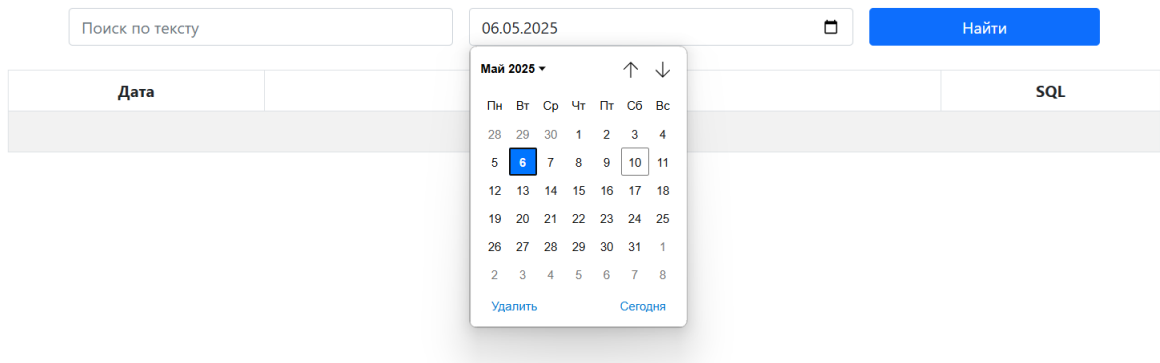


Рис.5.15. Окно выбора даты.

История запросов фильтруется в соответствии с указанными параметрами рис.5.13 и рис.5.14. Дату можно выбрать в специальном всплывающем окне рис.5.15.

Отображаются только запросы, выполненные в указанную дату.

Сценарий 14: Фильтрация истории запросов по тексту.

- Перейти на вкладку истории запросов.
- Ввести текст для поиска в истории.
- Нажать кнопку фильтрации.

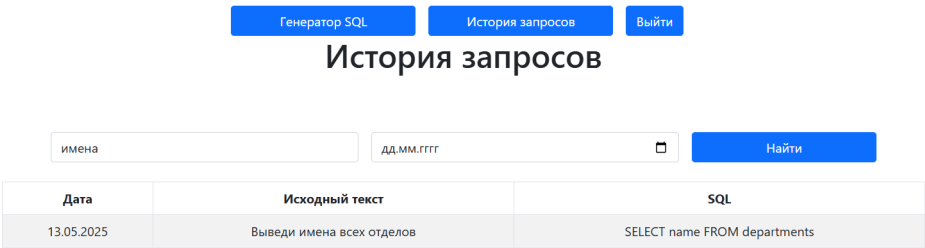


Рис.5.16. Фильтрация истории запросов по тексту.

История запросов фильтруется, и отображаются только те запросы, которые содержат введенный текст рис.5.16.

Сценарий 15: Выход из системы.

- Нажать кнопку «Выход» в интерфейсе приложения.

Пользователь перенаправляется на страницу входа рис.5.17.

Зарегистрироваться' (No account? [Register](#))." data-bbox="358 98 711 326"/>

Рис.5.17. Выход из системы.

5.2 Модульное тестирование

5.2.1 Тестирование модуля перевода

Модульное тестирование представляет собой неотъемлемую часть процесса разработки интеллектуальных программных систем, в том числе тех, что связаны с обработкой естественного языка. Под данным видом тестирования понимается проверка корректности функционирования отдельных компонентов программного обеспечения в изоляции от других модулей. Его основная цель — своевременное выявление логических ошибок, отклонений от требований и несоответствий в поведении отдельных функций.

В рамках разработки модуля генерации SQL-запросов из текстовых вопросов на естественном языке были протестированы ключевые составляющие архитектуры. Среди них — механизмы предобработки входных данных, корректность токенизации и детокенизации, стабильность работы модели при генерации запросов, правильность вычисления функции потерь и метрики качества. Особое внимание было уделено метрике BLEU, как одной из наиболее показательных при оценке качества генерации последовательностей, а также динамике изменения значений функции потерь на обучающей и валидационной выборках.

Экспериментальное тестирование модуля осуществлялось на заранее подготовленной выборке, содержащей пары вида «вопрос на естественном языке — соответствующий SQL-запрос». Обучение модели проводилось в течение 7 эпох

с сохранением промежуточных значений функции потерь и метрики BLEU. Для обеспечения объективности оценки производительности, выборка была разделена на тренировочную и валидационную части, что позволило контролировать способность модели к обобщению и выявлять признаки переобучения.

Каждая эпоха сопровождалась вычислением средней функции потерь на соответствующей выборке рис.5.18, а также подсчётом BLEU-оценки рис.5.19 по сгенерированным запросам. Накопленные значения использовались для визуального анализа прогресса обучения модели.



Рис.5.18. Динамика функции потерь.

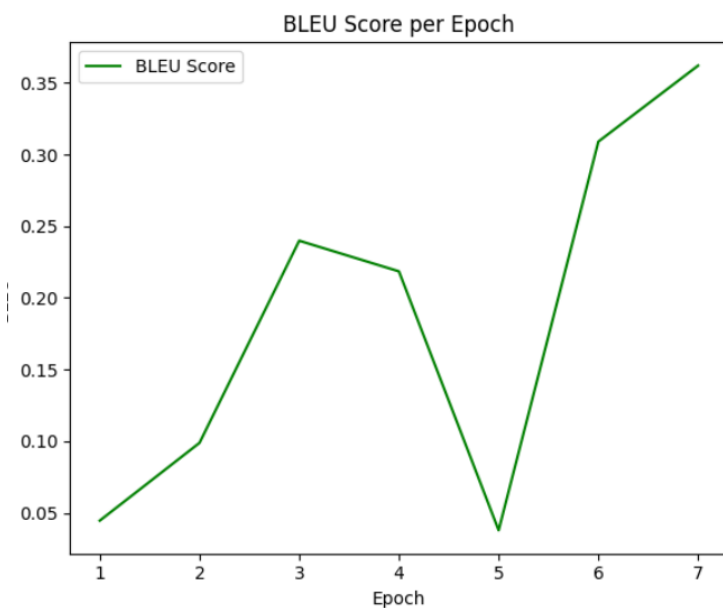


Рис.5.19. Динамика оценки blue.

На графике динамики функции потерь видно, что в течение первых трёх эпох происходит быстрое снижение как обучающей, так и валидационной ошибки. Это указывает на то, что модель достаточно быстро адаптировалась к обучающим данным и усвоила базовые синтаксические паттерны построения SQL-запросов.

Однако на четвёртой и особенно на пятой эпохе наблюдается незначительное увеличение значений функции потерь, особенно на тренировочной выборке. Такой всплеск может свидетельствовать о временном переобучении или переходной фазе в обучении, связанной с адаптацией модели к более сложным структурам или примерам. После этого значения вновь начинают снижаться, достигая к седьмой эпохе минимального уровня — 0.0786 на тренировочной и 0.0791 на валидационной выборке соответственно. Близость этих двух значений свидетельствует о хорошем обобщающем потенциале модели и отсутствии явного переобучения.

Отдельное внимание следует уделить динамике BLEU-оценки, отражающей качество сгенерированных SQL-запросов относительно эталонных. Начиная с первого значения около 0.045, BLEU стабильно растёт до третьей эпохи, достигая значения 0.24. Далее, как и в случае с функцией потерь, наблюдается временное снижение, после чего метрика демонстрирует уверенный рост, достигая значения 0.3620 на последней эпохе. Такой рост может быть интерпретирован как подтверждение того, что модель обучилась не только синтаксически допустимым, но и семантически близким к целевым SQL-запросам структурам.

Наблюдаемый рост качества генерации SQL-запросов, измеряемый BLEU-метрикой, коррелирует с постепенным снижением значения функции потерь. Анализ графиков позволяет также сделать вывод о том, что модель способна обобщать информацию, а не просто запоминать обучающие примеры. Это подтверждается стабильным поведением на валидационной выборке и высоким финальным BLEU-показателем.

Надёжность и точность работы модуля генерации SQL-запросов во многом зависит не только от качества самой модели, но и от корректного функционирования предварительных этапов обработки данных.

В процессе тестирования разработанного модуля генерации SQL-запросов на основе вопросов на естественном языке были реализованы различные типы модульных тестов, обеспечивающих всестороннюю проверку ключевых компонентов системы.

Tokenization and Detokenization Test — этот тест проверяет корректность процесса токенизации и обратной детокенизации. Целью является убедиться, что

```

test_full_pipeline (__main__.TestPreprocessingAndTokenization.test_full_pipeline) ... ok
test_preprocessing (__main__.TestPreprocessingAndTokenization.test_preprocessing) ... ok
test_tokenization_and_detokenization (__main__.TestPreprocessingAndTokenization.test_tokenization_and_detokenization) ... ok
test_bleu_score_high (__main__.TestSQLModule.test_bleu_score_high)
Оценка BLEU: совпадение вывода с эталоном должно давать высокий балл ... ok
test_bleu_score_low (__main__.TestSQLModule.test_bleu_score_low)
Оценка BLEU: неполное совпадение должно давать низкий балл ... ok
test_decode_consistency (__main__.TestSQLModule.test_decode_consistency)
Проверка, что повторное декодирование не приводит к пустой строке ... ok
test_empty_input (__main__.TestSQLModule.test_empty_input)
Проверка реакции на пустой ввод ... ok
test_known_input_1 (__main__.TestSQLModule.test_known_input_1)
Проверка корректности генерации для запроса по возрасту ... ok
test_token_ids_validity (__main__.TestSQLModule.test_token_ids_validity)
Проверка, что токены преобразуются в допустимые ID и обратно ... ok
test_tokenization_and_detokenization (__main__.TestSQLModule.test_tokenization_and_detokenization)
Проверка: токенизация и детокенизация не теряет смысл ... ok

-----
Ran 10 tests in 0.026s

OK
<unittest.main.TestProgram at 0x7993c20ff050>

```

Рис.5.20. Результаты тестов.

после преобразования текста в токены и обратно смысл исходного предложения сохраняется. Это важно для надёжной работы модели, особенно при работе с русским языком.

Empty Input Test проверяет устойчивость модели к пустому или некорректному вводу, модель должна вернуть безопасный универсальный SQL-запрос.

Known Input Tests проверяют корректность генерации SQL-запросов для заранее известных формулировок, например запросы по возрасту или оценке. Эти тесты позволяют убедиться, что система правильно интерпретирует семантику вопроса и воспроизводит корректный SQL.

BLEU Score Tests проверяют корректность вычисления оценки, то есть при полном совпадении двух запросов, оценка должна быть высокой, а при расхождениях, оценка снижается.

Token ID Validity Test проверяет, что результат токенизации - это допустимые целочисленные идентификаторы, соответствующие модели. Это подтверждает корректность работы с входными данными.

Decode Consistency Test служит для проверки, что процесс декодирования никогда не приводит к пустому результату. Это важно для надёжного вывода модели и работы пользовательского интерфейса.

Все тесты рис.5.20 в совокупности обеспечивают проверку как функциональной корректности, так и устойчивости модуля, подтверждая его готовность к использованию в рамках интеллектуальных систем обработки запросов.

5.2.2 Тестирование репозитория.

Класс UserRepositoryTest

Тест `testFindByLogin` в классе `UserRepositoryTest` предназначен для проверки корректности работы метода `findByLogin`, который должен находить пользователя по логину. Такой функционал критически важен для аутентификации и авторизации в приложении. Тест использует библиотеку Mockito для имитации поведения `userRepository`. Сначала создается объект `MyUser` с логином `"testUser"`. Затем с помощью `when(...).thenReturn(...)` задается поведение мока: при вызове `findByLogin("testUser")` он должен вернуть `Optional`, содержащий тестового пользователя. После этого метод вызывается, результат сохраняется и с помощью `assertTrue` проверяется, что объект действительно найден, а через `assertEquals` — что логин пользователя соответствует ожидаемому значению. Этот тест гарантирует, что поиск пользователя по логину работает так, как задумано, и возвращает корректный результат.

Тест `testExistsByLogin` проверяет другой важный метод — `existsByLogin`, который используется для проверки, зарегистрирован ли уже пользователь с данным логином. Это особенно важно при регистрации, чтобы избежать создания дубликатов. Здесь также используется Mockito, чтобы задать поведение мока: метод должен вернуть `true`, если пользователь с логином `"testUser"` существует. Затем вызывается сам метод, результат сохраняется и проверяется с помощью `assertTrue`, что результат действительно логически истинный. Таким образом, этот тест обеспечивает надёжную проверку наличия пользователя в базе по логину.

Класс `SQLHistoryRepositoryTest`

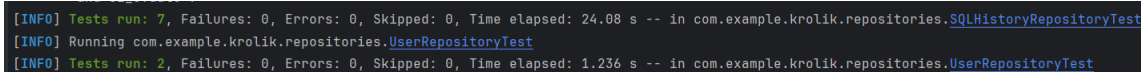
Класс `SQLHistoryRepositoryTest` представляет собой модульный тестовый класс для проверки работы репозитория `SQLHistoryRepository`, который отвечает за доступ к данным истории SQL-запросов.

В тесте явно указаны параметры подключения к тестовой базе данных PostgreSQL, которая, судя по всему, работает в Docker-контейнере с именем `postgres`. Используется база `auto_sql`, а Hibernate настраивается так, чтобы при запуске тестов автоматически создавать и удалять схему базы (`ddl-auto=create-drop`). Это означает, что каждый тест выполняется на свежей, «чистой» структуре таблиц, что исключает влияние предыдущих тестов на текущий. Также активирован вывод SQL-запросов и привязки параметров в логах, что помогает при отладке.

Перед выполнением каждого теста в методе `setup()` очищаются все записи из таблиц `sqlhistory` и `my_user` (через `deleteAll`), чтобы избежать загрязнения тестов. Далее создается тестовый пользователь, сохраняется в базу данных, создаются три записи истории запросов для этого пользователя.

Реализовано несколько тестов, каждый из них проверяет отдельные методы репозитория.

Тест `testFindByUser()` проверяет, что метод `findByUser()` возвращает все запросы, которые были сделаны конкретным пользователем. Тест `testFindByUserOrderByIdDesc()` проверяет, что данные верно фильтруются и сортируются. Другие тесты проверяют фильтрацию по тексту, по дате или одновременно по тексту и дате.



```
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 24.08 s -- in com.example.krolik.repositories.SQLHistoryRepositoryTest
[INFO] Running com.example.krolik.repositories.UserRepositoryTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.236 s -- in com.example.krolik.repositories.UserRepositoryTest
```

Рис.5.21. Результаты тестов.

5.2.3 Тестирование сервисов.

Класс `AuthenticationServiceTest`.

Модульные тесты класса `AuthenticationServiceTest` предназначены для проверки логики, связанной с регистрацией и проверкой существования пользователей в приложении. Эти тесты обеспечивают изоляцию логики сервиса от внешних компонентов и позволяют удостовериться в корректной работе методов `userExists` и `registerUser`.

Тест `userExists_shouldReturnTrue_whenUserExists` проверяет поведение метода `userExists` при условии, что пользователь с заданным логином уже существует в базе данных. Для этого с помощью Mockito задается поведение мока `userRepository.findByLogin(...)`, который возвращает `Optional` с объектом `MyUser`. После вызова метода `authenticationService.userExists(...)` результат сохраняется в переменную и проверяется с помощью `assertTrue`, что метод вернул `true`. Также с помощью `verify` проверяется, что метод репозитория был вызван ровно один раз. Этот тест необходим, чтобы убедиться, что логика распознавания существующего пользователя работает корректно и не приводит к ошибкам.

Тест `userExists_shouldReturnFalse_whenUserDoesNotExist` проверяет противоположную ситуацию: пользователь с заданным логином отсутствует в базе. Здесь `userRepository.findByLogin(...)` возвращает `Optional.empty()`, после чего проверяется, что `userExists` возвращает `false`. Также с помощью `verify` контролируется количество обращений к репозиторию. Этот тест позволяет удостовериться, что система корректно распознаёт несуществующего пользователя, что критично, например, при регистрации новых аккаунтов.

Тест `registerUser_shouldSaveUser_whenValidData` проверяет успешную регистрацию пользователя, если такого логина еще не существует. В данном случае не задается поведение `userRepository.findByLogin(...)`, а сразу мокается поведение `passwordEncoder.encode(...)`, который возвращает заранее определённую закодированную строку — `"encodedPassword"`. Далее вызывается метод `registerUser`, который должен создать нового пользователя, зашифровать его пароль и сохранить его в репозиторий. В конце теста с помощью `verify` проверяется, что метод `save` репозитория был вызван один раз с любым объектом класса `MyUser`. Это гарантирует, что процесс регистрации завершился успешно и пользователь был сохранён в базу данных.

Тест `registerUser_shouldThrowException_whenUserExists` предназначен для проверки обработки ошибки при попытке зарегистрировать пользователя с уже существующим логином. Здесь `userRepository.findByLogin(...)` возвращает `Optional` с пользователем, имитируя ситуацию дубликата логина. При вызове метода `registerUser(...)` ожидается выбрасывание исключения `RuntimeException` с сообщением "Пользователь с таким именем уже существует что и проверяется с помощью `assertThrows`. Дополнительно `verify(..., never())` удостоверяется, что метод `save` репозитория не был вызван, поскольку регистрация не должна происходить при конфликте логина. Этот тест важен для обеспечения корректной валидации данных на уровне бизнес-логики и предотвращения дублирования пользователей.

Класс `DatabaseConnectionServiceTest`.

тесты класса `DatabaseConnectionServiceTest` проверяют поведение сервиса подключения к базе данных, а именно корректность генерации URL и обработку исключений при попытке установить соединение с базой данных. Этот набор тестов охватывает как внутреннюю логику построения URL, так и обработку ошибок при подключении.

В тесте `buildDatabaseUrl_shouldReturnCorrectPostgresUrl` проверяется, что метод `buildDatabaseUrl`, принимающий тип базы данных, хост, порт и имя базы, корректно формирует строку подключения для PostgreSQL. Так как этот метод является `private`, он вызывается через Java Reflection API. Это позволяет протестировать внутреннюю бизнес-логику, не вынося метод в публичный интерфейс. В тесте передаются параметры `"postgresql localhost 5432, "mydb"` и ожидается, что будет возвращена строка `"jdbc:postgresql://localhost:5432/mydb"`.

Аналогично, в тесте `buildDatabaseUrl_shouldReturnCorrectMySQLUrl` проверяется, что URL для подключения к MySQL строится корректно. Пара-

метры "mysql "127.0.0.1 3306, "testdb" должны привести к генерации строки "jdbc:mysql://127.0.0.1:3306/testdb". Этот тест помогает удостовериться, что сервис корректно поддерживает различные типы баз данных и формирует строки подключения в соответствии с JDBC-форматом.

Тест `buildDatabaseUrl_shouldThrowExceptionForUnsupportedType` проверяет обработку ошибки при передаче неподдерживаемого типа базы данных. Здесь используется тип "sqlite" который не поддерживается. Через `Reflection` вызывается тот же приватный метод `buildDatabaseUrl`, и ожидается, что будет выброшено исключение `IllegalArgumentException` с сообщением "Unsupported DB type: sqlite".

Тест `getConnection_shouldThrowSQLException_whenInvalidCredentials` проверяет реакцию метода `getConnection` на некорректные данные для подключения к базе данных пользователя. На вход поступает несуществующие данные для подключения и ожидается, что будет выброшено исключение `SQLException`. В случае ввода некорректных данных метод не должен приводить к сбою работы системы.

Класс `DatabaseSchemaServiceTest`.

В представленном классе тестов `DatabaseSchemaServiceTest` проверяется функциональность класса `DatabaseSchemaService`, который используется для получения схемы базы данных. Тесты охватывают два сценария: отсутствие таблиц в базе данных и успешное извлечение схемы базы данных с таблицами, колонками и внешними ключами. Тестирование осуществляется с использованием моков для имитации работы с базой данных через JDBC.

Тест `getDatabaseSchema_shouldReturnEmptyString_whenNoTablesFound` проверяет, что метод `getDatabaseSchema` возвращает пустую строку в случае, если в базе данных нет таблиц. В этом тесте мокаются соединение с базой данных и запрос, который возвращает пустой результат. Конкретно, когда выполняется запрос для получения таблиц из схемы `public`, метод `next()` возвращает `false`, что указывает на отсутствие таблиц. В результате метод `getDatabaseSchema` должен вернуть пустую строку, что подтверждается ассертом `assertEquals(, result)`.

Тест `getDatabaseSchema_shouldReturnSchemaWithTablesColumnsAndFKs` проверяет, что метод `getDatabaseSchema` корректно извлекает схему базы данных, включая таблицы, их колонки и внешние ключи. В этом тесте создается более сложная настройка моков: сначала мокаются таблицы, затем колонки каждой таблицы, включая запросы для получения информации о внешних ключах и их связях. В тесте имитируется, что база данных содержит таблицу `users` с колонками

id и name, а также внешним ключом, связывающим department_id в таблице users с колонкой id в таблице departments.

В тесте используются моки для работы с ResultSet и PreparedStatement, что позволяет избежать реального взаимодействия с базой данных. Эти моки настроены так, чтобы возвращать нужные данные, например, возвращение имени таблицы из схемы или списка колонок, а также симуляция наличия внешнего ключа, который связывает таблицы. Затем, при вызове метода getDatabaseSchema, проверяется, что возвращенная строка включает информацию о таблице users с колонками и внешних ключах, что подтверждается акцептом assertTrue(result.contains("Table: users (id, name);")) и assertTrue(result.contains("Foreign Keys: users.department_id -> departments.id")).

Класс SqlExecutionServiceTest.

В классе тестов SqlExecutionServiceTest проверяется функциональность сервиса SqlExecutionService, который отвечает за выполнение SQL-запросов и обработку результатов. Тесты охватывают два основных случая: успешное выполнение запроса и обработку ошибок, например, когда возникает ошибка соединения.

Первый тест testExecuteQuery_ReturnsExpectedResults проверяет успешное выполнение SQL-запроса с возвращением правильных результатов. Для этого создаются моки для всех необходимых объектов: сервис соединения с базой данных, подключение, подготовленный запрос (PreparedStatement), результат запроса (ResultSet) и метаданные результата (ResultSetMetaData). Эти моки настраиваются так, чтобы имитировать успешное выполнение SQL-запроса, который возвращает два столбца (id и name) для двух пользователей. Затем вызывается метод executeQuery, и проверяется, что он возвращает правильные данные для этих пользователей, а также что все вызовы методов моков были сделаны в нужном количестве и с правильными параметрами.

Второй тест testExecuteQuery_ThrowsSQLException проверяет, что при возникновении ошибки соединения с базой данных (например, при неправильных данных для подключения) метод выбрасывает исключение SQLException. Для этого мокируется сервис соединения с базой данных так, чтобы он выбрасывал исключение при попытке получить соединение. Тест затем проверяет, что при вызове метода executeQuery с этим исключением корректно выбрасывается ошибка, и что в сообщении об ошибке содержится текст "Ошибка выполнения запроса подтверждающий правильную обработку ошибки в сервисе."

Класс UserRepositoryTest

```
[INFO] Running com.example.krolik.services.DatabaseSchemaServiceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.434 s -- in com.example.krolik.services.DatabaseSchemaServiceTest
[INFO] Running com.example.krolik.services.DatabaseConnectionServiceTest
Connecting to database with URL: jdbc:postgresql://localhost:5432/nonexistentdb
Using username: wronguser
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.071 s -- in com.example.krolik.services.DatabaseConnectionServiceTest
[INFO] Running com.example.krolik.services.AuthenticationServiceTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.199 s -- in com.example.krolik.services.AuthenticationServiceTest
[INFO] Running com.example.krolik.services.SqlExecutionServiceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.190 s -- in com.example.krolik.services.SqlExecutionServiceTest
[INFO] Running com.example.krolik.services.AppServiceTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.488 s -- in com.example.krolik.services.AppServiceTest
```

Рис.5.22. Результаты тестов.

5.2.4 Тестирование контроллеров.

Класс MainControllerTest

Тесты для метода `getHomePage` проверяют, что при запросе главной страницы возвращается корректное представление. В тесте используется `MockMvc` для имитации GET-запроса на главную страницу, и проверяется, что возвращается правильное представление ("home"). Важной частью теста является проверка, что состояние модели устанавливается в `AppState.MAIN`. Это достигается с помощью моков для сервиса `AppService`, который управляет состояниями модели. В конце теста проверяется, что возвращаемый HTTP-статус равен 200.

Тесты для метода `getHistoryPage` проверяют, что при передаче параметров поиска система корректно фильтрует историю запросов. Для этого создаются несколько тестов с разными параметрами: текст поиска и дата. В случае, если параметры не переданы, тест проверяет, что возвращается вся история запросов текущего пользователя. Для работы с репозиторием истории запросов используется мок `SQLHistoryRepository`. Также проверяется, что в модель добавляются правильные данные, такие как текст и дата поиска, и что в ответе отображаются нужные записи истории.

Для проверки работы без параметров поиска создается отдельный тест, который имитирует запрос без параметров, и проверяется, что вся история запросов отображается без фильтрации.

Класс AuthenticationControllerTest

В данном тестовом классе проводятся функциональные тесты для контроллера аутентификации `AuthenticationController`, который отвечает за обработку запросов, связанных с аутентификацией и регистрацией пользователей. Тесты используют фреймворк `Spring Boot` и библиотеку `MockMvc` для имитации HTTP-запросов и проверки правильности ответов от контроллера.

Первый тест, `loginPage_shouldReturnLoginView`, проверяет, что при обращении к URL `/login` контроллер возвращает страницу с именем представления `login`

и статусом HTTP 200 (OK). Это тестирует корректность отображения страницы входа пользователя. Второй тест, `logout_shouldRedirectToLoginWithLogoutParam`, проверяет, что при обращении к URL `/logout` происходит перенаправление на страницу входа с параметром `logout`, что подтверждает правильность выполнения операции выхода и перенаправления. Тест с именем `registerPage_shouldReturnRegisterViewAndModel` проверяет, что при запросе страницы регистрации по URL `/register` возвращается представление `register`, и модель, передаваемая в представление, содержит атрибут `userDto`. Этот тест гарантирует, что форма регистрации корректно инициализируется перед отправкой данных.

Следующий тест, `registerUser_shouldRedirectWhenValid`, проверяет успешную регистрацию пользователя, если введенные данные валидны. В этом случае, после выполнения POST-запроса на регистрацию с параметрами логина и пароля, система должна выполнить проверку, что пользователя не существует в базе данных, а затем перенаправить на страницу входа с параметром `registered`. В тесте используется заглушка для метода `userExists` сервиса `AuthenticationService`, который имитирует отсутствие пользователя в базе данных. После успешной регистрации сервис должен быть вызван ровно один раз для сохранения данных пользователя. В тесте `registerUser_shouldReturnToRegisterWhenValidationFails` проверяется ситуация, когда форма регистрации заполнена некорректно (например, пустой логин или несовпадающие пароли). В таком случае запрос должен вернуть страницу регистрации с отображением ошибок, а метод регистрации не должен быть вызван. В последнем тесте, `registerUser_shouldFailWhenUserAlreadyExists`, проверяется сценарий, когда попытка зарегистрировать пользователя с уже существующим логином должна привести к отображению ошибок на странице регистрации, и метод регистрации не будет вызван. Для этого используется заглушка для метода `userExists`, которая симулирует существование пользователя в базе данных.

Класс `SQLControllerTest`.

Метод `connectToDatabase` проверяется для того, чтобы убедиться, что система корректно обрабатывает подключение к базе данных. Тест использует мок для `DatabaseConnectionService`, чтобы имитировать успешное подключение. Проверяется, что при успешном подключении возвращается статус 200 и сообщение об успешном подключении.

Метод `generateSQL` проверяет, что запросы к Python API обрабатываются корректно. В тесте создается мок для `RestTemplate`, который имитирует ответ

от внешнего Python API. Тест отправляет запрос с текстом и схемой базы данных, проверяя, что возвращается корректно сгенерированный SQL-запрос. Также проверяется, что запрос сохраняется в репозитории истории SQL-запросов.

Метод `executeSQL` тестирует выполнение SQL-запросов. Для этого создается мок для `SqlExecutionService`, который имитирует выполнение запроса и возвращает фальшивые данные. Тест проверяет, что при успешном выполнении SQL-запроса система возвращает статус 200 и результаты выполнения. В случае ошибки при выполнении запроса проверяется, что возвращается ошибка с описанием.

В этих тестах используется подход с мокированием зависимостей, таких как сервисы и репозитории, с целью проверки логики контроллеров без реального взаимодействия с внешними сервисами или базой данных. Тесты направлены на то, чтобы удостовериться в правильности обработки запросов, генерации SQL-запросов, выполнении SQL и обработке ошибок.

Представленный отчёт рис.5.23 отражает состояние покрытия кода тестами в проекте `krolik`. Общий уровень покрытия инструкций составляет 79%, а покрытие ветвлений — 63%, что можно считать удовлетворительным показателем. Наиболее полно протестированы сервисные компоненты, где достигнут уровень покрытия инструкций в 97% и ветвлений в 84%. Это объясняется высокой значимостью бизнес-логики, реализованной в этом модуле, и необходимостью обеспечить её корректную и стабильную работу. Также полностью покрыт модуль конфигураций, что свидетельствует о внимательном подходе к качеству инфраструктурного слоя. Существенно ниже уровень покрытия наблюдается в модулях моделей, в частности, где покрытие отсутствует. Это обусловлено спецификой этих классов — они содержат исключительно простые структуры данных, не включающие сложной логики, поэтому их тестирование не представлялось необходимым на данном этапе разработки. Модуль контроллеров показал покрытие в 72%, что является рабочим результатом для слоя, взаимодействующего с HTTP-интерфейсом.

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.example.krolik.controllers		72%		50%	13	26	35	125	2	10	1	3
com.example.krolik.security		76%		0%	2	13	6	35	1	12	0	2
com.example.krolik.models.user		0%		n/a	7	7	7	7	7	7	1	1
com.example.krolik.services		97%		84%	4	27	2	87	0	13	0	6
com.example.krolik.models.history		0%		n/a	1	1	2	2	1	1	1	1
com.example.krolik		37%		n/a	1	2	2	3	1	2	0	1
com.example.krolik.configs		100%		n/a	0	2	0	2	0	2	0	1
Total	216 of 1,056	79%	22 of 60	63%	28	78	54	261	12	47	3	15

Рис.5.23. Покрытие тестами.

5.3 Интеграционное тестирование

Основные сложности при разработке приложения могут возникнуть при интеграции с внешними компонентами. В данном случае это подключение к базе данных пользователя и взаимодействие с моделью перевода естественного языка в SQL. Эти части системы требуют особого внимания из-за возможных ошибок соединения, различий в форматах данных и нестабильности внешних сервисов.

5.3.1 Интеграция с базой данных пользователя.

Класс DatabaseConnectionIntegrationTest.

Данный тестовый класс DatabaseConnectionIntegrationTest представляет собой интеграционный тест для проверки контроллера, отвечающего за подключение к базе данных в приложении Spring Boot. Класс аннотирован с помощью @SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT), что означает запуск полноценного контекста Spring-приложения с поднятием сервера на случайном порту. Таким образом, тест имитирует реальное взаимодействие клиента и сервера через HTTP-интерфейс.

Основной метод testSuccessfulConnection проверяет корректную работу эндпоинта /sql/connect. В начале теста с помощью библиотеки Mockito мокируется поведение метода getConnection сервиса DatabaseConnectionService. Вместо реального подключения к удалённой базе данных возвращается соединение с временной H2-базой данных в памяти, созданное через DriverManager.getConnection("jdbc:h2:mem:testdb "sa). Это исключает любые внешние зависимости и делает тест полностью изолированным и воспроизводимым.

Далее формируется JSON-запрос, имитирующий тело HTTP POST-запроса от клиента, включающее параметры подключения к базе данных, такие как адрес хоста, порт, тип СУБД, имя базы данных и учётные данные пользователя. Заголовки запроса также задаются явно: устанавливается Content-Type как application/json. Затем с помощью TestRestTemplate отправляется POST-запрос на эндпоинт /sql/connect, и ответ помещается в объект ResponseEntity<Map>.

Завершается тест серией утверждений с использованием библиотеки AssertJ. Проверяется, что HTTP-статус ответа равен HttpStatus.OK, то есть 200, и что

тело ответа содержит флаг "success" со значением true и сообщение "Connection successful". Эти утверждения гарантируют, что контроллер корректно обрабатывает валидный запрос и возвращает ожидаемый формат ответа.

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.786 s -- in com.example.krolik.integration.DatabaseConnectionIntegrationTest
```

Рис.5.24. Результаты теста.

Класс DatabaseSchemaIntegrationTest.

Класс DatabaseSchemaIntegrationTest реализует интеграционный тест, предназначенный для проверки корректности работы компонента DatabaseSchemaService в тестовой среде. Тестовая конфигурация запускается в контексте Spring Boot при помощи аннотации @SpringBootTest, с параметром webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT.

Аннотация @Import(TestSecurityConfig1.class) указывает на использование специальной конфигурации безопасности, в которой, как правило, отключена или упрощена аутентификация и авторизация, чтобы не мешать тестированию функционала. Активируется специальный профиль test, заданный через @ActiveProfiles("test"), что позволяет подключать тестовые настройки приложения, включая свойства и источники данных. Дополнительно указано использование тестового файла конфигурации application-test.yml через @TestPropertySource.

Главным объектом, тестируемым в данном классе, является бин DatabaseSchemaService, внедрённый с помощью @Autowired. Этот сервис предназначен для получения текстового представления схемы базы данных, в том числе таблиц и внешних ключей.

Метод testGetSchemaFromTestDatabase выполняет реальное подключение к тестовой базе данных, расположенной по адресу host.docker.internal, которая позволяет контейнеру Docker обращаться к хост-машине. В тесте передаются параметры подключения, включая порт 5432, тип СУБД PostgreSQL, имя базы postgres, логин postgres и пароль 123.

После вызова метода getDatabaseSchema возвращается строка, содержащая информацию о таблицах и связях в БД. С помощью библиотеки AssertJ производится проверка того, что возвращённая строка действительно содержит упоминания о таблицах departments и employees, а также о наличии внешнего ключа, связывающего employees.department_id с departments.id. Это указывает на корректное извлечение схемы базы данных, включая структуру и связи.

```
Connecting to database with URL: jdbc:postgresql://host.docker.internal:5432/postgres
Using username: postgres
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 5.005 s -- in com.example.integration.DatabaseSchemaIntegrationTest
```

Рис.5.25. Результаты теста.

5.3.2 Интеграция с моделью перевода.

Класс SQLGenerationIntegrationTest.

Интеграционный тест SQLGenerationIntegrationTest проверяет корректность работы REST-контроллера, отвечающего за генерацию SQL-запросов по входному текстовому описанию, в условиях изолированной тестовой среды. Этот тест выполняется с использованием встроенного HTTP-сервера на случайном порту, обеспечивая реалистичное взаимодействие с веб-контекстом приложения.

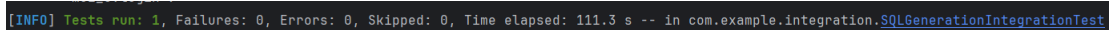
Аннотация `@SpringBootTest` с параметром `webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT` инициирует запуск всего Spring-приложения, включая веб-контроллеры, и использует случайный порт, который внедряется через поле `@LocalServerPort`. Использование аннотации `@WithMockUser` позволяет тесту выполняться от имени подставного пользователя `testUser` с ролью `USER`, что необходимо, так как эндпоинт `/sql/generate` защищён системой безопасности Spring Security.

Компонент `DatabaseSchemaService` замещается с помощью `@MockBean`, что позволяет подменить его поведение, не производя реальные подключения к базам данных. Метод `getDatabaseSchema` данного сервиса в рамках теста возвращает заранее заданную схему базы данных, содержащую описание двух таблиц `departments` и `employees`, а также внешнего ключа между ними. Это позволяет эмулировать типичный результат извлечения схемы из реальной базы без необходимости обращения к СУБД.

Для отправки HTTP-запроса к приложению используется экземпляр `TestRestTemplate`. Конструируется JSON-запрос, включающий ключевую текстовую фразу "Покажи все имена сотрудников а также параметры подключения к несуществующей базе данных. Этот JSON сериализуется при помощи `ObjectMapper` и передаётся вместе с заголовками (указан тип `application/json`) в HTTP-запросе на эндпоинт `/sql/generate`.

Основная цель теста — убедиться, что при корректной передаче текстового запроса и возвращаемой схеме, приложение успешно взаимодействует с Python-сервисом генерации SQL, возвращает HTTP-статус 200 (OK), и тело ответа

содержит фрагмент SQL-запроса, в частности ключевое слово `select`. Проверка проводится при помощи библиотеки `AssertJ`.



```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 111.3 s -- in com.example.integration.SQLGenerationIntegrationTest
```

Рис.5.26. Результаты теста.

5.4 Апробация инструмента для перевода естественного языка в SQL-код

Апробация — это этап опытной эксплуатации программной системы в максимально приближённых к реальности условиях. В отличие от тестирования, которое проводится в контролируемой среде с заранее определёнными входными данными и сценариями, апробация предполагает использование системы в «полевых» условиях с реальными пользователями, задачами и инфраструктурой. Целью апробации является проверка того, насколько система готова к практическому использованию, насколько удобно и корректно она работает в процессе взаимодействия с конечными пользователями.

5.4.1 Развертывание системы

Развертывание системы в реальных условиях требует ответственного комплексного подхода. Для этого необходимо подготовить инфраструктуру, настроить окружения и обеспечить доступ конечных пользователей. В начале нужно подготовить серверное оборудование или облачную платформу для размещения серверов. В зависимости от нагрузки и масштабирования стоит рассмотреть использование виртуализации, контейнеризации и балансировщиков нагрузки.

Далее необходимо установить и настроить все компоненты системы. Это базы данных, серверного приложения, вспомогательных сервисов. Важно обеспечить безопасность хранения и передачи данных, то есть система должна работать по защищенным протоколам и поддерживать масштабирование при увеличении числа пользователей.

После технического развертывания проводится тестирование в продуктивной среде, обучение пользователей и настройка технической поддержки. Важно предусмотреть систему логирования, мониторинга и быстрого реагирования на инциденты, чтобы обеспечить стабильную работу и своевременное устранение возможных сбоев. Такой подход позволяет создать устойчивую, безопасную и

удобную для пользователей информационную систему, готовую к эксплуатации в реальных бизнес-условиях.

5.4.2 Апробация в моделируемых условиях.

Поскольку в рамках данной работы не было возможности провести апробацию в условиях конкретного предприятия или организации, в том числе из-за отсутствия заказчика и доступа к производственным базам данных, опытная эксплуатация была выполнена в условиях, максимально приближённых к реальным. В качестве тестовой среды использовался локальный сервер, а в роли пользователей выступал сам разработчик, выполнявший типовые сценарии использования системы. Для симуляции реальных данных использовалась реальная база PostgreSQL с рабочей схемой и набором запросов, характерных для бизнес-применений рис.5.27.

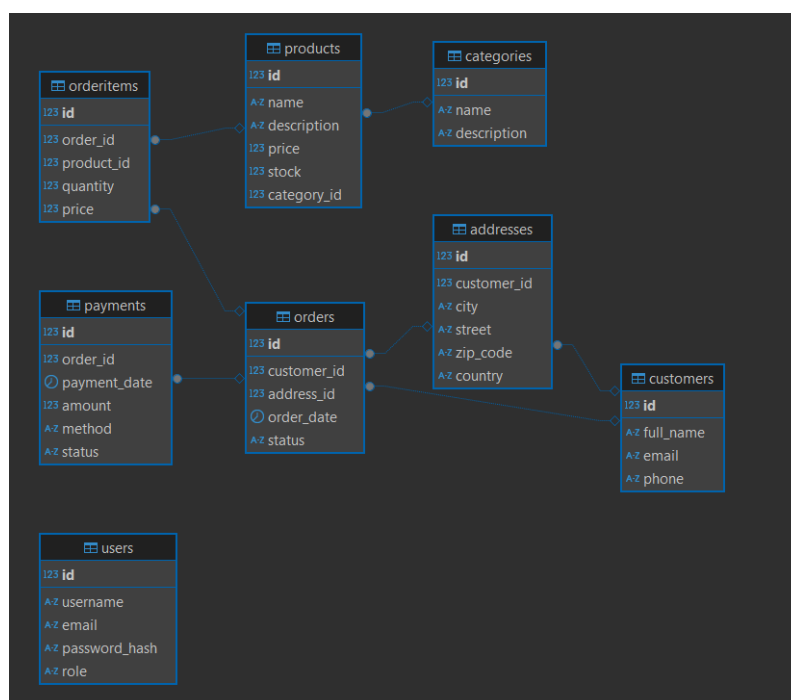


Рис.5.27. Схема базы данных.

Данная база данных представляет собой структуру хранения данных в реальном проекте интернет-магазина и содержит восемь взаимосвязанных таблиц, обеспечивающих хранение и управление информацией о пользователях, клиентах, товарах, заказах и оплате. В центре модели находятся таблицы Customers и Orders, описывающие клиентов и их заказы соответственно. Каждый клиент может иметь несколько адресов доставки, представленных в таблице Addresses, а заказ ссылается как на клиента, так и на один из его адресов. Таблица OrderItems служит для

детализации заказов, связывая каждый заказ с одним или несколькими товарами, указанными в таблице Products.

Товары сгруппированы по категориям через внешнюю связь с таблицей Categories, что позволяет удобно фильтровать и организовывать ассортимент. Таблица Products содержит подробную информацию о каждом товаре, включая цену, остаток на складе и принадлежность к категории. Информация об оплате заказов хранится в таблице Payments, где указывается сумма, способ оплаты и статус транзакции, что позволяет отслеживать процесс завершения заказа.

Дополнительно в структуру включена таблица Users, предназначенная для административных и служебных пользователей, которые имеют доступ к управлению магазином. Модель охватывает все ключевые аспекты функционирования интернет-магазина — от регистрации пользователей и оформления заказов до обработки платежей и логистики доставки.

В рамках проекта интернет-магазина приложение для генерации SQL-запросов на основе естественного языка может быть полезно нескольким категориям пользователей:

- Аналитикам и бизнес-пользователям, не обладающим глубокими знаниями SQL, но нуждающимся в получении отчетов и выгрузок из базы данных. Они смогут просто формулировать запросы на естественном языке и получать соответствующий SQL и данные без участия разработчиков.
- Менеджерам проекта или продукт-менеджерам, которым важно оперативно анализировать информацию — например, по продажам, остаткам товаров или активности пользователей — без необходимости вручную писать SQL-запросы или обращаться каждый раз к техотделу.
- Разработчикам и тестировщикам, которым может быть удобно быстро получать примеры запросов для отладки, проверки логики базы данных или генерации тестовых кейсов, особенно на этапе проектирования или сопровождения приложения.

Для проведения апробации был реализован эксперимент, в ходе которого имитировалась работа бизнес-сотрудников с типичными аналитическими запросами к базе данных. Проверялась способность системы корректно интерпретировать запросы на естественном языке и преобразовывать их в валидный SQL.

1. Получить список всех клиентов с указанием email рис.5.28.

Модель вывела корректный запрос.

2. Вывести все категории товаров рис.5.29.

Модель вывела корректный запрос.

3. Узнать количество заказов по каждому клиенту рис.5.30.
4. Получить список товаров, у которых количество на складе меньше 10 рис.5.31.

Модель не справилась с запросом, не сгруппировала заказы по клиентам.

5. Найти всех клиентов из определённого региона, например, "Москва"рис.5.33.

Модель не справилась с более сложным запросом, не смогла соединить таблицы.

5.5 Вывод

В процессе тестирования были проведены различные типы тестов, охватывающие все главные компоненты приложения. Ручное функциональное тестирование подтвердило корректность работы пользовательских сценариев. Модульные тесты подтвердили корректность работы компонентов изолированно. Было протестировано поведение контроллеров, сервисов и модели перевода.

Интеграционное тестирование обеспечило корректность взаимодействия компонентов приложения и внешних сервисов. Было протестировано взаимодействие приложения с внешней базой данных пользователя и интеграция с моделью перевода. Все тесты подтвердили корректную ожидаемую и стабильную работу системы.

Модель продемонстрировала удовлетворительные результаты: в большинстве случаев она успешно генерировала корректные SQL-запросы, соответствующие поставленным задачам. В отдельных ситуациях наблюдались неточности, связанные с интерпретацией сложных формулировок, однако даже в таких случаях создавался пригодный шаблон запроса, который пользователь мог отредактировать. Это подчеркивает перспективность дальнейшего развития приложения — улучшение модели перевода позволит добиться более высокой точности генерации и значительно упростит работу с базами данных для пользователей.

Генератор SQL

Опишите нужный запрос:

выведи полные имена и email всех клиентов

Сгенерировать SQL

Редактор SQL

```
SELECT full_name, email FROM customers
```

Выполнить

Результаты запроса

full_name	email
Иван Иванов	ivan@example.com
Мария Смирнова	maria@example.com

Рис.5.28. Результат.

Опишите нужный запрос:

выведи все категории товаров

Сгенерировать SQL

Редактор SQL

```
SELECT DISTINCT name FROM categories
```

Выполнить

Результаты запроса

name
Одежда
Электроника

Рис.5.29. Результат.

Опишите нужный запрос:

выведи количество заказов для каждого клиента

Сгенерировать SQL

Редактор SQL

```
SELECT COUNT(*) FROM orders GROUP BY customer_id
```

Выполнить

Результаты запроса

count
1
1

Рис.5.30. Результат.

Генератор SQL

Опишите нужный запрос:

Узнать количество заказов по каждому клиенту

Сгенерировать SQL

Редактор SQL

SELECT customer_id, COUNT(*) FROM orders;

Выполнить

Рис.5.31. Результат.

Опишите нужный запрос:

Узнать количество заказов по каждому клиенту

Сгенерировать SQL

Редактор SQL

SELECT customer_id, COUNT(*) AS total_orders FROM orders GROUP BY customer_id;

Выполнить

Результаты запроса

customer_id	total_orders
2	1
1	1

Рис.5.32. Результат.

Генератор SQL

Опишите нужный запрос:

Найти всех клиентов из из региона "Москва"

Сгенерировать SQL

Редактор SQL

SELECT name FROM customers WHERE region = 'Москва';

Выполнить

Рис.5.33. Результат.

ЗАКЛЮЧЕНИЕ

В результате выполнения работы была достигнута поставленная цель — был создан инструмент, способный переводить запросы на русском естественном языке в SQL-код, при этом запросы можно выполнять прямо внутри приложения. Данный инструмент особенно актуален в условиях современного бизнеса, где доступ к данным затруднен, и компаниям необходимо нанимать специалистов для работы с данными. Несмотря на то, что знание SQL является одним из важнейших навыков, далеко не все специалисты им обладают, поэтому создание доступного инструмента имеет большое практическое значение, особенно в условиях малого и среднего бизнеса.

На первом этапе были изучены существующие решения для перевода естественного языка в SQL. В процессе анализа были выявлены их сильные и слабые стороны. Были реализованы две модели перевода, была выбрана та, которая показала наилучшие результаты.

Далее был спроектирован и реализован сервер на Java Spring Boot. Он отвечает за взаимодействие пользователя и модели перевода. Был разработан пользовательский интерфейс, с помощью которого можно выводить запросы, редактировать их и отображать результат запроса. Кроме этого реализовано сохранение истории запросов.

Система прошла несколько этапов тестирования. Тесты подтвердили корректную работу приложения, весь ключевой функционал приложения были проверены и показали свою эффективность.

По итогам работы удалось создать полноценное веб-приложение, объединяющее все ключевые элементы — от интерфейса до логики взаимодействия с данными. Несмотря на то, что система пока не всегда идеально обрабатывает сложные или нестандартные запросы, она успешно справляется с большинством задач, что подтверждает её практическую ценность и потенциал для развития.

Задачи проекта были достигнуты.

- Проведен обзор существующих решений, выявлены преимущества и недостатки каждого из них.
- Были разработаны требования к инструменту по переводу естественного языка в SQL-код.
- Была спроектирована и реализована система, удовлетворяющая разработанным требованиям.

- Было проведено комплексное тестирование и апробация.
- Был проведен анализ полученных результатов и сделаны выводы по проделанной работе.

В перспективе эту работу следует развивать. Например, за счёт использования более продвинутых языковых моделей можно повысить точность и адаптивность перевода. Также полезно было бы добавить поддержку нескольких языков, включая английский и другие распространённые. В будущем можно также реализовать поддержку более сложных SQL-запросов и оптимизацию под разные СУБД, что сделает инструмент более универсальным.

ГИПОТЕЗЫ ДЛЯ ДАЛЬНЕЙШЕЙ РАБОТЫ.

Разработка инструментов для автоматического перевода естественного языка в SQL имеет большой потенциал и может серьёзно повлиять на то, как пользователи взаимодействуют с базами данных. Созданная архитектура и применённые методы машинного обучения показали, что такую задачу можно решать эффективно даже при использовании ограниченного набора данных.

В будущем ожидается улучшение точности преобразования запросов с помощью более совершенных моделей обработки естественного языка. Совершенствование алгоритмов машинного обучения и оптимизация архитектуры системы позволят сделать инструмент еще более удобным и точным. Тема имеет высокий потенциал для развития, особенно в контексте увеличивающейся потребности в автоматизированных системах работы с данными.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Attention Is All You Need / A. Vaswani [и др.]. — 2023.
2. Сервис, который помогает найти работу и подобрать персонал. — URL: <https://hh.ru> (дата обращения: 06.04.2025).
3. Dataherald AI Documentation. — URL: <https://dataherald.readthedocs.io/en/latest/introduction.html> (дата обращения: 22.01.2025).
4. DB-GPT Documentation. — URL: <http://docs.dbgpt.cn/docs/overview/> (дата обращения: 12.01.2025).
5. Job Search Career Community. — URL: <https://www.glassdoor.com/index.htm> (дата обращения: 06.04.2025).
6. Relational database. — URL: https://en.wikipedia.org/wiki/Relational_database (дата обращения: 17.04.2025).
7. SQLthroughAI. — URL: <https://sqlthroughai.com/> (дата обращения: 22.01.2025).
8. The Illustrated Transformer. — URL: <https://jalammar.github.io/illustrated-transformer/> (дата обращения: 27.02.2025).
9. Types of application architecture. — URL: <https://scilead.ru/article/6673-vidi-arkhitekturi-prilozhenij> (дата обращения: 06.02.2025).
10. Understanding LSTM Networks. — URL: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/> (дата обращения: 27.02.2025).
11. Yale University's Spider 1.0 NLP Dataset. — URL: <https://www.kaggle.com/datasets/jeromeblanchet/yale-universitys-spider-10-nlp-dataset>.
12. Распознавание именованных сущностей: механизм, методики, сценарии использования и реализация. — URL: <https://habr.com/ru/articles/776774/> (дата обращения: 14.01.2025).
13. Чуб Вадим Сергеевич М. Д. В. РЕКУРРЕНТНЫЕ НЕЙРОННЫЕ СЕТИ. ОБЗОР МЕТОДОВ И ПРИЛОЖЕНИЙ. — 2023.
14. Sanyal H., Shukla S., Agrawal R. Natural Language Processing Technique for Generation of SQL Queries Dynamically. — 2021.

Приложение 1

Сравнительная таблица

Таблица П1.1

Характеристика	Dataherald	DB-GPT	SQLthroughAI
Алгоритм/Модель	Dataherald использует машинное обучение и NLP-модели для обработки естественного языка и преобразования его в SQL-запросы. Использует модульную архитектуру.	DB-GPT интегрирует LLM, обученные на специализированных корпусах, для обеспечения точного и контекстно-зависимого взаимодействия с базами данных.	SQLthroughAI использует технологии искусственного интеллекта от OpenAI, адаптированные специально для преобразования естественного языка в SQL-запросы.
Поддерживаемые базы данных	PostgreSQL, MySQL, SQLite (Дополнительные базы данных можно подключить через стандартные драйверы JDBC)	Платформа поддерживает различные базы данных, включая PostgreSQL и MySQL, обеспечивая гибкость в выборе СУБД.	Инструмент поддерживает множество SQL-диалектов, включая MySQL, MongoDB и Oracle PL/SQL. Если необходимый диалект не поддерживается, пользователи могут связаться с разработчиками для его добавления.
Языковая поддержка	Только английский язык	DB-GPT поддерживает английский и китайский языки	Нет информации

Продолжение на следующей странице.

Характеристика	Dataherald	DB-GBT	SQLthroughAI
Тип интерфейса	Dataherald предоставляет REST API для интеграции с другими приложениями. Есть возможность работы через командную строку.	Платформа предоставляет веб-интерфейс на основе Gradio для интерактивного взаимодействия, а также API для интеграции с другими приложениями.	Веб-интерфейс, позволяющий пользователям вводить запросы на естественном языке и получать соответствующие SQL-запросы.
Документация	Документация хорошо структурирована и содержит примеры использования, настройку API и рекомендации по интеграции.	Подробная документация доступна на GitHub и официальном сайте проекта, предоставляя инструкции по установке, настройке и использованию.	Информации нет.
Настраиваемость	Dataherald позволяет адаптировать работу модели под конкретные наборы данных. Возможна настройка словарей, синонимов и других параметров для улучшения точности запросов.	Платформа позволяет настраивать модели и рабочие процессы в соответствии с конкретными требованиями пользователя, обеспечивая гибкость в использовании.	Инструмент позволяет пользователям настраивать базы данных путем загрузки файлов или ручного ввода столбцов
Лицензия	Dataherald распространяется под лицензией Apache 2.0	DB-GPT распространяется под лицензией MIT, что позволяет свободное использование и модификацию.	Информация отсутствует

Продолжение на следующей странице.

Характеристика	Dataherald	DB-GBT	SQLthroughAI
Стоимость	Бесплатно	Бесплатно	Конкретной стоимости нет, но около \$181
Масштабируемость	Dataherald хорошо масштабируется для работы с большими наборами данных. Производительность может зависеть от мощности серверов и конфигурации базы данных.	Благодаря поддержке различных сред развертывания и возможности работы с большими объемами данных, DB-GPT обеспечивает высокую масштабируемость.	Информация о масштабируемости инструмента на сайте отсутствует.
Дополнительный функционал	Автоматическая генерация отчетов и визуализаций, отслеживание изменений.	Платформа включает функции мониторинга метрик, диагностики, генерации отчетов и поддержку пользовательской обратной связи для улучшения качества взаимодействия.	Нет

Код проекта

П2.1 Код модели перевода

```

import os
import json
from pathlib import Path
5 from flask import Flask, request, jsonify

import torch
from torch.utils.data import Dataset, DataLoader
from torch.optim import AdamW
10 from transformers import (
    MBart50Tokenizer,
    MBartForConditionalGeneration,
    get_scheduler,
)
15 import tqdm

# Константы и конфигурация
MODEL_PATH = "bart_text2sql_question_only"
MODEL_NAME = "sn4kebyt3/ru-bart-large"
20 MAX_LENGTH = 128
DEVICE = torch.device("cuda" if torch.cuda.is_available()
    else "cpu")
SPIDER_DATA_PATH = Path(__file__).parent / "spider"

# Flask-приложение
25 app = Flask(__name__)
app.config["JSON_AS_ASCII"] = False

tokenizer = None
model = None
30

class SpiderDataset(Dataset):
    def __init__(self, data_path, schema_path, tokenizer,
        max_length=128):
        self.tokenizer = tokenizer
35 self.max_length = max_length
        self.schema_dict = self._load_schema(schema_path)

```

```

self.data = self._load_data(data_path)

def _load_data(self, data_path):
40 with open(data_path, "r", encoding="utf-8") as f:
    data = json.load(f)

    return [
        item for item in data
45 if self.tokenizer(
f"Schema: {self.schema_dict[item['db_id']] | Question: {
    item['question_ru']}",
truncation=False, return_tensors="pt"
)["input_ids"].shape[1] <= self.max_length
    ]

50 def _load_schema(self, schema_path):
    with open(schema_path, "r", encoding="utf-8") as f:
        schema_data = json.load(f)

55     schema_dict = {}
    for db in schema_data:
        db_id = db["db_id"]
        schema_dict[db_id] = self._schema_to_string(db)
    return schema_dict

60 def _schema_to_string(self, db):
    tables = []
    for i, table in enumerate(db["table_names_original"]):
        cols = [col[1] for col in db["column_names_original"] if col
            [0] == i]
65     tables.append(f"Table: {table} ({', '.join(cols)})")

    fks = []
    for fk in db.get("foreign_keys", []):
        col1 = db["column_names_original"][fk[0]][1]
70     col2 = db["column_names_original"][fk[1]][1]
        t1 = db["table_names_original"][db["column_names_original"][
            fk[0]][0]]
        t2 = db["table_names_original"][db["column_names_original"][
            fk[1]][0]]
        fks.append(f"{t1}.{col1} -> {t2}.{col2}")

75     schema_str = "; ".join(tables)
    if fks:
        schema_str += "; Foreign Keys: " + ", ".join(fks)

```

```

return schema_str

80 def __len__(self):
    return len(self.data)

def __getitem__(self, idx):
    item = self.data[idx]
85 question = item["question_ru"]
    sql = item["query"]
    db_id = item["db_id"]
    schema = self.schema_dict[db_id]
    input_text = f"Schema: {schema} | Question: {question}"
90
    inputs = self.tokenizer(input_text, max_length=self.
        max_length, truncation=True, padding="max_length",
        return_tensors="pt")
    targets = self.tokenizer(sql, max_length=self.max_length,
        truncation=True, padding="max_length", return_tensors="pt"
        ")

    return {
95     "input_ids": inputs["input_ids"].squeeze(),
        "attention_mask": inputs["attention_mask"].squeeze(),
        "target_ids": targets["input_ids"].squeeze(),
        "question": input_text,
        "sql": sql,
100 }

def train_model():
    global tokenizer, model
105
    tokenizer = MBart50Tokenizer.from_pretrained(MODEL_NAME)
    model = MBartForConditionalGeneration.from_pretrained(
        MODEL_NAME).to(DEVICE)

    train_data_path = SPIDER_DATA_PATH / "ru_train_spider.json"
110 schema_path = SPIDER_DATA_PATH / "tables.json"
    dataset = SpiderDataset(str(train_data_path), str(
        schema_path), tokenizer)

    print(f"Загружено обучающих примеров: {len(dataset)}")

115 loader = DataLoader(dataset, batch_size=8, shuffle=True)
    optimizer = AdamW(model.parameters(), lr=2e-5)

```

```

scheduler = get_scheduler("linear", optimizer=optimizer,
                           num_warmup_steps=500, num_training_steps=7 * len(loader))

for epoch in range(7):
120 model.train()
    total_loss = 0
    progress_bar = tqdm.tqdm(loader, desc=f"Epoch {epoch + 1}")

    for batch in progress_bar:
125 optimizer.zero_grad()
        outputs = model(
            input_ids=batch["input_ids"].to(DEVICE),
            attention_mask=batch["attention_mask"].to(DEVICE),
            labels=batch["target_ids"].to(DEVICE),
130 )
        loss = outputs.loss
        loss.backward()
        optimizer.step()
        scheduler.step()

135 total_loss += loss.item()
    progress_bar.set_postfix({"loss": loss.item()})

    print(f"Эпоха {epoch+1} завершена. Средний loss: {total_loss
        / len(loader):.4f}")

140 model.save_pretrained(MODEL_PATH)
    tokenizer.save_pretrained(MODEL_PATH)
    print(f"Модель сохранена в: {MODEL_PATH}")

145 def load_model():
    global tokenizer, model
    try:
        tokenizer = MBart50Tokenizer.from_pretrained(MODEL_PATH)
150 model = MBartForConditionalGeneration.from_pretrained(
            MODEL_PATH).to(DEVICE)
        print("Модель успешно загружена.")
        return True
    except Exception as e:
        print(f"Ошибка загрузки модели: {e}")
155 return False

def generate_sql(question, schema):

```

```

input_text = f"Schema: {schema} | Question: {question}"
160 inputs = tokenizer(
    input_text,
    max_length=MAX_LENGTH,
    truncation=True,
    padding="max_length",
165 return_tensors="pt",
).to(DEVICE)

with torch.no_grad():
    output_ids = model.generate(
170 input_ids=inputs["input_ids"],
    attention_mask=inputs["attention_mask"],
    max_length=MAX_LENGTH,
    num_beams=4,
    early_stopping=True,
175 do_sample=True,
    temperature=0.7,
    top_k=50,
    top_p=0.9,
    )

180 return tokenizer.decode(output_ids[0], skip_special_tokens=
    True)

@app.route("/sql", methods=["POST"])
185 def handle_sql():
    print(">>> Получен запрос:", request.get_data(as_text=True))
        # добавь это
    if not request.is_json:
        return jsonify({"error": "Требуется JSON-запрос"}), 400

190 data = request.get_json()
    question = data.get("text")
    schema = data.get("schema")

    if not question or not schema:
195 return jsonify({"error": "Поля 'text' и 'schema' обязательны
        "}), 400

    sql = generate_sql(question, schema)
    return jsonify({"sql": sql})

```

200

```

if __name__ == "__main__":
# Проверка наличия данных
if not SPIDER_DATA_PATH.exists():
print(f"Не найдена папка с данными: {SPIDER_DATA_PATH}")
205 exit(1)

train_flag = os.getenv("TRAIN_MODEL", "false").lower() == "
    true"

if train_flag:
210 print("TRAIN_MODEL=true: запускаем обучение...")
    train_model()

if not load_model():
print("Модель не загружена. Запустите обучение указав флаг
    TRAIN_MODEL=true при запуске приложения")
215 exit(1)

app.run(host="0.0.0.0", port=5000)

```

П2.2 Код Java приложения

AuthenticationController.

```

package com.example.krolik.controllers;

import org.springframework.stereotype.Controller;
5 import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.
    ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;
10 import com.example.krolik.models.user.UserDto;
import com.example.krolik.services.AuthenticationService;

import lombok.RequiredArgsConstructor;

15 @Controller
@RequiredArgsConstructor
public class AuthenticationController {
20

```

```

private final AuthenticationService authenticationService;

@GetMapping("/login")
public String loginPage() {
25     return "login";
}

@GetMapping("/logout")
public String logout() {
30     return "redirect:/login?logout"; // Spring Security аемо
        матически очистит сессию
}

@GetMapping("/register")
35 public String registerPage(Model model) {
    model.addAttribute("userDto", new UserDto());
    return "register";
}

40 @PostMapping("/register")
public String registerUser(@ModelAttribute("userDto")
    UserDto userDto,
    BindingResult bindingResult,
    Model model) {

45     // Валидация имени пользователя
    if (userDto.getLogin() == null || userDto.getLogin().
        trim().isEmpty()) {
        bindingResult.rejectValue("login", "error.login", "Имя
            пользователя обязательно");
    } else if (userDto.getLogin().length() < 3 || userDto.
        getLogin().length() > 20) {
        bindingResult.rejectValue("login", "error.login", "Имя
            пользователя должно быть от 3 до 20 символов");
50     }

    // Валидация пароля
    if (userDto.getPassword() == null || userDto.getPassword
        ().trim().isEmpty()) {
        bindingResult.rejectValue("password", "error.password"
            , "Пароль обязателен");
55     } else if (userDto.getPassword().length() < 6) {
        bindingResult.rejectValue("password", "error.password"
            , "Пароль должен быть не менее 6 символов");
    }
}

```

```

    }

    // Проверка совпадения паролей
60 if (!userDto.getPassword().equals(userDto.
    getConfirmPassword())) {
        bindingResult.rejectValue("confirmPassword", "error.
            confirmPassword", "Пароли не совпадают");
    }

    // Проверка существования пользователя
65 if (authenticationService.userExists(userDto.getLogin())
    ) {
        bindingResult.rejectValue("login", "error.login", "Пол
            ьзователь с таким именем уже существует");
    }

    if (bindingResult.hasErrors()) {
70         model.addAttribute("errors", bindingResult);
        return "register";
    }

    authenticationService.registerUser(userDto);
75 return "redirect:/login?registered";
}

}

```

MainController.

```

package com.example.krolik.controllers;

import java.time.LocalDate;
5 import java.util.ArrayList;
import java.util.List;

import com.example.krolik.models.user.MyUser;
import com.example.krolik.repositories.UserRepository;
10 import org.springframework.format.annotation.DateTimeFormat;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
15 import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.
    SecurityContextHolder;

```

```

import com.example.krolik.models.history.SQLHistory;
import com.example.krolik.repositories.SQLHistoryRepository;
import com.example.krolik.services.AppService;
import com.example.krolik.services.AppService.AppState;

import lombok.RequiredArgsConstructor;

@Controller
@RequiredArgsConstructor
public class MainController {

    private final AppService appService;
    private final SQLHistoryRepository sqlHistoryRepository;
    private final UserRepository userRepository;

    @GetMapping
    public String getHomePage(Model model) {
        appService.setModelState(AppState.MAIN, model);
        return "home";
    }

    @GetMapping("/history")
    public String getHistoryPage(
        @RequestParam(required = false) String searchText,
        @RequestParam(required = false) @DateTimeFormat(pattern="
        yyyy-MM-dd") LocalDate searchDate,
        Model model) {

        appService.setModelState(AppState.HISTORY, model);

        Authentication authentication = SecurityContextHolder.
            getContext().getAuthentication();
        String login = authentication.getName();
        MyUser currentUser = userRepository.findByLogin(login).
            orElseThrow();

        List<SQLHistory> history = new ArrayList<>();

        if (searchText != null && searchDate != null) {
            history = sqlHistoryRepository.
                findByUserAndTextContainingAndDate(currentUser,
                    searchText, searchDate);
        } else if (searchText != null) {

```

```

        history = sqlHistoryRepository.
            findByUserAndTextContaining(currentUser, searchText
        );
    } else if (searchDate != null) {
60         history = sqlHistoryRepository.findByUserAndDate(
            currentUser, searchDate);
    } else {
        history = sqlHistoryRepository.findByUserOrderByIdDesc
            (currentUser);
    }

65     model.addAttribute("history", history);
    model.addAttribute("searchText", searchText);
    model.addAttribute("searchDate", searchDate != null ?
        searchDate.toString() : "");
    return "home";
    }
70 }
}

```

SQLController.

```

package com.example.krolik.controllers;

import java.sql.Connection;
import java.sql.ResultSet;
5 import java.sql.SQLException;
import java.time.LocalDate;
import java.util.Map;
import java.util.HashMap;
10 import java.util.List;
import java.util.ArrayList;

import com.example.krolik.models.user.MyUser;
import com.example.krolik.repositories.UserRepository;
15 import org.springframework.beans.factory.annotation.Value;
import org.springframework.http.HttpEntity;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpMethod;
import org.springframework.http.HttpStatus;
20 import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.
    RequestMapping;

```

```

25 import org.springframework.web.bind.annotation.
    RestController;
import org.springframework.web.client.RestTemplate;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.
    SecurityContextHolder;

30
import com.example.krolik.models.history.SQLHistory;
import com.example.krolik.models.request.SQLRequest;
import com.example.krolik.repositories.SQLHistoryRepository;
import com.example.krolik.services.DatabaseSchemaService;
35 import com.example.krolik.services.SqlExecutionService;
import com.example.krolik.services.DatabaseConnectionService
    ;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;

40 @RestController
@RequestMapping("/sql")
public class SQLController {

    private final SQLHistoryRepository SQLHistoryRepository;
45 private final SqlExecutionService sqlExecutionService;
private final RestTemplate restTemplate;
private final DatabaseSchemaService databaseSchemaService;
private final DatabaseConnectionService
    databaseConnectionService;
private final UserRepository userRepository;

50

    @Value("${python.url}")
    private String pythonApiUrl;

55 // Конструктор для внедрения зависимостей
public SQLController(SQLHistoryRepository
    SQLHistoryRepository,
    SqlExecutionService sqlExecutionService,
    RestTemplate restTemplate,
    DatabaseSchemaService databaseSchemaService,
60 DatabaseConnectionService databaseConnectionService,
    UserRepository userRepository) {
    this.SQLHistoryRepository = SQLHistoryRepository;
    this.sqlExecutionService = sqlExecutionService;
    this.restTemplate = restTemplate;

```

```

65     this.databaseSchemaService = databaseSchemaService;
        this.databaseConnectionService =
            databaseConnectionService;
        this.userRepository = userRepository;
    }

70     // Эндпоинт для подключения к базе данных
    @PostMapping("/connect")
    public ResponseEntity<?> connectToDatabase(@RequestBody
        SQLRequest request) {
        try {
            // Пытаемся подключиться к базе данных
75            Connection connection = databaseConnectionService.
                getConnection(
                    request.getDbHost(),
                    request.getDbPort(),
                    request.getDbType(),
                    request.getDbName(),
80                    request.getDbUsername(),
                    request.getDbPassword()
                );
            // Если подключение успешно
            return ResponseEntity.ok(Map.of("success", true, "
                message", "Connection successful"));
85        } catch (SQLException e) {
            // Если не удалось подключиться
            return ResponseEntity.status(HttpStatus.BAD_REQUEST).
                body(Map.of(
                    "success", false,
                    "error", "Failed to connect: " + e.getMessage()
90                ));
        }
    }

    // Эндпоинт для генерации SQL-запроса
95    @PostMapping("/generate")
    public ResponseEntity<?> generateSQL(@RequestBody
        SQLRequest request) {
        try {
            System.out.println("Полученные данные запроса: ");
            System.out.println("text: " + request.getText());
100           System.out.println("dbHost: " + request.getDbHost());
            System.out.println("dbPort: " + request.getDbPort());
            System.out.println("dbType: " + request.getDbType());
            System.out.println("dbName: " + request.getDbName());

```

```

System.out.println("dbUsername: " + request.
    getDbUsername());
105 System.out.println("dbPassword: " + request.
    getDbPassword());
    // Выводим сообщение о начале получения схемы
System.out.println("Получаем схему базы данных для: "
    + request.getDbUsername() + "@" + request.getDbHost
    () + ":" + request.getDbPort() + "/" + request.
    getDbName());
    // Получаем схему базы данных
String schema = databaseSchemaService.
    getDatabaseSchema(
110 request.getDbHost(),
    request.getDbPort(),
    request.getDbType(),
    request.getDbName(),
    request.getDbUsername(),
115 request.getDbPassword()
    );

    // Выводим сообщение о полученной схеме
System.out.println("Схема базы данных получена: " +
    schema);
120

    // Подготовка запроса к Python API
System.out.println("Формируем запрос к Python API с те
    кстом запроса: " + request.getText());

    // Подготовка запроса к Python API
125 HttpHeaders headers = new HttpHeaders();
    headers.setContentType(MediaType.APPLICATION_JSON);

    // Формируем JSON-параметры запроса в виде Map
Map<String, Object> payload = new HashMap<>();
130 payload.put("text", request.getText());
    payload.put("schema", schema);

    // Сериализуем JSON с помощью ObjectMapper
ObjectMapper objectMapper = new ObjectMapper();
135 String requestJson = objectMapper.writeValueAsString(
    payload);

    // Оборачиваем в HttpEntity с заголовками
HttpEntity<String> entity = new HttpEntity<>(
    requestJson, headers);

```

```

140      // Выводим сообщение об отправке запроса
      System.out.println("Отправляем запрос к Python API: "
          + pythonApiUrl);

      // Вызов Python API
      ResponseEntity<String> response = restTemplate.
          exchange(
145      pythonApiUrl,
          HttpMethod.POST,
          entity,
          String.class
      );

150      // Выводим сообщение о полученном ответе от API
      System.out.println("Ответ от Python API: " + response.
          getStatusCode());

      // Обработка ответа от API
155      if (response.getStatusCode() == HttpStatus.OK) {
          // Парсим JSON ответ
          ObjectMapper mapper = new ObjectMapper();
          JsonNode rootNode = mapper.readTree(response.getBody
              ());
          String sql = rootNode.path("sql").asText();

160      // Выводим сгенерированный SQL
      System.out.println("SQL-запрос успешно сгенерирован:
          " + sql);

      Authentication authentication =
          SecurityContextHolder.getContext().
              getAuthentication();
165      String login = authentication.getName();
      MyUser currentUser = userRepository.findByLogin(
          login).orElseThrow();

      // Сохраняем в историю
      SQLHistory sqlHistory = new SQLHistory();
170      sqlHistory.setText(request.getText());
      sqlHistory.setSql(sql);
      sqlHistory.setDate(LocalDate.now());
      sqlHistory.setUser(currentUser);
      SQLHistoryRepository.save(sqlHistory);

175

```

```

        return ResponseEntity.ok(Map.of(
            "success", true,
            "sql", sql
        ));
180     } else {
        return ResponseEntity.ok(Map.of(
            "success", false,
            "error", response.getBody()
        ));
185     }

    } catch (Exception e) {
        return ResponseEntity.ok(Map.of(
            "success", false,
190         "error", e.getMessage()
        ));
    }
}

195 @PostMapping("/execute")
public ResponseEntity<?> executeSQL(@RequestBody
    SQLRequest request) {
    try {
        // Выполнение SQL-запроса на основе данных подключения
        List<Map<String, Object>> results =
            sqlExecutionService.executeQuery(
200         request.getHost(),
            request.getPort(),
            request.getType(),
            request.getName(),
            request.getUsername(),
205         request.getPassword(),
            request.getSqlQuery()
        );

        // Обработка результатов запроса
210         return ResponseEntity.ok(Map.of(
            "success", true,
            "data", results
        ));

215     } catch (SQLException e) {
        return ResponseEntity.ok(Map.of(
            "success", false,

```

```

        "error", "Ошибка выполнения запроса: " + e.getMessage
            ()
        ));
220     }
    }

}

```

Model SQLHistory.

```

package com.example.krolik.models.history;

import java.time.LocalDate;
5  import java.time.format.DateTimeFormatter;

import com.example.krolik.models.user.MyUser;
import jakarta.persistence.*;
import lombok.AllArgsConstructor;
10 import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@NoArgsConstructor
15 @AllArgsConstructor
@Getter
@Setter
@Entity
public class SQLHistory {

20     @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

25     @Column(columnDefinition = "TEXT")
    private String text;
    @Column(columnDefinition = "TEXT")
    private String sql;
    private LocalDate date;

30     @ManyToOne
    @JoinColumn(name = "user_id", nullable = false)
    private MyUser user;

35     public String getFormattedDate() {
        DateTimeFormatter formatter = DateTimeFormatter.
            ofPattern("dd.MM.yyyy");
    }
}

```

```

        return this.date.format(formatter);
    }
}

```

Model SQLRequest.

```

package com.example.krolik.models.request;

import lombok.AllArgsConstructor;
5 import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@NoArgsConstructor
10 @AllArgsConstructor
@Getter
@Setter
public class SQLRequest {

15     private String text; // описание запроса
    private String dbHost;
    private Integer dbPort;
    private String dbType;
    private String dbName;
20     private String dbUsername;
    private String dbPassword;
    private String sqlQuery;

}

```

Model MyUser.

```

package com.example.krolik.models.user;

import java.util.Collection;
5 import java.util.Collections;
import java.util.List;

import com.example.krolik.models.history.SQLHistory;
10 import jakarta.persistence.*;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.
    SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.
    UserDetails;

```

```

15  import lombok.AllArgsConstructor;
    import lombok.Getter;
    import lombok.NoArgsConstructor;
    import lombok.Setter;

20  @NoArgsConstructor
    @AllArgsConstructor
    @Getter
    @Setter
    @Entity
25  @Table
    public class MyUser implements UserDetails {

        @Id
        @GeneratedValue(strategy = GenerationType.IDENTITY)
30  @Column(name = "id")
        private Long id;

        @Column(nullable = false, unique = true)
        private String login;
35  private String password;
        private String role;

        @OneToMany(mappedBy = "user", cascade = CascadeType.ALL,
                     fetch = FetchType.LAZY)
40  private List<SQLHistory> sqlHistories;

        @Override
        public Collection<? extends GrantedAuthority>
            getAuthorities() {
            return Collections.singletonList(new
                SimpleGrantedAuthority("ROLE_" + role));
        }
45

        @Override
        public String getPassword() {
            return this.password;
        }

50  @Override
        public String getUsername() {
            return this.login;
        }

55  @Override

```

```

    public boolean isAccountNonExpired() {
        return true;
    }

    @Override
    public boolean isAccountNonLocked() {
        return true;
    }

    @Override
    public boolean isCredentialsNonExpired() {
        return true;
    }

    @Override
    public boolean isEnabled() {
        return true;
    }
}

```

SQLHistoryRepository.

```

package com.example.krolik.repositories;

import java.time.LocalDate;

import java.util.List;

import com.example.krolik.models.user.MyUser;
import org.springframework.data.jpa.repository.JpaRepository;
;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import com.example.krolik.models.history.SQLHistory;

public interface SQLHistoryRepository extends JpaRepository<
    SQLHistory, Long> {

    List<SQLHistory> findByUser(MyUser user);

    List<SQLHistory> findByUserOrderByIdDesc(MyUser user);
}

```

```

    List<SQLHistory> findByUserAndTextContaining(MyUser user,
        String text);

    List<SQLHistory> findByUserAndDate(MyUser user, LocalDate
        date);
25
    List<SQLHistory> findByUserAndTextContainingAndDate(MyUser
        user, String text, LocalDate date);

    // Или с использованием @Query для более сложных запросов
    @Query("SELECT h FROM SQLHistory h WHERE " +
30
        "LOWER(h.text) LIKE LOWER(CONCAT('%', :text, '%')) AND " +
        "CAST(h.date AS date) = :date")
    List<SQLHistory> searchTextAndDate(@Param("text") String
        text,
        @Param("date") LocalDate date);

    @Query("SELECT h FROM SQLHistory h ORDER BY h.id DESC")
35
    List<SQLHistory> findAllOrderByIdDesc();

}

```

UserRepository.

```

package com.example.krolik.repositories;

import java.util.Optional;
5
import org.springframework.data.jpa.repository.JpaRepository;

import com.example.krolik.models.user.MyUser;

10
public interface UserRepository extends JpaRepository<MyUser
    , Long> {

    Optional<MyUser> findByLogin(String login);
    boolean existsByLogin(String login);
}

```

CustomUserDetailsService.

```

package com.example.krolik.security;

import java.util.Optional;
5

```

```

import org.springframework.security.core.userdetails.
    UserDetails;
import org.springframework.security.core.userdetails.
    UserDetailsService;
import org.springframework.security.core.userdetails.
    UsernameNotFoundException;
import org.springframework.stereotype.Service;
10 import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import com.example.krolik.models.user.MyUser;
15 import com.example.krolik.repositories.UserRepository;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;

20 @Service
@RequiredArgsConstructor
@Slf4j
public class CustomUserDetailsService implements
    UserDetailsService {

25     private final UserRepository userRepository;
    private static final Logger log = LoggerFactory.getLogger(
        CustomUserDetailsService.class);

    @Override
30     public UserDetails loadUserByUsername(String login) throws
        UsernameNotFoundException {
        Optional<MyUser> optionalUser = userRepository.
            findByLogin(login);
        if (optionalUser.isEmpty()) {
            log.info("Пользователь не найден");
            throw new UsernameNotFoundException("User not found
35             with login: " + login);
        }

        log.info("Пользователь:" + userRepository.findByLogin(
            login).get());
        return optionalUser.get();
40     }
}

```

SecurityConfig.

```

package com.example.krolik.security;

import org.springframework.context.annotation.Bean;
5 import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;
import org.springframework.security.authentication.
    AuthenticationManager;
import org.springframework.security.config.annotation.
    authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.web.
    builders.HttpSecurity;
10 import org.springframework.security.config.http.
    SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.
    BCryptPasswordEncoder;
import org.springframework.security.crypto.password.
    PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
@Profile("!test")
15 @Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(
        HttpSecurity http) throws Exception {
20     return http
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(authz -> authz
        .requestMatchers("/login", "/register").permitAll()
        .anyRequest().authenticated())
25     .rememberMe(rememberMe -> {
        rememberMe
        .key("rememberMe")
        .tokenValiditySeconds(Integer.MAX_VALUE)
        .alwaysRemember(true)
30     .useSecureCookie(true);
    })
    .formLogin(form -> form
    .loginPage("/login")
    .defaultSuccessUrl("/", true)
35     .permitAll())
    .logout(logout -> logout

```

```

        .logoutUrl("/logout") // URL, который вызывается для
            logout
        .logoutSuccessUrl("/login?logout") // Куда перенаправляем
            после выхода
        .invalidateHttpSession(true)
40    .deleteCookies("JSESSIONID")
        .permitAll()
        .sessionManagement(session -> session
        .sessionCreationPolicy(SessionCreationPolicy.ALWAYS))
        .build();
45    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
50    }

    @Bean
    public AuthenticationManager authenticationManager(
        AuthenticationConfiguration authenticationConfiguration
    ) throws Exception {
        return authenticationConfiguration.
            getAuthenticationManager();
55    }

}

```

AppService.

```

package com.example.krolik.services;

import org.springframework.security.authentication.
    AnonymousAuthenticationToken;
5    import org.springframework.security.core.Authentication;
import org.springframework.stereotype.Service;
import org.springframework.ui.Model;

import com.example.krolik.models.user.MyUser;
10    import com.example.krolik.repositories.UserRepository;

import lombok.RequiredArgsConstructor;

@Service
15    @RequiredArgsConstructor
    public class AppService {

```

```

private final UserRepository userRepository;

20 private String getModelAppState(AppState appState) {
    return appState.toString().toLowerCase();
}

public void setModelState(AppState state, Model model) {
25     model.addAttribute("appState", getModelAppState(state));
}

public MyUser getUser(Authentication authentication) {
    if (authentication != null && !(authentication
        instanceof AnonymousAuthenticationToken) &&
        authentication.isAuthenticated()) {
30     MyUser userDetails = (MyUser) authentication.
        getPrincipal();
        return userRepository.findById(userDetails.getId()).
            orElse(null);
    }

    return null;
35 }

public static enum AppState {
    MAIN, HISTORY
40 }
}

```

AuthenticationService.

```

package com.example.krolik.services;

import org.springframework.security.crypto.password.
    PasswordEncoder;
5 import org.springframework.stereotype.Service;

import com.example.krolik.models.user.MyUser;
import com.example.krolik.models.user.UserDto;
import com.example.krolik.repositories.UserRepository;
10

import lombok.RequiredArgsConstructor;

@Service
@RequiredArgsConstructor

```

```

15 public class AuthenticationService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

20 public boolean userExists(String username) {
    return userRepository.findByLogin(username).isPresent();
}

    public void registerUser(UserDto userDto) {
25     if (userExists(userDto.getLogin())) {
        throw new RuntimeException("Пользователь с таким именем
            уже существует");
    }
    MyUser user = new MyUser();
    user.setLogin(userDto.getLogin());
30 user.setPassword(passwordEncoder.encode(userDto.
    getPassword()));
    userRepository.save(user);
}

}

```

DatabaseConnectionService.

```

package com.example.krolik.services;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

import org.springframework.stereotype.Service;

10 @Service
public class DatabaseConnectionService {

    public Connection getConnection(String dbHost, int dbPort,
        String dbType, String dbName, String dbUsername,
        String dbPassword) throws SQLException {
        String url = buildDatabaseUrl(dbType, dbHost, dbPort,
            dbName);
15 System.out.println("Connecting to database with URL: " +
        url);
    System.out.println("Using username: " + dbUsername);
    return DriverManager.getConnection(url, dbUsername,
        dbPassword);
}

```

```

    }

    private String buildDatabaseUrl(String dbType, String
20         dbHost, int dbPort, String dbName) {
        switch (dbType) {
            case "postgresql":
                return "jdbc:postgresql://" + dbHost + ":" + dbPort +
                    "/" + dbName;
            case "mysql":
25         return "jdbc:mysql://" + dbHost + ":" + dbPort + "/" +
                dbName;
            case "oracle":
                return "jdbc:oracle:thin:@" + dbHost + ":" + dbPort +
                    ":" + dbName;
            default:
                throw new IllegalArgumentException("Unsupported DB
30         type: " + dbType);
        }
    }
}

```

DatabaseSchemaService.

```

package com.example.krolik.services;

import java.sql.Connection;
5   import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

import org.springframework.stereotype.Service;

10  @Service
public class DatabaseSchemaService {

    private final DatabaseConnectionService
        databaseConnectionService;

15  public DatabaseSchemaService(DatabaseConnectionService
        databaseConnectionService) {
        this.databaseConnectionService =
            databaseConnectionService;
    }
}

```

```

20 public String getDatabaseSchema(String dbHost, int dbPort,
    String dbType, String dbName, String dbUsername,
    String dbPassword) throws SQLException {
    StringBuilder schema = new StringBuilder();

    try (Connection connection = databaseConnectionService.
        getConnection(dbHost, dbPort, dbType, dbName,
            dbUsername, dbPassword)) {

25         // Получаем список таблиц
        String query = "SELECT table_name FROM
            information_schema.tables WHERE table_schema = '
            public'";
        PreparedStatement statement = connection.
            prepareStatement(query);
        ResultSet resultSet = statement.executeQuery();

30         while (resultSet.next()) {
            String tableName = resultSet.getString("table_name")
                ;
            schema.append("Table: ").append(tableName).append("
                (");

            // Получаем столбцы таблицы
35             String columnsQuery = "SELECT column_name FROM
                information_schema.columns WHERE table_name = ?";
            PreparedStatement columnStatement = connection.
                prepareStatement(columnsQuery);
            columnStatement.setString(1, tableName);
            ResultSet columnsResultSet = columnStatement.
                executeQuery();

40             StringBuilder columns = new StringBuilder();
            while (columnsResultSet.next()) {
                columns.append(columnsResultSet.getString("
                    column_name")).append(", ");
            }

45             if (columns.length() > 0) {
                columns.setLength(columns.length() - 2);
            }

            schema.append(columns.toString()).append(");\n");

50             // Получаем oid таблицы

```

```

String oidQuery = "SELECT oid FROM pg_class WHERE
    relname = ?";
PreparedStatement oidStmt = connection.
    prepareStatement(oidQuery);
oidStmt.setString(1, tableName);
55 ResultSet oidRs = oidStmt.executeQuery();

if (oidRs.next()) {
    int tableOid = oidRs.getInt("oid");

60     // Получаем внешние ключи
    String foreignKeysQuery =
        "SELECT conname, conrelid::regclass AS table_name,
            a.attname AS column_name, " +
        "confrelid::regclass AS foreign_table_name, af.
            attname AS foreign_column_name " +
        "FROM pg_constraint AS c " +
65 "JOIN pg_attribute AS a ON a.attnum = ANY(c.conkey
        ) AND a.attrelid = c.conrelid " +
        "JOIN pg_attribute AS af ON af.attnum = ANY(c.
            confkey) AND af.attrelid = c.confrelid " +
        "WHERE confrelid != 0 AND c.conrelid = ?";

    PreparedStatement fkStatement = connection.
        prepareStatement(foreignKeysQuery);
70 fkStatement.setInt(1, tableOid);
    ResultSet fkResultSet = fkStatement.executeQuery()
        ;

    while (fkResultSet.next()) {
        String fkTable = fkResultSet.getString("
            foreign_table_name");
75 String fkColumn = fkResultSet.getString("
            foreign_column_name");
        String column = fkResultSet.getString("
            column_name");
        schema.append("Foreign Keys: ").append(tableName)
            .append(".").append(column)
            .append(" -> ").append(fkTable).append(".").
            append(fkColumn).append("\n");
80     }
    }
}

```

```

85         return schema.toString();
    }

}

```

SqlExecutionService.

```

package com.example.krolik.services;

import org.springframework.stereotype.Service;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.HashMap;

@Service
public class SqlExecutionService {

    private final DatabaseConnectionService
        databaseConnectionService;

    public SqlExecutionService(DatabaseConnectionService
        databaseConnectionService) {
        this.databaseConnectionService =
            databaseConnectionService;
    }

    // Метод для выполнения SQL-запроса и получения результатов
    в виде списка
    public List<Map<String, Object>> executeQuery(String
        dbHost, Integer dbPort, String dbType, String dbName,
        String dbUsername, String dbPassword, String sqlQuery)
        throws SQLException {
        try (Connection connection = databaseConnectionService.
            getConnection(dbHost, dbPort, dbType, dbName,
                dbUsername, dbPassword);
            PreparedStatement statement = connection.
                prepareStatement(sqlQuery);
            ResultSet resultSet = statement.executeQuery()) {

            // Список для хранения результатов
            List<Map<String, Object>> results = new ArrayList<>();

```

```

// Получение метаданных для обработки столбцов
ResultSetMetaData metaData = resultSet.getMetaData();
int columnCount = metaData.getColumnCount();

35 while (resultSet.next()) {
    // Хранение строки как карты (key - название столбца
    // , value - значение)
    Map<String, Object> row = new HashMap<>();
    for (int i = 1; i <= columnCount; i++) {
        String columnName = metaData.getColumnName(i);
        Object value = resultSet.getObject(i);
40         row.put(columnName, value);
    }
    results.add(row);
}

45 return results;
} catch (SQLException e) {
    // Логирование и передача ошибки
    throw new SQLException("Ошибка выполнения запроса: " +
50         e.getMessage(), e);
}
}
}

```

history.html

```

<th:block th:fragment="bit()">
<div class="d-flex row justify-content-center">
<h1 class="text-center mb-4">История запросов</h1>

5 <!-- Форма поиска -->
<form th:action="@{/history}" method="get" class="mb-4 m-5 d
    -flex justify-content-center">
<input type="text" class="form-control w-25 me-3" name="
    searchText"
    th:value="${searchText}" placeholder="Поиск по тексту">
10 <input type="date" class="form-control w-25 me-3" name="
    searchData"
    th:value="${searchDate}">
<button type="submit" class="btn btn-primary" style="width:
    15%">Найти</button>
</form>

15 <!-- Таблица истории -->
<div class="table-responsive d-flex justify-content-center">

```

```

20 <table class="table table-striped table-bordered table-hover
    w-75">
    <thead >
    <tr>
    <th class="text-center">Дата</th>
    <th class="text-center">Исходный текст</th>
    <th class="text-center">SQL</th>
    </tr>
    </thead>
25 <tbody>
    <tr th:each="h : ${history}">
    <td class="text-center" th:text="${h.getFormattedDate()}"></td>
    <td class="text-center" th:text="${h.text}"></td>
    <td class="text-center" th:text="${h.sql}"></td>
30 </tr>
    <tr th:if="${history.empty}">
    <td colspan="3" class="text-center text-muted">Нет записей</td>
    </tr>
    </tbody>
35 </table>
    </div>
    </div>
    </th:block>

```

main.html

```

5 <th:block th:fragment="bit()">
    <div class="d-flex row justify-content-center py-5 align-
        items-center">
    <h1 class="text-center mb-4">Генератор SQL-запросов</h1>

    <div class="form-section bg-white p-4 rounded shadow-sm mb-4">
        <h3 class="mb-3">Подключение к базе данных</h3>
10 <form id="dbConnectionForm">
    <div class="row g-3">
    <div class="col-md-6">
    <label for="dbHost" class="form-label">Хост</label>
    <input type="text" class="form-control" id="dbHost" value="
        localhost" required>
15 </div>
    <div class="col-md-3">

```

```

20 <label for="dbPort" class="form-label">Порт</label>
    <input type="number" class="form-control" id="dbPort" value=
        "5432" required>
    </div>
    <div class="col-md-3">
        <label for="dbType" class="form-label">Тип БД</label>
        <select class="form-select" id="dbType" required>
            <option value="postgresql" selected>PostgreSQL</option>
            <option value="mysql">MySQL</option>
25 <option value="oracle">Oracle</option>
        </select>
    </div>
    <div class="col-md-6">
        <label for="dbName" class="form-label">Имя базы данных</
            label>
30 <input type="text" class="form-control" id="dbName" required
        >
    </div>
    <div class="col-md-3">
        <label for="dbUsername" class="form-label">Пользователь</
            label>
        <input type="text" class="form-control" id="dbUsername"
            required>
35 </div>
    <div class="col-md-3">
        <label for="dbPassword" class="form-label">Пароль</label>
        <input type="password" class="form-control" id="dbPassword"
            required>
        </div>
40 <div class="col-12">
    <button type="submit" class="btn btn-primary">Подключиться</
        button>
    </div>
    </div>
    </form>
45 </div>

    <!-- Форма генерации SQL -->
    <div class="rounded mb-4 align-items-center d-flex row
        justify-content-center">
        <h3 class="mb-3">Генератор SQL</h3>
50 <form id="sqlGeneratorForm" class="w-100">
    <div class="mb-3">
        <label for="queryDescription" class="form-label">Опишите нуж
            ный запрос:</label>

```

```

55 <textarea class="form-control" id="queryDescription" rows="3
    "
    placeholder="Пример: Выбрать всех пользователей старше 25 ле
        т"></textarea>
</div>
<button type="submit" class="btn btn-success">
    Сгенерировать SQL
</button>
</form>
60 </div>

<!-- Редактор SQL -->
<div class="form-section bg-white p-4 rounded shadow-sm mb-4
    ">
<h3 class="mb-3">Редактор SQL</h3>
65 <div class="mb-3">
<textarea class="form-control font-monospace" id="sqlQuery"
    rows="8"
    placeholder="Здесь появится сгенерированный SQL или введите
        свой запрос"></textarea>
</div>
<div class="d-flex gap-2">
70 <button id="executeSqlBtn" class="btn btn-primary">
    Выполнить
</button>
</div>
</div>
75

<!-- Результаты запроса -->
<div class="form-section bg-white p-4 rounded shadow-sm" id=
    "resultsSection" style="display: none;">
<h3 class="mb-3">Результаты запроса</h3>
<div class="table-responsive">
80 <table class="table table-striped table-hover" id="
    resultsTable">
<thead>
<tr id="resultsHeader"></tr>
</thead>
<tbody id="resultsBody"></tbody>
85 </table>
</div>
<div class="alert alert-info mt-3 mb-0" id="resultsInfo"></
    div>
</div>
</div>

```

```

90 <!-- Скрипты -->
<script>
document.getElementById('dbConnectionForm').addEventListener
  ('submit', function(e) {
95     e.preventDefault();

    const dbHost = document.getElementById('dbHost').value;
    const dbPort = document.getElementById('dbPort').value;
    const dbType = document.getElementById('dbType').value;
    const dbName = document.getElementById('dbName').value;
100    const dbUsername = document.getElementById('dbUsername').
        value;
    const dbPassword = document.getElementById('dbPassword').
        value;

    fetch('/sql/connect', {
        method: 'POST',
105        headers: {
            'Content-Type': 'application/json',
            'X-Requested-With': 'XMLHttpRequest'
        },
        body: JSON.stringify({
110            dbHost, dbPort, dbType, dbName, dbUsername, dbPassword
        })
    })
    .then(response => response.json())
    .then(data => {
115        if (data.success) {
            alert('Успешное подключение к базе данных!');
        } else {
            alert('Ошибка подключения: ' + (data.error || 'Неизвестная ошибка'));
        }
120    })
    .catch(error => {
        console.error('Error:', error);
        alert('Произошла ошибка при подключении');
    });
125 });

document.getElementById('sqlGeneratorForm').addEventListener
  ('submit', function(e) {
    e.preventDefault();

```

```

130     const description = document.getElementById('
        queryDescription').value;
    if (!description.trim()) {
        alert('Пожалуйста, введите описание запроса');
        return;
    }

135     const btn = document.querySelector('#sqlGeneratorForm
        button[type="submit"]');
    btn.disabled = true;
    btn.innerHTML = '<span class="spinner-border spinner-
        border-sm" role="status"></span> Генерация...';

140     fetch('/sql/generate', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json',
            'X-Requested-With': 'XMLHttpRequest'
145     },
        body: JSON.stringify({
            text: description,
            dbHost: document.getElementById('dbHost').value,
            dbPort: parseInt(document.getElementById('dbPort').
                value),
150            dbType: document.getElementById('dbType').value,
            dbName: document.getElementById('dbName').value,
            dbUsername: document.getElementById('dbUsername').
                value,
            dbPassword: document.getElementById('dbPassword').
                value
        })
155    })
    .then(response => response.json())
    .then(data => {
        if (data.success) {
            document.getElementById('sqlQuery').value = data.sql;
160        } else {
            alert('Ошибка генерации: ' + (data.error || 'Неизвестн
                ая ошибка'));
        }
    })
    .catch(error => {
165        console.error('Error:', error);
        alert('Произошла ошибка при генерации SQL');
    })

```

```

    .finally(() => {
        btn.disabled = false;
170         btn.innerHTML = '<i class="bi bi-magic me-2"></i>Сгенери
            ровать SQL';
    });
});

document.getElementById('executeSqlBtn').addEventListener('
    click', function() {
175     const sql = document.getElementById('sqlQuery').value;
    if (!sql.trim()) {
        alert('Пожалуйста, введите или сгенерируйте SQL-запрос')
        ;
        return;
    }

180     // Ограничиваем ввод только для SELECT запросов
    if (!/^SELECT/i.test(sql)) {
        alert('Допустимы только SELECT запросы');
        return;
185     }

    const dbHost = document.getElementById('dbHost').value;
    const dbPort = document.getElementById('dbPort').value;
    const dbType = document.getElementById('dbType').value;
190     const dbName = document.getElementById('dbName').value;
    const dbUsername = document.getElementById('dbUsername').
        value;
    const dbPassword = document.getElementById('dbPassword').
        value;

    fetch('/sql/execute', {
195         method: 'POST',
        headers: {
            'Content-Type': 'application/json',
            'X-Requested-With': 'XMLHttpRequest'
        },
        body: JSON.stringify({
200             dbHost, dbPort, dbType, dbName, dbUsername, dbPassword
                , sqlQuery: sql
            })
        })
    .then(response => response.json())
205     .then(data => {
        if (data.success) {

```

```

        showResults(data.data);
    } else {
        alert('Ошибка выполнения запроса: ' + (data.error || '
            Неизвестная ошибка'));
    }
210  })
    .catch(error => {
        console.error('Error:', error);
        alert('Произошла ошибка при выполнении SQL-запроса');
215  });
});

function showResults(data) {
    const resultsSection = document.getElementById('
        resultsSection');
220  const resultsHeader = document.getElementById('
        resultsHeader');
    const resultsBody = document.getElementById('resultsBody')
        ;
    const resultsInfo = document.getElementById('resultsInfo')
        ;

    resultsHeader.innerHTML = '';
225  resultsBody.innerHTML = '';

    if (!data || data.length === 0) {
        resultsInfo.textContent = 'Запрос не вернул результатов
            ';
        resultsSection.style.display = 'block';
230  return;
    }

    // Заголовки таблицы
    Object.keys(data[0]).forEach(col => {
235  const th = document.createElement('th');
        th.textContent = col;
        resultsHeader.appendChild(th);
    });

    // Данные таблицы
240  data.forEach(row => {
        const tr = document.createElement('tr');
        Object.values(row).forEach(val => {
            const td = document.createElement('td');
245  td.textContent = val;

```

```

        tr.appendChild(td);
    });
    resultsBody.appendChild(tr);
});
250
    resultsInfo.textContent = 'Найдено записей: ${data.length}';
    resultsSection.style.display = 'block';
}
</script>
255 </th:block>

```

home.html

```

<!DOCTYPE html>
<html lang="en">
<head>
5 <meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-
    scale=1.0, maximum-scale=1.0, user-scalable=no">
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/
    dist/css/bootstrap.min.css" rel="stylesheet"
    integrity="sha384-EVSTQN3/
        azprG1Anm3QDgpJLIm9Nao0Yz1ztcQTwFspd3yD65VohhpuuCOmLASjC"
        crossorigin="anonymous">
<title>Krolik</title>
10 </head>

<body class="" style="height: 100vh; overflow-x: hidden">
<div class="d-flex justify-content-center w-100">
<a class="btn btn-primary me-3 mt-3" th:href="@{/}" style="
    width: 15%">Генератор SQL</a>
15 <a class="btn btn-primary me-3 mt-3" th:href="@{/history}"
    style="width: 15%">История запросов</a>
<form th:action="@{/logout}" method="post">
<button type="submit" class="btn btn-primary me-3 mt-3" >Вый
    ти</button>
</form>
</div>
20 <th:block th:switch="${appState}">
<th:block th:case="'main'">
<div th:replace="~{main/main :: bit()}"></div>
</th:block>
<th:block th:case="'history'">
25 <div th:replace="~{main/history :: bit()}"></div>
</th:block>

```

```
</th:block>  
</body>  
</html>
```